

Arquitectura de Microservicios y Mapa de Dominios

Santo Pegasus Soluciones

Documento: Arquitectura de Microservicios y Mapa de Dominios

Versión: 1.0.0

Fecha de Emisión: Junio 2026

Departamento: Ingeniería de Software / Chapter de Back-end

Clasificación: Interno — Uso Técnico Restringido

TABLA DE CONTENIDOS

1. Introducción y Visión General de la Arquitectura
2. Diagrama Textual de la Arquitectura General
3. Catálogo Completo de Microservicios
4. Mapa de Dependencias entre Servicios
5. Patrones de Comunicación
6. Estrategia de Bases de Datos
7. API Gateway
8. Infraestructura en AWS
9. Observabilidad Distribuida
10. Estrategia de Versionado de APIs
11. Seguridad entre Servicios
12. Mapa de Squads y Ownership
13. Roadmap Técnico
14. Architecture Decision Records (ADRs)
15. Disposiciones Finales y Proceso de Actualización

SECCIÓN 1 — INTRODUCCIÓN Y VISIÓN GENERAL DE LA ARQUITECTURA

1.1 Propósito del Documento

Este documento constituye el mapa arquitectónico oficial de Santo Pegasus Soluciones. Su propósito es servir como referencia canónica para todos los ingenieros, Tech Leads, Product Managers y stakeholders técnicos que necesiten comprender cómo están organizados, interconectados y desplegados los sistemas que componen el portafolio de productos de la empresa.

Este artefacto debe leerse en conjunto con la Guía Oficial de Ingeniería Back-end (v2.4.0), que establece los estándares de codificación, patrones de diseño y prácticas de seguridad obligatorias para todos los servicios aquí descritos. Ambos documentos forman el núcleo de la base de conocimiento técnico de Santo Pegasus y son fuentes de verdad para la toma de decisiones arquitectónicas.

1.2 Contexto Empresarial

Santo Pegasus Soluciones es una empresa de tecnología especializada en el desarrollo de productos digitales para el sector de salud y servicios profesionales. El producto principal de la compañía es Agendio, una plataforma SaaS de agendamiento de consultas médicas que conecta pacientes, médicos y clínicas a través de una experiencia digital integrada.

La plataforma Agendio opera en modelo multi-tenant, atendiendo a redes de clínicas, hospitales de pequeño y mediano porte, y consultorios independientes en todo Brasil. La criticidad del dominio de salud exige que la arquitectura priorice disponibilidad, seguridad de datos sensibles, conformidad con la LGPD (Lei Geral de Proteção de Dados) y auditabilidad completa de todas las operaciones.

1.3 Por Qué Microservicios

La adopción de una arquitectura de microservicios en Santo Pegasus no fue una decisión tomada de forma prematura. La empresa comenzó con un monolito funcional, y la migración para microservicios fue guiada por necesidades reales e inmediatas de escala, autonomía de equipos y velocidad de entrega, en consonancia con los principios fundamentales documentados en la Guía de Ingeniería Back-end.

Las motivaciones concretas que justificaron la transición son:

- **Escalabilidad Independiente:** El servicio de agendamiento (`agendio-scheduling-service`) presenta picos de carga en horarios matutinos (entre 07h00 y 09h00) que no impactan otros dominios. En arquitectura monolítica, escalar ese módulo implicaría escalar toda la aplicación, desperdiciando recursos computacionales y aumentando costos operativos.
- **Autonomía de Squads:** Con equipos organizados por dominio de negocio, la arquitectura de microservicios permite que cada squad tenga ownership completo de su ciclo de vida de software — desde el código hasta el deploy en producción — sin coordinación burocrática con otros equipos para releases.

- **Resiliencia e Isolamento de Fallas:** Un defecto crítico en el ``payment-service`` no debe causar la indisponibilidad del agendamiento de consultas. El aislamiento de procesos garantiza que las fallas se contengan dentro del dominio afectado.
- **Flexibilidad Tecnológica:** Diferentes dominios tienen diferentes requisitos técnicos. El ``medical-records-service`` se beneficia de MongoDB para el almacenamiento de documentos clínicos flexibles y semi-estructurados, mientras que el ``auth-service`` requiere la consistencia fuerte de PostgreSQL para la gestión de credenciales.
- **Velocidad de Entrega:** La independencia de deploys permite que múltiples squads liberen nuevas funcionalidades en el mismo día sin coordinación de releases ni ventanas de mantenimiento compartidas.

1.4 Principios Arquitectónicos que Guían las Decisiones

Todas las decisiones arquitectónicas en Santo Pegasus están ancladas en los siguientes principios, que deben ser internalizados por todo el equipo de ingeniería:

#	Principio	Descripción
P-01	High Cohesion, Low Coupling	Cada microservicio debe encapsular un dominio de negocio bien definido. Dependencias entre servicios deben ser minimizadas y, cuando existentes, preferiblemente asíncronas.
P-02	Database per Service	Cada microservicio posee y controla exclusivamente su propia base de datos. El acceso directo a la base de datos de otro servicio está terminantemente prohibido.
P-03	API First	Contratos de API (OpenAPI/Swagger) son definidos antes de la implementación. Esto garantiza que los consumidores puedan desarrollar en paralelo usando mocks.
P-04	Security by Default	Todas las comunicaciones entre servicios son autenticadas. Ningún endpoint interno es accesible sin validación de identidad. mTLS es obligatorio para comunicación interna.
P-05	Observability by Design	Logs estructurados, métricas y rastreo distribuido no son añadidos post-hoc; son parte del scaffolding inicial de cada nuevo servicio.
P-06	Fail Fast, Recover Gracefully	Los servicios implementan circuit breakers, timeouts y políticas de retry con backoff exponencial para garantizar resiliencia ante fallas de dependencias.

P-07	Compliance First	Decisiones sobre almacenamiento, transmisión y procesamiento de datos sensibles de salud son guiadas por la LGPD y buenas prácticas de seguridad antes de cualquier requisito funcional.
P-08	Infrastructure as Code	Toda la infraestructura AWS es definida como código (Terraform/AWS CDK). Ningún recurso de producción es creado manualmente via Console.
P-09	Evolutionary Architecture	La arquitectura acepta que cambiará. Decisiones son tomadas con horizontes de tiempo claros y revisadas periódicamente en los foros de Architecture Review.
P-10	SOLID in Every Service	Los principios SOLID de orientación a objetos son aplicados tanto a nivel de clase como a nivel de servicio. Cada microservicio tiene una única responsabilidad de dominio.

1.5 Ecosistema Tecnológico Principal

El ecosistema tecnológico de Santo Pegasus está standardizado para garantizar consistencia operacional y reducir la carga cognitiva de los equipos:

- **Lenguaje & Runtime:** Java 17+ (LTS), con adopción gradual de Java 21 (Virtual Threads)
- **Framework Principal:** Spring Boot 3+, Spring Security, Spring Cloud
- **Contenedorización:** Docker (imágenes inmutables), orquestados en AWS ECS Fargate
- **Bases de Datos:** PostgreSQL (relacional), MongoDB/AWS DocumentDB (documentos), Redis/AWS ElastiCache (caché)
- **Mensajería:** AWS SQS (colas y eventos entre dominios)
- **Autenticación:** JWT + OAuth 2.0 / OpenID Connect, gestionado por `auth-service`
- **Observabilidad:** SLF4J + Logback, Spring Boot Actuator, Micrometer, Prometheus, Datadog
- **CI/CD:** GitHub Actions + Pipelines con Docker, Flyway/Liquibase para migrations
- **Secrets Management:** AWS Secrets Manager + Spring Cloud Config

SECCIÓN 2 — DIAGRAMA TEXTUAL DE LA ARQUITECTURA GENERAL

2.1 Visión de Alto Nivel

El diagrama a continuación representa la topología completa de la arquitectura de microservicios de Santo Pegasus, mostrando el flujo de tráfico desde los clientes externos hasta los servicios internos, bases de datos y sistemas de mensajería.

...

CLIENTES EXTERNOS

[Web App React] [Mobile App iOS/Android] [Clínicas API Partners]

▼ HTTPS / TLS 1.3

AWS API GATEWAY (Kong / AWS API GW)

- Routing
- Rate Limiting
- Auth Token Validation
- SSL Termination
- CORS
- Request Logging



auth- user- agendio- payment- ai-assistant-
service service scheduling service service
:8081 :8082 -service :8085 :8087
:8083

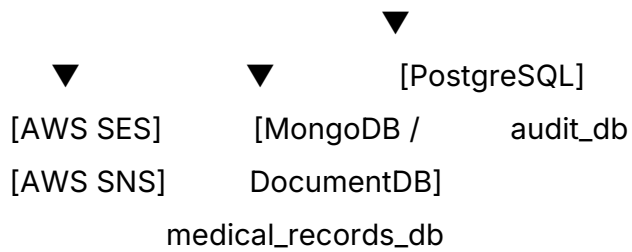


[PostgreSQL] [PostgreSQL] [PostgreSQL] [Pinecone DB]
auth_db users_db payments_db (Vector Store)



agendio- medical- audit-
notif- records- service
service service :8088

:8086



CAPA DE MENSAJERÍA ASÍNCRONA

AWS SQS QUEUES

```
appointment-created.fifo → [agendio-notification-service]
appointment-cancelled.fifo → [agendio-notification-service]
                             → [audit-service]
payment-confirmed.fifo → [agendio-scheduling-service]
                        → [audit-service]
user-created.fifo      → [agendio-notification-service]
                        → [audit-service]
audit-events.fifo      → [audit-service]
```

INFRAESTRUCTURA COMPARTIDA

AWS	AWS	AWS Secrets
ElastiCache	CloudWatch	Manager
(Redis)	+ Datadog	+ Spring Cloud
Caché Global	Observability	Config

...

2.2 Flujo de una Requisición Típica (Agendamiento de Consulta)

...

Cliente (App Mobile)

POST /v1/appointments [Bearer JWT]



API Gateway

- Valida JWT via auth-service (cache Redis, TTL 5min)
- Aplica rate limit (100 req/min por tenant)
- Routea para agendio-scheduling-service



agendio-scheduling-service

- Valida disponibilidad del médico (consulta propia DB)
- Verifica perfil del paciente via user-service (REST síncrono)
- Crea registro de consulta en scheduling_db (PostgreSQL)
- Publica evento "appointment-created" en SQS
- Retorna 201 Created con payload de la consulta



[Async] agendio-notification-service consume SQS

- Envía email de confirmación via AWS SES
- Envía SMS via AWS SNS



[Async] audit-service consume SQS

- Registra acción en audit_db con Trace ID completo

...

SECCIÓN 3 — CATÁLOGO COMPLETO DE MICROSERVICIOS

3.1 Resumen del Catálogo

Servicio	Puerto	Dominio	Base de Datos	Squad Owner
`auth-service`	8081	Identidad & Seguridad	PostgreSQL	Squad Hermes
`user-service`	8082	Usuarios & Perfiles	PostgreSQL	Squad Hermes
`agendio-scheduling-service`	8083	Agendamiento (Core)	PostgreSQL	Squad Agendio Core
`agendio-notification-service`	8084	Notificaciones	— (stateless)	Squad Agendio Core
`payment-service`	8085	Pagos & Facturación	PostgreSQL	Squad Pagamentos
`medical-records-service`	8086	Historial Clínico	MongoDB	Squad Clínico

`ai-assistant-service`	8087	Asistencia IA (RAG)	Pinecone (Vector)	Squad IA
`audit-service`	8088	Auditoría & Compliance	PostgreSQL	Squad Governance

3.2 `auth-service` — Autenticación y Autorización

Repositorio Git: `git@github.com:santopegasus/auth-service.git`

Squad Owner: Squad Hermes

Tech Lead: Isabella Carvalho

Puerto: 8081

Responsabilidad

El `auth-service` es la raíz de confianza del ecosistema de Santo Pegasus. Es responsable de la emisión, validación y revocación de tokens JWT, de la gestión del ciclo de vida de credenciales de usuarios, y de la integración con el flujo OAuth 2.0 / OpenID Connect para clientes de terceros. Ningún otro servicio valida credenciales directamente; toda autenticación pasa obligatoriamente por este servicio.

Tecnologías

- **Framework:** Spring Boot 3.x + Spring Security 6
- **Autenticación:** JWT (RS256), OAuth 2.0, OpenID Connect (OIDC)
- **Base de Datos:** PostgreSQL 15 (`auth\_db`) via AWS RDS
- **Caché de Tokens:** AWS ElastiCache (Redis) — TTL configurable por tipo de token
- **Secrets:** AWS Secrets Manager para llaves RSA privadas

APIs Expuestas

Método	Endpoint	Descripción
`POST`	`/v1/auth/login`	Autenticación con email/password, retorna JWT + Refresh Token
`POST`	`/v1/auth/refresh`	Renovación de token via Refresh Token
`POST`	`/v1/auth/logout`	Revocación de token (blacklist en Redis)
`POST`	`/v1/auth/oauth2/token`	Flujo OAuth 2.0 Client Credentials
`GET`	`/v1/auth/validate`	Validación de token (usado por API Gateway)
`POST`	`/v1/auth/password/reset`	Solicitud de reset de contraseña

`GET`	`/.well-known/jwks.json`	JWKS endpoint para validación pública de tokens
-------	--------------------------	---

Dependencias con Otros Servicios

- **`user-service` (REST síncrono):** Para validar existencia y status del usuario durante el login
- **`audit-service` (SQS asíncrono):** Publica eventos de **`auth.login.success`**, **`auth.login.failure`**, **`auth.token.revoked`**

Modelo de Datos (auth_db)

...

users_credentials → id, user_id, password_hash, is_active, mfa_secret

refresh_tokens → id, token_hash, user_id, expires_at, is_revoked

roles → id, name, permissions (JSONB)

user_roles → user_id, role_id

oauth_clients → client_id, client_secret_hash, scopes, redirect_uris

...

3.3 `user-service` — Gestión de Perfiles de Usuarios

Repositorio Git: ``git@github.com:santopegasus/user-service.git``

Squad Owner: Squad Hermes

Tech Lead: Isabella Carvalho

Puerto: 8082

Responsabilidad

Gestiona los perfiles de todos los actores del sistema: pacientes, médicos, administradores de clínicas y operadores. Es la fuente de verdad para datos de perfil, como nombre, contacto, especialidades médicas (para el rol Doctor), y configuraciones de cuenta. No almacena credenciales de acceso — esa responsabilidad es exclusiva del **`auth-service`**.

Tecnologías

- **Framework:** Spring Boot 3.x
- **Base de Datos:** PostgreSQL 15 (**`users\_db`**) via AWS RDS
- **Caché:** Redis para cachear perfiles frecuentemente accedidos (TTL 10min)
- **Mapeo:** MapStruct para conversión Entidad → DTO
- **Documentación:** springdoc-openapi (Swagger UI)

APIs Expuestas

Método	Endpoint	Descripción
`POST`	<code>`/v1/users`</code>	Creación de nuevo usuario
`GET`	<code>`/v1/users/{id}`</code>	Consulta de perfil por ID

`PUT`	`/v1/users/{id}`	Actualización completa de perfil
`PATCH`	`/v1/users/{id}/status`	Activación / desactivación de cuenta
`GET`	`/v1/doctors`	Listado de médicos con filtros (especialidad, cidade)
`GET`	`/v1/doctors/{id}/availability`	Disponibilidad del médico (proxy para scheduling)
`GET`	`/v1/users/{id}/preferences`	Preferencias de notificación del usuario

Dependencias

- **`audit-service` (SQS asíncrono):** Eventos de ``user.created``, ``user.updated``, ``user.deactivated``
- **`agendio-notification-service` (SQS asíncrono):** Evento ``user.created`` para disparo de email de bienvenida

Modelo de Datos (users_db)

...

users → id (UUID), name, email, phone, cpf_hash, role, status, created_at

doctors → user_id (FK), crm, specialty, bio, consultation_price

patients → user_id (FK), birth_date, health_plan, emergency_contact

addresses → id, user_id (FK), street, city, state, zip_code

preferences → user_id (FK), notify_email, notify_sms, language

...

3.4 `agendio-scheduling-service` — Núcleo de Agendamiento

Repositorio Git: ``git@github.com:santopegasus/agendio-scheduling-service.git``

Squad Owner: Squad Agendio Core

Tech Lead: Rodrigo Mendes

Puerto: 8083

Responsabilidad

Es el servicio más crítico del ecosistema. Gestiona toda la lógica de negocio de agendamiento de consultas médicas: creación, confirmación, cancelación y reagendamiento de consultas; gestión de la agenda de disponibilidad de médicos; control de conflictos de horarios; y reglas de negocio específicas de cada clínica (como políticas de cancelación y límites de consultas por paciente). Ninguna regla de agendamiento reside fuera de este servicio.

Tecnologías

- **Framework:** Spring Boot 3.x + Spring Data JPA

- **Base de Datos:** PostgreSQL 15 (`scheduling\_db`) via AWS RDS — transacciones ACID críticas
- **Migrations:** Flyway — versionado de schema en Git junto al código
- **Mensajería:** AWS SQS Producer (publica eventos de ciclo de vida de consultas)
- **Circuit Breaker:** Resilience4j para llamadas a `user-service` y `payment-service`

APIs Expuestas

Método	Endpoint	Descripción
`POST`	`/v1/appointments`	Crear nuevo agendamiento
`GET`	`/v1/appointments/{id}`	Consultar agendamiento por ID
`PATCH`	`/v1/appointments/{id}/cancel`	Cancelar consulta
`PATCH`	`/v1/appointments/{id}/reschedule`	Reagendar consulta
`GET`	`/v1/appointments/patient/{patientId}`	Historial de consultas del paciente
`GET`	`/v1/schedules/doctor/{doctorId}`	Agenda de disponibilidad del médico
`POST`	`/v1/schedules/doctor/{doctorId}/slots`	Configurar slots de disponibilidad
`PUT`	`/v1/schedules/doctor/{doctorId}/block`	Bloquear período (vacaciones, ausencias)

Dependencias

- **`user-service` (REST síncrono, Resilience4j):** Validar paciente y médico existentes
- **`payment-service` (REST síncrono):** Verificar status de pago antes de confirmar consulta
- **`audit-service` (SQS):** Eventos de toda alteración de estado de consulta
- **`agendio-notification-service` (SQS):** Publicar eventos para disparo de notificaciones

Eventos SQS Publicados

...

- appointment-created.fifo
- appointment-confirmed.fifo
- appointment-cancelled.fifo
- appointment-rescheduled.fifo
- appointment-reminder-due.fifo (publicado por scheduler interno, T-24h e T-1h)

...

Modelo de Datos (scheduling_db)

...

appointments → id (UUID), patient_id, doctor_id, clinic_id, status,
scheduled_at, duration_min, type, notes, created_at
schedule_slots → id, doctor_id, day_of_week, start_time, end_time, is_available
blocked_periods → id, doctor_id, start_date, end_date, reason
clinics → id, name, cnpj, address, max_appointments_per_day
...

3.5 `agendio-notification-service` — Notificaciones

Repositorio Git: ``git@github.com:santopegasus/agendio-notification-service.git``

Squad Owner: Squad Agendio Core

Tech Lead: Rodrigo Mendes

Puerto: 8084

Responsabilidad

Servicio stateless responsable de orquestar el envío de todas las notificaciones de la plataforma Agendio: confirmaciones de consulta, recordatorios automáticos, alertas de cancelación y mensajes de bienvenida. Consume eventos de SQS y decide el canal de entrega (email via AWS SES o SMS via AWS SNS) basado en las preferencias del usuario (consultadas en `user-service`). No posee base de datos propia; su estado es integralmente derivado de los eventos recibidos.

Tecnologías

- **Framework:** Spring Boot 3.x + Spring Cloud AWS
- **Email:** AWS SES (Simple Email Service) con templates HTML/Thymeleaf
- **SMS:** AWS SNS (Simple Notification Service)
- **Mensajería:** AWS SQS Consumer (escucha múltiples colas)
- **Stateless:** Sin base de datos propia — idempotencia garantizada via SQS deduplication ID

Dependencias

- **`user-service` (REST síncrono):** Consultar preferencias de notificación del usuario
- **AWS SES:** Para envío de emails transaccionales
- **AWS SNS:** Para envío de SMS
- **SQS (Consumer):** Consume eventos de `agendio-scheduling-service` y `user-service`

Templates de Notificación Gestionados

Evento	Canal	Template
`appointment-created`	Email + SMS	Confirmación con código e detalhes
`appointment-reminder-due (T-24h)`	Email + SMS	Recordatorio 24h antes
`appointment-reminder-due (T-1h)`	SMS	Recordatorio 1h antes

`appointment-cancelled`	Email	Confirmación de cancelamiento
`appointment-rescheduled`	Email + SMS	Nuevo horario confirmado
`user-created`	Email	Bienvenida a la plataforma
`password-reset-requested`	Email	Link de reset (expira en 30min)

3.6 `payment-service` — Procesamiento de Pagos

Repositorio Git: `git@github.com:santopegasus/payment-service.git`

Squad Owner: Squad Pagamentos

Tech Lead: Lucas Andrade

Puerto: 8085

Responsabilidad

Gestiona el ciclo de vida completo de transacciones financieras de la plataforma: procesamiento de pagos via Pix y cartão de crédito/débito, integración con el gateway de pagamento externo (Stripe), gestión de estornos y reembolsos, y generación de recibos. Toda operación financiera es idempotente y completamente auditada. El servicio no almacena datos de cartão en sus propias bases de datos — el tokenizado es responsabilidad del gateway externo (PCI-DSS compliance).

Tecnologías

- **Framework:** Spring Boot 3.x
- **Base de Datos:** PostgreSQL 15 (`payments\_db`) via AWS RDS — ACID para consistencia financiera
- **Gateway Externo:** Stripe API (tarjeta) + Banco Central API (Pix via PSP parceiro)
- **Idempotencia:** Chave de idempotência obrigatória em todos endpoints de criação
- **Webhooks:** Endpoint seguro para receber callbacks do gateway (validação de assinatura HMAC)

APIs Expuestas

Método	Endpoint	Descripción
`POST`	`/v1/payments`	Iniciar transacción de pago
`GET`	`/v1/payments/{id}`	Consultar status de pago
`POST`	`/v1/payments/{id}/refund`	Solicitar reembolso
`GET`	`/v1/payments/appointment/{appointmentId}`	Pago vinculado a una consulta
`POST`	`/v1/webhooks/stripe`	Recibir eventos del gateway Stripe

`POST`	`/v1/webhooks/pix`	Recibir confirmaciones de Pix
--------	--------------------	-------------------------------

Dependencias

- **`audit-service` (SQS):** Todo evento de transacción financiera es publicado para auditoría
- **`agendio-scheduling-service` (SQS Consumer):** Recibe `payment-confirmed` para confirmar consulta
- **Stripe API (REST externo):** Gateway de cartão
- **PSP Parceiro (REST externo):** Gateway Pix

Modelo de Datos (payments_db)

...

transactions → id (UUID), appointment_id, patient_id, amount, currency, status, method, gateway_tx_id, idempotency_key, created_at

refunds → id, transaction_id (FK), amount, reason, status, created_at

payment_methods → id, patient_id, type (PIX/CARD), stripe_token, last4, is_default

receipts → id, transaction_id (FK), pdf_url, issued_at

...

3.7 `medical-records-service` — Historial Clínico (LGPD Compliance)

Repositorio Git: `git@github.com:santopegasus/medical-records-service.git`

Squad Owner: Squad Clínico

Tech Lead: Ana Paula Fonseca

Puerto: 8086

Responsabilidad

El servicio más sensible del ecosistema desde la perspectiva de protección de datos. Gestiona el historial clínico de los pacientes: anamneses, diagnósticos, prescripciones, resultados de exámenes y archivos adjuntos (PDFs, imágenes). El acceso a los datos clínicos es estrictamente controlado por Role-Based Access Control (RBAC) y toda consulta a registros es auditada. El diseño del servicio cumple integralmente con los requisitos de la LGPD: consentimiento explícito del paciente es requerido para compartición de datos, y los registros son anonimizables por solicitud.

Tecnologías

- **Framework:** Spring Boot 3.x + Spring Security (RBAC granular)
- **Base de Datos:** MongoDB via AWS DocumentDB (`medical\_records\_db`) — esquema flexible para evolución de formularios clínicos
- **Almacenamiento de Archivos:** AWS S3 (exámenes em PDF, imagens DICOM) con URLs pre-firmadas de duración limitada (15 minutos)

- **Criptografía:** Campos sensibles (diagnósticos, anotaciones del médico) encriptados en reposo con AWS KMS
- **Anonimización:** Pipeline de anonimización basado en requisición formal, ejecutado como job asíncrono

Reglas de Acceso RBAC

Rol	Permiso
`ROLE_PATIENT`	Leer sus propios registros únicamente
`ROLE_DOCTOR`	Leer y escribir registros de sus pacientes activos
`ROLE_ADMIN_CLINIC`	Leer (sin datos sensibles) registros de su clínica
`ROLE_AUDITOR`	Leer metadatos de acceso (sin contenido clínico)
`ROLE_SYSTEM`	Acceso de sistema para integraciones internas

APIs Expuestas

Método	Endpoint	Descripción
`POST`	`/v1/records`	Crear registro clínico (solo DOCTOR)
`GET`	`/v1/records/patient/{patientId}`	Listar historial del paciente
`GET`	`/v1/records/{id}`	Consultar registro específico
`PUT`	`/v1/records/{id}`	Actualizar registro (solo DOCTOR propietario)
`POST`	`/v1/records/{id}/attachments`	Adjuntar archivo (PDF/imagen)
`GET`	`/v1/records/{id}/attachments/{fileId}`	URL pre-firmada S3 (TTL 15min)
`POST`	`/v1/records/patient/{patientId}/consent`	Registrar consentimiento de compartición
`POST`	`/v1/records/patient/{patientId}/anonymize`	Solicitar anonimización (LGPD Art. 18)

Dependencias

- **`auth-service` (JWT validation):** Extrae roles del token para RBAC
- **`audit-service` (SQS):** Todo acceso a registros clínicos genera evento de auditoría
- **AWS S3:** Almacenamiento de archivos adjuntos
- **AWS KMS:** Gestión de llaves de encriptación en reposo

3.8 `ai-assistant-service` — Asistente de IA con Arquitectura RAG

Repositorio Git: ``git@github.com:santopegasus/ai-assistant-service.git``

Squad Owner: Squad IA

Tech Lead: Fernanda Rocha

Puerto: 8087

Responsabilidad

Provee capacidades de inteligencia artificial a la plataforma Agendio. La funcionalidad principal es un asistente de triaje inteligente que ayuda al paciente a identificar la especialidad médica más adecuada para sus síntomas, basado en una arquitectura RAG (Retrieval-Augmented Generation). El servicio usa LangChain4j para la orquestación del flujo de IA, Pinecone como vector database para el store de conocimiento médico indexado, y un LLM externo (OpenAI GPT-4o) como modelo generativo. Ninguna credencial de la API de IA es hardcodeda en el código; todas las keys son inyectadas via AWS Secrets Manager en tiempo de ejecución.

Tecnologías

- Framework: Spring Boot 3.x + LangChain4j
- LLM: OpenAI GPT-4o (via API externa)
- Vector Store: Pinecone (embeddings de conocimiento médico)
- Embeddings: OpenAI text-embedding-3-large
- Caché de Respuestas: Redis (TTL 1h para consultas frecuentes / idénticas)
- Rate Limiting Interno: Límite de 50 consultas/paciente/día para control de costos

APIs Expuestas

Método	Endpoint	Descripción
`POST`	<code>`/v1/ai/triage`</code>	Triagem de sintomas, sugere especialidade
`POST`	<code>`/v1/ai/chat`</code>	Chat assistido com histórico de contexto
`GET`	<code>`/v1/ai/chat/{sessionId}/history`</code>	Histórico de conversa da sessão
`POST`	<code>`/v1/ai/knowledge/index`</code>	Indexar novo documento médico (Admin only)
`DELETE`	<code>`/v1/ai/chat/{sessionId}`</code>	Encerrar e excluir sessão (LGPD)

Flujo RAG Interno

...

1. Paciente envía pregunta sobre síntomas
2. ai-assistant-service genera embedding de la pregunta
3. Consulta Pinecone para recuperar top-K chunks de documentos médicos relevantes

- 4. Construye prompt enriquecido (pregunta + contexto recuperado)
 - 5. Envía prompt al LLM (GPT-4o)
 - 6. Retorna respuesta + referencias a las fuentes usadas
 - 7. Registra interacción (anonimizada) para mejora futura del modelo
- ...

Dependencias

- ``user-service` (REST)`: Verificar perfil del paciente antes de la triagem
 - ``audit-service` (SQS)`: Eventos de uso de IA para compliance
 - `OpenAI API (externo)`: LLM y embeddings
 - `Pinecone API (externo)`: Vector database
 - `Redis`: Caché de respuestas y sesiones de chat
-

3.9 ``audit-service`` — Registro de Auditoría y Compliance

Repositorio Git: ``git@github.com:santopegasus/audit-service.git``

Squad Owner: Squad Governance

Tech Lead: Marco Antônio Lima

Puerto: 8088

Responsabilidad

El ``audit-service`` es el sistema de registro histórico inmutable de todas las acciones sensibles del ecosistema de Santo Pegasus. Es consumidor puro — no expone APIs de escritura a otros servicios; recibe todos sus datos exclusivamente via mensajería SQS. Provee APIs de consulta para auditores internos y para geração de relatórios de compliance. Los registros de auditoría jamás son modificados o eliminados; únicamente son archivados en AWS S3 Glacier tras 90 días para retención de largo plazo.

Tecnologías

- **Framework:** Spring Boot 3.x
- **Base de Datos:** PostgreSQL 15 (``audit_db``) via AWS RDS — immutable append-only
- **Archivado:** AWS S3 Glacier para retención de largo plazo (regulatory compliance)
- **Mensajería:** AWS SQS Consumer exclusivo (múltiples colas)
- **Búsqueda:** Índices PostgreSQL optimizados para queries por Trace ID, user_id, timestamp

APIs Expuestas (Solo Lectura — Roles Restringidos)

Método	Endpoint	Rol Requerido
<code>`GET`</code>	<code>`/v1/audit/events`</code>	<code>`ROLE_AUDITOR`</code> , <code>`ROLE_ADMIN`</code>
<code>`GET`</code>	<code>`/v1/audit/events/{traceId}`</code>	<code>`ROLE_AUDITOR`</code>
<code>`GET`</code>	<code>`/v1/audit/events/user/{userId}`</code>	<code>`ROLE_AUDITOR`</code>

`GET`	`/v1/audit/report/monthly`	`ROLE_ADMIN`
`GET`	`/v1/audit/events/service/{serviceName}`	`ROLE_AUDITOR`

Modelo de Datos (audit_db)

...

audit_events → id (UUID), trace_id, span_id, service_name, action,
actor_id, actor_role, resource_type, resource_id,
payload_summary (NO dados sensíveis), timestamp, ip_address

...

SECCIÓN 4 — MAPA DE DEPENDENCIAS ENTRE SERVICIOS

4.1 Tabla de Dependencias

Servicio Consumidor	Servicio Proveedor	Tipo	Protocolo	Criticidad
`api-gateway`	`auth-service`	Síncrono	REST / Redis Cache	Alta
`agendiao-scheduling-service`	`user-service`	Síncrono	REST (WebClient)	Alta
`agendiao-scheduling-service`	`payment-service`	Síncrono	REST (WebClient)	Alta
`payment-service`	`audit-service`	Asíncrono	SQS	Média
`agendiao-scheduling-service`	`agendiao-notification-service`	Asíncrono	SQS	Média
`agendiao-scheduling-service`	`audit-service`	Asíncrono	SQS	Média
`user-service`	`agendiao-notification-service`	Asíncrono	SQS	Baixa
`user-service`	`audit-service`	Asíncrono	SQS	Média
`auth-service`	`user-service`	Síncrono	REST (WebClient)	Alta
`auth-service`	`audit-service`	Asíncrono	SQS	Média
`medical-records-service`	`audit-service`	Asíncrono	SQS	Alta
`ai-assistant-service`	`user-service`	Síncrono	REST	Baixa

SECCIÓN 5 — PATRONES DE COMUNICACIÓN

5.1 REST Síncrono con WebClient

Para comunicaciones síncronas entre servicios, Santo Pegasus adopta el uso del Spring WebClient (no-bloqueante, reactivo), disponible en el módulo `spring-webflux`. La elección de WebClient sobre el deprecated RestTemplate se justifica por su mejor uso de threads y soporte nativo a operaciones non-blocking.

Reglas obligatorias para comunicación REST síncrona:

- Todos los clients HTTP entre servicios utilizan mTLS para autenticación mútua
- Timeout de conexión: 3 segundos (configurable por variable de entorno)
- Timeout de lectura: 10 segundos
- Circuit Breaker via Resilience4j en todas las llamadas a servicios externos
- Retry con backoff exponencial (max 3 tentativas, jitter de 500ms)
- Las respuestas de error deben seguir RFC 7807 (Problem Details)

```
```java
// Ejemplo canónico de WebClient con Resilience4j
@Service
public class UserServiceClient {
 private final WebClient webClient;
 private final CircuitBreaker circuitBreaker;
 public Mono<UserProfileDTO> getUserProfile(UUID userId) {
 return circuitBreaker.executeSupplier(() ->
 webClient.get()
 .uri("/v1/users/{id}", userId)
 .retrieve()
 .onStatus(HttpStatusCode::is4xxClientError, this::handleClientError)
 .bodyToMono(UserProfileDTO.class)
 .timeout(Duration.ofSeconds(10))
);
 }
}
```
```

5.2 gRPC para Alta Performance

Para comunicaciones de alta frecuencia y baja latencia entre servicios internos que no requieren pasar por el API Gateway — como el `agendio-scheduling-service` consultando disponibilidad en tiempo real — Santo Pegasus adopta gRPC como protocolo de comunicação.

Serviços que implementam gRPC (atualmente):

- `agendiao-scheduling-service` → `user-service` (endpoint de validação rápida de médico)
- `auth-service` → `user-service` (endpoint de verificação de status em login)

Vantagens adotadas:

- Protocol Buffers (protobuf) garantem payload menor e serialização mais rápida que JSON
- HTTP/2 multiplexing reduz latência em chamadas concorrentes
- Contrato fortemente tipado via `.proto` files versionados em repositório compartilhado

```
```protobuf
```

```
// user-service.proto (exemplo)
```

```
syntax = "proto3";
```

```
service UserService {
```

```
 rpc ValidateUser (ValidateUserRequest) returns (ValidateUserResponse);
```

```
}
```

```
message ValidateUserRequest {
```

```
 string user_id = 1;
```

```
}
```

```
message ValidateUserResponse {
```

```
 bool is_active = 1;
```

```
 string role = 2;
```

```
 string full_name = 3;
```

```
}
```

```
```
```

5.3 Mensageria Assíncrona com AWS SQS

Para comunicação entre domínios (cross-domain events), adotamos AWS SQS FIFO Queues para garantir ordenação e exactly-once delivery. A mensageria assíncrona é o padrão preferido para eventos que não requerem resposta imediata, desacoplando os produtores dos consumidores.

Padrões obrigatórios para SQS:

- Filas FIFO para garantir ordenação e deduplicação (`MessageDeduplicationId` obrigatório)
- Dead Letter Queues (DLQ) configuradas com `maxReceiveCount = 3` para todas as filas principais
- Idempotência no lado consumidor: toda operação deve ser segura para reprocessamento
- Envelope de evento padronizado:

```
```json
```

```
{
```

```
 "eventId": "uuid-v4",
```

```
"eventType": "appointment.created",
"source": "agendio-scheduling-service",
"traceId": "propagado-do-header-http",
"timestamp": "2026-06-16T10:30:00Z",
"schemaVersion": "1.0",
"payload": {
 "appointmentId": "uuid",
 "patientId": "uuid",
 "doctorId": "uuid",
 "scheduledAt": "2026-06-20T14:00:00Z"
}
}
...

```

## SECCIÓN 6 — ESTRATEGIA DE BASES DE DATOS

### 6.1 Database per Service Pattern

Cada microservicio posee y controla exclusivamente su propia base de datos. El acceso directo a la base de datos de otro servicio está terminantemente prohibido. La integración entre datos de distintos dominios ocurre exclusivamente via APIs o mensajería.

### 6.2 Mapa de Bases de Datos por Servicio

Servicio	Motor	Instancia AWS	Base de Datos	Justificativa
`auth-service`	PostgreSQL 15	RDS Multi-AZ	`auth_db`	Consistencia fuerte para credenciales
`user-service`	PostgreSQL 15	RDS Multi-AZ	`users_db`	Dados relacionais de perfil
`agendio-scheduling-service`	PostgreSQL 15	RDS Multi-AZ	`scheduling_db`	ACID para agendamentos críticos
`payment-service`	PostgreSQL 15	RDS Multi-AZ	`payments_db`	ACID para transações financeiras
`medical-records-service`	MongoDB 6	AWS DocumentDB	`medical_records_db`	Esquema flexível para registros clínicos
`ai-assistant-service`	Pinecone	Pinecone Cloud	`knowledge_vectors`	Vector store para RAG

`audit-service`	PostgreSQL 15	RDS Multi-AZ	`audit_db`	Append-only, alta integridade
Caché Global	Redis 7	ElastiCache	Múltiplos namespaces	TTL-based caching cross-service

6.3 Redis — Estrategia de Caché

Namespace	Contenido	TTL	Propietario
`auth:token:{iti}`	Blacklist de tokens revogados	Até expiração do JWT	`auth-service`
`auth:jwtks`	JWKS keys públicas (cache no API GW)	5 minutos	`auth-service`
`user:profile:{userId}`	Perfil do usuário serializado	10 minutos	`user-service`
`schedule:availability:{doctorId}:{date}`	Slots disponíveis do médico	2 minutos	`scheduling-service`
`ai:response:{queryHash}`	Respostas de IA cacheadas	1 hora	`ai-assistant-service`

6.4 Migrations con Flyway

Todas las bases de datos PostgreSQL utilizan Flyway para versionado de schema. Las reglas son:

- Scripts nombrados como `V{versao}\_{descripcion}.sql` (ej: `V2\_\_add\_appointment\_notes\_column.sql`)
- **Scripts de migração são imutáveis após merge em `main`**
- Migrations são executadas automaticamente no startup do serviço (integradas ao pipeline CI/CD)
- **Scripts DDL manuais em produção são terminantemente proibidos**

---

SECCIÓN 7 — API GATEWAY

7.1 Responsabilidades

El API Gateway es el único punto de entrada para todos los clientes externos. Su posición en la arquitectura le otorga responsabilidades críticas de seguridad, observabilidad y gestión de tráfico:

Responsabilidad	Detalles
Routing	Redirige cada ruta para o microserviço correspondente com base no path prefix (`/auth/`, `/users/`, `/appointments/`)

Autenticación Centralizada	Valida token JWT en cada requisição (via Redis cache del `auth-service`) antes de enrutar
Rate Limiting	100 req/min por tenant (ajustável por plano); 10 req/min para endpoints sensibles
SSL/TLS Termination	Toda comunicação externa usa TLS 1.3; internamente se usa mTLS
CORS	Configuração centralizada de origens permitidas por ambiente
Request Logging	Todo request é logado com Trace ID para observabilidade
Circuit Breaking	Se um serviço interno não responde, o Gateway retorna 503 imediatamente
Transformaciones	Adição de headers internos (X-User-Id, X-User-Role, X-Tenant-Id) para context propagation

## 7.2 Tecnología

- **Kong Gateway deployado em AWS ECS Fargate**
- Plugins utilizados: `jwt`, `rate-limiting`, `request-id`, `prometheus`, `cors`, `acl`
- **Configurado via Kong Admin API com `Declarative Config (deck)` versionado em Git**

## 7.3 Mapeo de Rutas

...

```

/v1/auth/ → auth-service:8081
/v1/users/ → user-service:8082
/v1/doctors/ → user-service:8082
/v1/appointments/ → agendio-scheduling-service:8083
/v1/schedules/ → agendio-scheduling-service:8083
/v1/payments/ → payment-service:8085
/v1/records/ → medical-records-service:8086
/v1/ai/ → ai-assistant-service:8087
/v1/audit/ → audit-service:8088 (somente roles AUDITOR/ADMIN)

```

...

---

# SECCIÓN 8 — INFRAESTRUCTURA EN AWS

## 8.1 Mapa de Servicios AWS Utilizados

Servicio AWS	Uso	Servicio(s) Consumidor(es)
--------------	-----	----------------------------



ECS Fargate	Orquestación de contenedores Docker (sin EC2 gestionado)	Todos los microservicios
RDS PostgreSQL Multi-AZ	Bases de datos relacionales con failover automático	auth, user, scheduling, payment, audit
AWS DocumentDB	MongoDB-compatible para documentos clínicos flexibles	`medical-records-service`
ElastiCache (Redis)	Caché distribuido y blacklist de tokens	auth, user, scheduling, ai-assistant
SQS FIFO	Mensajería asíncrona entre dominios	Todos (producer/consumer)
SES	Envío de emails transaccionales	`agendio-notification-service`
SNS	Envío de SMS	`agendio-notification-service`
S3	Archivos de exámenes, PDFs, backups de audit	medical-records, audit
S3 Glacier	Archivado de logs de auditoría (90+ días)	`audit-service`
Secrets Manager	Gestión de credenciales, llaves de API, certificados	Todos los microservicios
KMS	Encriptación de datos sensibles en reposo	`medical-records-service`
CloudWatch	Logs, métricas nativas AWS, alarmes básicos	Todos
ECR	Registry privado de imagens Docker	CI/CD Pipeline
Route 53	DNS y health checks	API Gateway
Certificate Manager (ACM)	Certificados TLS	API Gateway, Load Balancer
VPC + Private Subnets	Isolamento de red — microservicios nunca expostos publicamente	Todos

## 8.2 Topología de Red

...

AWS VPC (10.0.0.0/16)

Public Subnets

[ALB / API Gateway] [NAT Gateway]

Private Subnets (App)

[ECS Fargate Tasks — Microservicios]

[ElastiCache Redis]

Private Subnets (Data)

[RDS PostgreSQL Multi-AZ]

[DocumentDB Cluster]

...

### 8.3 Estrategia de Deploy — ECS Fargate

- Cada microservicio es una ECS Service dedicada con auto-scaling basado en CPU (target: 60%)
- Task Definitions versionadas; rollback en < 2 minutos via ECS rolling deployment
- Imagens Docker almacenadas no AWS ECR con scanning de vulnerabilidades automático
- Blue-Green deployment via AWS CodeDeploy para servicios críticos (`auth-service`, `agendario-scheduling-service`, `payment-service`)
- Health checks configurados en cada ECS Task apuntando para `/actuator/health`

---

## SECCIÓN 9 — OBSERVABILIDAD DISTRIBUIDA

### 9.1 Los Tres Pilares

La observabilidad en Santo Pegasus se basa en los tres pilares fundamentales descritos na Guía de Ingeniería Back-end: logs, métricas y rastreo distribuido.

## 9.2 Propagación del Trace ID

El `Trace ID` es el hilo conductor que permite rastrear el ciclo de vida completo de una requisición a través de todos los microservicios involucrados.

### Flujo de propagación:

...

Cliente → API Gateway (genera Trace ID si inexistente)

→ header: X-B3-TraceId: abc123, X-B3-SpanId: span01

→ auth-service (propaga headers, genera Span propio)

→ header: X-B3-TraceId: abc123, X-B3-SpanId: span02

→ agendio-scheduling-service (propaga, genera Span propio)

→ header: X-B3-TraceId: abc123, X-B3-SpanId: span03

→ SQS Message (Trace ID incluido no envelope do evento)

→ agendio-notification-service (propaga via envelope SQS)

→ audit-service (propaga via envelope SQS)

...

### Implementación técnica:

- **Spring Cloud Sleuth (integrado com Micrometer Tracing)** para propagação automática
- **OpenTelemetry Collector** deployado como sidecar em cada ECS Task
- **Dados de tracing** exportados para Datadog APM via OTLP

## 9.3 Logging Estructurado

Conforme a Guía de Ingeniería Back-end (v2.4.0):

- **SLF4J + Logback** como stack de logging em todos os serviços
- **Logs em formato JSON estruturado com campos padronizados:** `traceId`, `spanId`, `service`, `level`, `message`, `timestamp`, `userId` (quando disponível)
- **Nível predeterminado em produção: INFO;** nível DEBUG somente via feature flag temporária
- **Prohibido registrar información sensible (PII):** contraseñas, CPF, datos de pago
- **Logs ingeridos em Datadog Log Management via CloudWatch Logs subscription filter**
- En el proyecto Agendio, el ID da consulta pode ser registrado para rastreio, mas os dados pessoais dos pacientes devem ser omitidos

## 9.4 Métricas con Spring Boot Actuator + Micrometer

- Cada servicio expone `/actuator/metrics`, `/actuator/health`, `/actuator/prometheus`
- **Prometheus coleta métricas via scraping (ECS Service Discovery)**
- **Datadog recebe métricas via integração nativa Prometheus → Datadog Agent**

**Métricas clave monitoreadas por servicio:**

Métrica	Descripción	Alerta configurada
`http.server.requests.duration`	Latência P95 por endpoint	> 2s → Warning; > 5s → Critical
`http.server.requests.error.rate`	Taxa de erros 5xx	> 1% → Warning; > 5% → Critical
`jvm.memory.used`	Uso de memória JVM	> 85% → Warning
`db.pool.connections.active`	Conexiones activas al pool DB	> 80% da pool → Warning
`sqs.messages.received`	Throughput de mensagens SQS	Anomalia detectada → Alert
`circuit.breaker.state`	Estado do Circuit Breaker	OPEN → Critical imediato

9.5 Dashboards en Datadog

Dashboard	Squad	Contenido
Platform Overview	Todos	Visão geral saúde de todos os serviços
Auth & Security	Squad Hermes	Login rates, token errors, anomalias de segurança
Scheduling Core	Squad Agendio Core	Agendamentos/hora, taxa de cancelamento, latência
Payments	Squad Pagamentos	Volume transaccional, taxa de erro de gateway, chargeback
Clinical Records	Squad Clínico	Acessos a registros, erros RBAC, uploads S3
AI Assistant	Squad IA	Latência de inferência, custo estimado de tokens LLM
Governance	Squad Governance	Eventos de auditoria, anomalias de compliance

SECCIÓN 10 — ESTRATEGIA DE VERSIONADO DE APIS

10.1 Política de Versionado

Todas as APIs de Santo Pegasus seguem versionado explícito via path prefix (`/v1/`, `/v2/`). O versionado via headers (`Accept: application/vnd.santopegasus.v2+json`) foi avaliado e descartado por dificultar a observabilidade e o roteamento no API Gateway.

Reglas:

- Novas versiones de API são criadas somente quando há breaking changes

- Adições retrocompatíveis (novos campos opcionais, novos endpoints) não requerem nova versão
- **Uma versão obsoleta (deprecated) permanece ativa por mínimo 6 meses após a publicação da nova versão**

## 10.2 Proceso de Deprecación

...

### FASE 1 — Anuncio (Día 0)

- Publicar changelog no repositório e canal #engineering-announcements
- Adicionar header: Deprecation: true, Sunset: <data> em todas respostas
- Atualizar documentação Swagger com banner de deprecation



### FASE 2 — Período de Suporte (Mês 1 ao 6)

- Ambas versões ativas e suportadas
- Métricas de adoção monitoradas no Datadog
- Comunicação ativa para clientes ainda na versão antiga



### FASE 3 — Sunset (Mês 6+)

- Versão antiga retorna HTTP 410 Gone com link para migração
- Remoção do código após 30 dias de 410

...

## 10.3 Versiones Actuales por Servicio

Servicio	Versión Activa	Versión Deprecated	Sunset
`auth-service`	v1	—	—

`user-service`	v1	—	—
`agendiao-scheduling-service`	v2	v1	Dez/2026
`payment-service`	v1	—	—
`medical-records-service`	v1	—	—
`ai-assistant-service`	v1 (beta)	—	—
`audit-service`	v1	—	—

## SECCIÓN 11 — SEGURIDAD ENTRE SERVICIOS

### 11.1 Autenticación y Autorización

Conforme documentado na Guia de Ingeniería Back-end (v2.4.0), o controle de acceso es gestionado de forma centralizada utilizando Spring Security. Todos os endpoints são protegidos; nenhum endpoint interno é acessível sem validação de identidade. Tokens JWT com flujo OAuth 2.0 / OpenID Connect são o mecanismo padrão. A extração e validação de permissões (roles) ocorrem a nivel de endpoint.

### 11.2 mTLS para Comunicación Interna

Toda comunicação serviço-a-serviço dentro da VPC privada usa mTLS (Mutual TLS). Isso garante que ambos os lados de uma conexão se autenticuem, prevenindo ataques de spoofing e man-in-the-middle em comunicações internas.

Implementación:

- **Certificados gerenciados por AWS Certificate Manager (ACM) Private CA**
- Certificados rotacionados automaticamente a cada 90 días
- Cada microserviço possui um certificado de cliente único para autenticação mútua
- **Service Mesh (AWS App Mesh) como camada de controle de mTLS (em rollout progressivo — ver Roadmap)**

### 11.3 Service Accounts y Principio de Privilegio Mínimo

Cada microservicio opera com um IAM Role exclusivo na AWS, seguindo estritamente o principio de privilegio mínimo:

Servicio	IAM Role	Permisos concedidos
`auth-service`	`iam-role-auth-service`	Secrets Manager (auth/), ElastiCache, SQS (publish audit-events)

`agendiao-scheduling-service`	`iam-role-scheduling-service`	RDS (scheduling_db), SQS (publish/consume scheduling queues)
`medical-records-service`	`iam-role-medical-records`	DocumentDB, S3 (medical-files/), KMS, SQS (publish audit-events)
`payment-service`	`iam-role-payment-service`	RDS (payments_db), Secrets Manager (payment/), SQS
`ai-assistant-service`	`iam-role-ai-assistant`	Secrets Manager (ai/), ElastiCache, SQS
`audit-service`	`iam-role-audit-service`	RDS (audit_db), SQS (consume all audit queues), S3 Glacier

**Credenciais de servicios externos (Stripe, OpenAI, Pinecone): armazenadas exclusivamente em AWS Secrets Manager e injetadas em tempo de execução via Spring Cloud Config. Jamais escritas no código ou repositórios Git. Ferramentas de escaneio automático (`gitleaks`, `trufflehog`) são executadas nos pipelines de CI/CD para detectar e bloquear commits que contenham secrets.**

## 11.4

# Santo Pegasus Soluciones: Guía Oficial de Ingeniería Back-end

**Versión del Documento:** 2.4.0

**Última Actualización:** Octubre de 2025

**Departamento:** Ingeniería de Software / Chapter de Back-end

## 1. Introducción

Bienvenido a la Guía de Ingeniería Back-end de **Santo Pegasus Soluciones**. Este documento sirve como la "fuente de la verdad" para todos los desarrolladores de nuestro equipo. Nuestro objetivo es garantizar que el código que producimos sea escalable, sostenible, legible y seguro.

La alineación con estas directrices es obligatoria para todos los nuevos proyectos y fuertemente recomendada para refactorizaciones de sistemas heredados. La conformidad con esta guía asegura que Santo Pegasus mantenga un alto estándar de entrega técnica, facilitando la colaboración entre diferentes equipos y la evolución continua de nuestra infraestructura de software.

## 2. Principios Fundamentales de Ingeniería

En Santo Pegasus, basamos nuestras decisiones arquitectónicas y de codificación en principios universales de ingeniería de software. Todos los desarrolladores deben tener los siguientes conceptos internalizados para garantizar que el código producido sea escalable, sostenible, legible y seguro:

### 2.1. SOLID

Los cinco principios de la programación orientada a objetos que facilitan el mantenimiento y la expansión del código:

- **S - Single Responsibility Principle (SRP):** Una clase debe tener uno, y solo un, motivo para cambiar. Este principio fundamental dicta que la cohesión debe ser alta, garantizando que cada componente del sistema ejecute una única tarea lógica. En Santo Pegasus, es obligatorio separar las responsabilidades en capas distintas: por ejemplo, la lógica de persistencia y comunicación con la base de datos debe residir exclusivamente en los **Repositories**, jamás insertándose directamente en los **Controllers**, que deben enfocarse solo en el manejo de peticiones y respuestas HTTP. Al aplicar el SRP, reducimos el acoplamiento, facilitamos la creación de pruebas unitarias y minimizamos el



riesgo de efectos secundarios durante mantenimientos futuros.

- **O - Open/Closed Principle (OCP):** Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación. En Santo Pegasus, priorizamos el uso de interfaces y abstracciones para permitir que se agreguen nuevos comportamientos sin necesidad de alterar el código fuente original ya consolidado. Esto es crucial para evitar la introducción de regresiones en sistemas estables y facilitar la implementación de nuevos algoritmos de procesamiento, como ocurre en nuestros servicios de pago.
- **L - Liskov Substitution Principle (LSP):** Las clases derivadas deben ser capaces de sustituir a sus clases base sin comprometer la integridad de la aplicación. En Santo Pegasus, esto significa que una subclase nunca debe eliminar el comportamiento de la superclase o lanzar excepciones inesperadas para métodos heredados, garantizando que el polimorfismo funcione de forma predecible en todos nuestros servicios.
- **I - Interface Segregation Principle (ISP):** Es preferible tener múltiples interfaces específicas a una única interfaz genérica. No se debe obligar a una clase a implementar métodos que no utilizará. En Santo Pegasus, aplicamos este principio para evitar la creación de "interfaces gordas" que contaminan el código y aumentan el acoplamiento innecesario. Al segmentar interfaces, garantizamos que las clases clientes dependan solo de los métodos que realmente necesitan, facilitando el mantenimiento y la evolución modular de nuestros servicios back-end.
- **D - Dependency Inversion Principle (DIP):** Dependencia de abstracciones y no de implementaciones concretas. En Santo Pegasus, este principio es la base para un sistema verdaderamente desacoplado. En lugar de que una clase de alto nivel instancie directamente una clase de bajo nivel, ambas deben depender de interfaces. Utilizamos extensivamente la **Inyección de Dependencias de Spring** para gestionar el ciclo de vida de los objetos y viabilizar este principio, permitiendo que las implementaciones reales sean intercambiadas sin afectar el flujo principal del negocio. Esto facilita el mantenimiento, permite la sustitución de proveedores externos y hace que las pruebas unitarias con *mocks* sean mucho más simples y eficaces.

## 2.2. DRY, KISS y YAGNI

Además de SOLID, aplicamos los principios DRY (Don't Repeat Yourself) para evitar la duplicación de lógica, KISS (Keep It Simple, Stupid) para mantener un diseño simple y legible, y YAGNI (You Aren't Gonna Need It) para evitar la implementación de funcionalidades innecesarias para el momento actual. En **Santo Pegasus Soluciones**, la aplicación de **DRY** significa que, al identificar la misma lógica en múltiples lugares, esta debe extraerse a métodos utilitarios o servicios base, garantizando que una alteración en una regla de negocio deba hacerse en un solo lugar.

El principio **KISS** es nuestro norte para el diseño de sistemas; priorizamos la legibilidad y la facilidad de mantenimiento en detrimento de soluciones excesivamente ingeniosas o complejas que puedan oscurecer el funcionamiento del código para otros miembros del equipo.

Finalmente, **YAGNI** nos protege contra el desperdicio de esfuerzo de ingeniería, orientando al

equipo a implementar estrictamente lo necesario para los requisitos actuales, sin agregar funcionalidades basadas en suposiciones de necesidades futuras que pueden nunca concretarse.

### 2.2.1. DRY (Don't Repeat Yourself)

Evite la duplicación de código. Si encuentra la misma lógica escrita en dos lugares diferentes, extráigala a un método utilitario, servicio o clase base. La aplicación de este principio garantiza que una alteración en una regla de negocio se realice en un solo lugar, reduciendo drásticamente el riesgo de inconsistencias en el sistema. Además de la extracción a métodos, recomendamos el uso de constantes para valores fijos y la centralización de validaciones comunes en clases utilitarias o en los propios DTOs utilizando Bean Validation, evitando que la misma validación se repita en múltiples Services.

### 2.2.2. KISS (Keep It Simple, Stupid)

La simplicidad es el objetivo principal del diseño en **Santo Pegasus Soluciones**. Este principio sirve como nuestro norte para el desarrollo de sistemas, donde priorizamos la legibilidad y la facilidad de mantenimiento sobre soluciones excesivamente ingeniosas o complejas que puedan oscurecer el funcionamiento del código para otros miembros del equipo.

Evitar la complejidad innecesaria significa que un código legible y simple de entender es siempre preferible a un código "inteligente", pero oscuro y complejo. En la práctica, esto se traduce en evitar abstracciones prematuras y garantizar que la lógica del negocio se exprese de forma clara y directa, facilitando la evolución continua de nuestra infraestructura de software y la colaboración entre diferentes equipos.

### 2.2.3. YAGNI (You Aren't Gonna Need It)

El principio **YAGNI** actúa como una salvaguarda contra el desperdicio de esfuerzo de ingeniería en **Santo Pegasus Soluciones**. Orienta al equipo a no agregar funcionalidades basadas en suposiciones de necesidades futuras que pueden nunca concretarse.

En la práctica, la aplicación de este principio implica:

- **Enfoque en el requisito actual:** Implemente estrictamente lo que es necesario para atender el requisito de negocio presente.
- **Reducción de complejidad:** Al evitar el desarrollo prematuro de recursos "útiles en el futuro", mantenemos la base de código más limpia, facilitando el mantenimiento y la comprensión por parte de otros desarrolladores.
- **Eficiencia de recursos:** Protegemos al equipo contra el gasto de tiempo en códigos que pueden necesitar ser eliminados o reescritos totalmente en caso de que la necesidad futura cambie de dirección.

Este principio es un complemento esencial al diseño simple (KISS), garantizando que la

evolución de nuestra infraestructura esté guiada por necesidades reales e inmediatas.

## 3. Patrones Arquitectónicos: Java y Spring Boot

Nuestro ecosistema principal está basado en **Java 17+** y **Spring Boot 3+**. Para mantener la consistencia, adoptamos una arquitectura en capas (Layered Architecture):

### 3.1. División de Capas

- **Controllers (@RestController)**: Responsables exclusivamente de recibir peticiones HTTP, validar los parámetros de entrada (usando Bean Validation), llamar al Service adecuado y retornar la respuesta HTTP correcta (con los debidos Status Codes).  
**Ninguna regla de negocio debe residir aquí.**
- **Services (@Service)**: El corazón de la aplicación. Aquí reside toda la regla de negocio. El Service orquesta las llamadas a los repositorios y servicios externos.
- **Repositories (@Repository)**: Interfaces que extienden JpaRepository o MongoRepository. Responsables de la comunicación exclusiva con la base de datos.

### 3.2. Transferencia de Datos (DTOs y Mappers)

**Nunca** retorne entidades de base de datos (ej: clases anotadas con @Entity) directamente en los Controllers.

- Use el Patrón **DTO (Data Transfer Object)** para la comunicación entre la API y los clientes.
- Utilice bibliotecas como **MapStruct** para convertir Entidades en DTOs y viceversa, evitando código *boilerplate* de mapeo manual.

### 3.3. Manejo Global de Excepciones

Todas las APIs de Santo Pegasus deben implementar un @ControllerAdvice o @RestControllerAdvice.

- Las excepciones de negocio deben ser personalizadas (ej: ResourceNotFoundException, BusinessException).
- El retorno de error debe seguir la especificación RFC 7807 (Problem Details for HTTP APIs), estandarizando la estructura JSON de errores devuelta al *front-end*.

### 3.4. Documentación de APIs

Para garantizar la interoperabilidad y facilitar la integración con aplicaciones front-end y clientes externos, todas las APIs deben estar rigurosamente documentadas utilizando la especificación OpenAPI (Swagger). La documentación debe generarse automáticamente a partir del código (utilizando bibliotecas como springdoc-openapi) y exponer claramente los *endpoints*,

parámetros esperados, esquemas de *payload* y códigos de respuesta HTTP de forma actualizada.

### 3.5. Comunicación Síncrona entre Microservicios

En escenarios donde la comunicación síncrona entre servicios internos es estrictamente necesaria (evitando el acoplamiento temporal siempre que sea posible), estandarizamos la utilización del WebClient del ecosistema Spring WebFlux para llamadas REST. Para rutas críticas que demandan altísimo rendimiento y baja latencia, se fomenta fuertemente el uso de gRPC. La utilización de clientes heredados, como RestTemplate, está descontinuada para nuevos proyectos.

## 4. Patrones de Diseño (Design Patterns)

Fomentamos el uso de Design Patterns consolidados (GoF) para resolver problemas recurrentes y garantizar una base de código flexible:

1. **Strategy:** Este patrón es esencial en nuestros flujos de pago. Permite alternar algoritmos de procesamiento (como Pix, Tarjeta o Boleto) dinámicamente en tiempo de ejecución. Al encapsular cada método de pago en su propia clase de estrategia, eliminamos estructuras condicionales (if/else o switch/case) extensas y complejas en el servicio principal, facilitando la adición de nuevos métodos sin impactar el código existente. En términos prácticos, el sistema se vuelve más modular, ya que la lógica específica de cada proveedor o medio de pago queda aislada, permitiendo que el *Service* solo orqueste la ejecución a través de una interfaz común.
2. **Builder:** Este patrón es un requisito obligatorio para la instanciación de objetos complejos, garantizando la inmutabilidad de las instancias y la legibilidad del código al evitar el uso de constructores con largas listas de parámetros. Priorizamos la utilización de la anotación `@Builder` de la biblioteca **Lombok**, lo que reduce drásticamente el código *boilerplate*. Su aplicación es fundamental en escenarios de pruebas unitarias, permitiendo la creación de escenarios de prueba específicos con claridad y agilidad. Además, el uso de builders facilita la evolución de las clases, ya que la adición de nuevos campos no rompe las instancias existentes en otras partes del sistema, manteniendo la integridad y facilitando mantenimientos futuros.
3. **Factory Method:** Este patrón de diseño creacional se aplica cuando la creación de un objeto exige una lógica decisoria compleja o validaciones previas que no deben sobrecargar a la clase cliente. Al centralizar la responsabilidad de instanciación en un método o clase específica, promovemos el desacoplamiento entre el cliente y las implementaciones concretas, facilitando la introducción de nuevos tipos de objetos sin la necesidad de modificar el código que los utiliza.

- **Aplicaciones Prácticas:** Utilizamos el *Factory Method* para instanciar componentes que dependen de configuraciones dinámicas o parámetros específicos del entorno, garantizando que el objeto resultante esté siempre en un estado válido y listo para uso.
  - **Beneficios para el Equipo:** La adopción de este patrón reduce la complejidad en los **Services**, pues elimina la necesidad de múltiples bloques *if/else* o *switch* destinados solo a la creación de instancias, haciendo el flujo principal de negocio mucho más legible y fácil de mantener.
4. **Observer:** Este patrón de comportamiento es fundamental para mantener el desacoplamiento en nuestra arquitectura orientada a eventos. En Santo Pegasus, se implementa vía *ApplicationEventPublisher* de Spring para gestionar comunicaciones asíncronas de forma eficiente.
- **Casos de Uso:** Se utiliza principalmente para envíos de correos electrónicos post-registro, integración con servicios de notificación (como el microservicio que utiliza AWS SES y SQS) y actualizaciones de registros de auditoría, sin que el *Service* de negocio necesite conocer los detalles de estas implementaciones.
  - **Ventajas:** Al adoptar el *Observer*, garantizamos que los servicios principales permanezcan enfocados exclusivamente en sus reglas de negocio fundamentales, reduciendo la complejidad cognitiva del código y facilitando la inclusión de nuevos oyentes (*listeners*) para un mismo evento sin alterar el código del publicador.

## 5. Implementaciones de IA y Uso de RAG

Con la llegada de las IAs Generativas, Santo Pegasus integró componentes inteligentes en sus back-ends. Para casos donde el LLM necesita consultar nuestros datos internos, utilizamos la arquitectura **RAG (Retrieval-Augmented Generation)**. Este enfoque permite que el modelo genere respuestas basadas en hechos y conocimientos específicos de la empresa, que no formaban parte de su entrenamiento original.

El funcionamiento del RAG en Santo Pegasus sigue cuatro etapas principales: primero, la pregunta del usuario se usa como query para buscar fragmentos semánticamente relevantes en una base de datos externa (Retrieval); luego, esos fragmentos son extraídos (Context Extraction); después, el contexto recuperado se combina con la pregunta original para formar un prompt robusto; y, finalmente, el LLM genera la respuesta fundamentada (Generation). Esta estrategia es vital para automatizar atenciones operativas, como dudas frecuentes de RRHH sobre beneficios y políticas internas, liberando a los equipos para funciones estratégicas.

### 5.1. Reglas para Pipelines RAG

- **Chunking (División de Texto):** Textos largos deben ser divididos en fragmentos (chunks) de máximo **512 tokens**, con un *overlap* (superposición) de 50 tokens para

mantener el contexto entre los párrafos.

- **Embeddings:** El modelo estándar homologado para generar vectores en Santo Pegasus es el text-embedding-004.
- **Base de Datos Vectorial:** Utilizamos **Pinecone** o **Qdrant** en producción. Las bases vectoriales en memoria (como FAISS) están permitidas solo para pruebas locales.
- **Recuperación (Retrieval):** El número máximo de documentos recuperados (K) para componer el contexto del *prompt* del LLM debe ser **K=4**. Un número mayor causa degradación del rendimiento y alucinaciones.
- **Orquestación:** Todo el flujo de RAG debe ser orquestado vía **LangChain** o **LlamaIndex** en la capa de Service.

## 6. Control de Versiones y GitFlow

### 6.1. Patrón de Nomenclatura de Ramas (Branches)

Utilizamos una variación simplificada de GitFlow. El nombre de las ramas debe seguir el patrón:

- feature/TICKET-123-nombre-de-la-feature
- bugfix/TICKET-456-descripcion-del-bug
- hotfix/TICKET-789-correccion-critica-prod

### 6.2. Commits Semánticos (Conventional Commits)

Los mensajes de commit deben obligatoriamente seguir la especificación de *Conventional Commits*:

- feat: agrega endpoint de pago PIX
- fix: corrige cálculo de impuesto en la factura
- refactor: mejora rendimiento del mapper de usuarios
- docs: actualiza readme con instrucciones de setup local
- test: agrega pruebas unitarias para el UserService

## 7. Proceso de Code Review

El Code Review es el pilar de nuestra calidad de software. Ningún código va a la rama main sin pasar por ese proceso.

### 7.1. Reglas para el Autor (Quien abre el Pull Request)

1. Garantice que el código compila y que la cobertura de pruebas unitarias sea **igual o superior al 80%** (medido vía JaCoCo/SonarQube).
2. El PR no debe contener más de 400 líneas de código alteradas. Si es mayor, divídalo en

PRs más pequeños.

3. Complete el *template* del PR explicando "Qué se hizo", "Por qué se hizo" y "Cómo probar".

## 7.2. Reglas para el Revisor

1. **Aprobaciones:** Todo Pull Request (PR) en Santo Pegasus exige **al menos 2 aprobaciones** de miembros Senior o Semi-Senior (Pleno) antes del merge. Este criterio garantiza que el código sea revisado desde diferentes perspectivas, reduciendo fallas técnicas y diseminando el conocimiento sobre las alteraciones entre el equipo.
2. **Tono y Etiqueta:** La comunicación debe estar siempre pautada por la empatía y la educación. El enfoque de la revisión debe ser exclusivamente la mejora del código, nunca una crítica al autor. Se recomienda el uso de preguntas sugestivas para incentivar la reflexión del desarrollador, como: "*¿Qué te parece extraer esto a un método privado?*", evitando imperativos como "*Cambia esto a un método privado*".
3. **Criterios de Revisión:** El revisor actúa como un guardián de la base de código, debiendo validar minuciosamente la legibilidad (facilidad de comprensión), la seguridad (prevención de vulnerabilidades), el rendimiento (uso eficiente de recursos) y, fundamentalmente, si la implementación respeta rigurosamente todas las directrices establecidas en la Guía de Ingeniería Back-end.

## 8. Seguridad y Buenas Prácticas Generales

### 8.1. Autenticación y Autorización

El control de acceso a nuestras APIs debe gestionarse de forma centralizada utilizando Spring Security. Adoptamos el uso de tokens JWT (JSON Web Tokens) basados en el flujo del protocolo OAuth 2.0 u OpenID Connect (OIDC). La extracción y validación de permisos (*roles*) deben ocurrir a nivel de *endpoint*, garantizando que solo los clientes autenticados y con los privilegios correctos ejecuten operaciones sensibles.

### 8.2. Credenciales y Secrets

NUNCA haga *hardcode* (escribir directamente en el código) de contraseñas, claves de API, URLs de base de datos o tokens de servicios de IA. La exposición de credenciales en repositorios de código es una de las vulnerabilidades más críticas y puede llevar al compromiso total de la infraestructura. Para garantizar la protección de estos datos sensibles, siga rigurosamente las directrices a continuación:

- **Uso de Bóvedas de Contraseñas:** En entornos de producción y homologación, es obligatorio el uso de soluciones de gestión de secretos, como **HashiCorp Vault**, **AWS Secrets Manager** o **Google Cloud Secret Manager**. Integre estos servicios a Spring

Boot utilizando *Spring Cloud Config* para que los secretos sean inyectados en tiempo de ejecución sin tocar nunca el disco o el repositorio Git.

- **Sanitización de Repositorios:** Utilice herramientas de escaneo automático, como *gitleaks* o *trufflehog*, en nuestros pipelines de CI/CD para detectar y bloquear commits que contengan cadenas de texto que se asemejen a secretos (claves privadas, tokens JWT, contraseñas).
- **Segregación de Acceso:** Las credenciales deben ser exclusivas por entorno. Jamás utilice una contraseña de base de datos de producción en un entorno de desarrollo. El principio de privilegio mínimo debe ser aplicado, garantizando que cada microservicio tenga acceso solo a los secretos estrictamente necesarios para su operación.
- **Rotación de Credenciales:** Establezca un proceso de rotación periódica para claves de API y tokens de servicios externos, minimizando el impacto en caso de que ocurra una filtración accidental.

### 8.3. Variables de Entorno

Para garantizar la seguridad y la flexibilidad de las configuraciones en diferentes entornos (desarrollo, homologación y producción), utilice siempre **variables de entorno** para gestionar datos sensibles y parámetros volátiles. En el ecosistema Java y Spring Boot, la recuperación de estos valores debe hacerse preferiblemente a través de:

- **Inyección con @Value:** Utilice la anotación `@Value("${NOMBRE_DE_LA_VARIABLE}")` para inyectar valores directamente en atributos de clases gestionadas por Spring.
- **Acceso Nativo:** Use `System.getenv()` cuando sea necesario acceder a variables fuera del contexto de inyección de Spring.

Durante el **desarrollo local**, está permitida y fomentada la utilización de archivos **.env** para emular el entorno de ejecución. Sin embargo, por cuestiones críticas de seguridad, estos archivos **deben estar obligatoriamente listados en el .gitignore** para evitar la filtración de credenciales en el repositorio Git.

### 8.4. Logging

La utilización de `System.out.println()` para fines de depuración o rastreo en entornos profesionales está estrictamente prohibida en **Santo Pegasus**. Esta práctica es ineficiente, ya que no permite la categorización por niveles de severidad, dificulta el filtrado en herramientas de agregación de logs y causa un impacto negativo en el rendimiento de la aplicación debido al comportamiento bloqueante del flujo de salida estándar. Estandarizamos el uso de **SLF4J** (Simple Logging Facade for Java) como interfaz de abstracción, teniendo a **Logback** como framework de implementación nativo de Spring Boot. Las siguientes directrices deben seguirse rigurosamente para la gestión de logs:

- **Niveles de Log en Producción:** El nivel predeterminado debe configurarse como **INFO**. El nivel **DEBUG** solo debe habilitarse temporalmente para la solución de problemas



específicos, buscando evitar el almacenamiento innecesario de grandes volúmenes de datos.

- **Manejo de Errores:** Las fallas inesperadas o excepciones de sistema deben ser registradas obligatoriamente en el nivel **ERROR**. Es fundamental que el mensaje de error venga acompañado de la *stacktrace* completa para viabilizar el análisis de causa raíz por parte de ingeniería.
- **Privacidad y Seguridad:** Está terminantemente prohibido registrar información sensible (PII) en los logs, tales como contraseñas, claves de API, números de CPF o datos de pago. En el proyecto *Agendio*, por ejemplo, el ID de la consulta puede registrarse para rastreo, pero los datos personales de los pacientes deben omitirse.
- **Logs Estructurados:** Siempre que sea posible, utilice logs en formato JSON para facilitar la ingesta por sistemas como ELK Stack o Datadog.

## 8.5. Validación de Entradas

La integridad de los datos y la seguridad de la aplicación dependen fundamentalmente de la premisa de que el back-end es la última y más importante línea de defensa. En Santo Pegasus, seguimos la regla de **nunca confiar en los datos enviados por el cliente**, independientemente de cuán robusta sea la validación en el front-end. Implementamos validaciones estrictas en múltiples capas para mitigar riesgos de *SQL Injection*, *Cross-Site Scripting* (XSS) y la persistencia de datos corruptos que puedan comprometer la lógica de negocio. Para garantizar la conformidad técnica, las siguientes prácticas son obligatorias:

- **Bean Validation (JSR 380):** Utilice anotaciones como `@NotNull`, `@Size`, `@Email` y `@Pattern` directamente en los DTOs de entrada de los Controllers.
- **Sanitización de Cadenas de Texto (Strings):** Elimine caracteres especiales o scripts maliciosos de entradas de texto que se mostrarán posteriormente a otros usuarios.
- **Validación de Tipos y Formatos:** Asegúrese de que los IDs, fechas y valores monetarios sigan estrictamente el formato esperado por la aplicación antes de cualquier procesamiento por capas de servicio.

## 8.6. Observabilidad Integral: Métricas y Tracing

Además del uso ya establecido de SLF4J/Logback para logs estructurados, nuestros servicios deben ser plenamente observables para garantizar la resiliencia operativa. La observabilidad en **Santo Pegasus** se basa en tres pilares fundamentales: logs, métricas y rastreo.

- **Métricas de Rendimiento y Salud:** Es obligatoria la exposición de métricas detalladas utilizando **Spring Boot Actuator** y **Micrometer**. Se debe configurar la exportación nativa hacia **Prometheus**, permitiendo el monitoreo en tiempo real de latencia, tasa de errores, consumo de CPU y uso de memoria.
- **Rastreo Distribuido (Distributed Tracing):** En nuestra arquitectura de microservicios, es obligatorio implementar soluciones de rastreo para propagar el *Trace ID* y el *Span ID* entre las peticiones. Esto facilita la depuración de flujos complejos y la identificación de

cuellos de botella en sistemas distribuidos.

- **Health Checks:** Los endpoints de *health* deben reflejar el estado real de las dependencias críticas (bases de datos, brokers de mensajes), permitiendo que el orquestador de contenedores tome decisiones precisas sobre el ciclo de vida de las instancias.

La implementación rigurosa de estas prácticas asegura que el equipo de ingeniería pueda responder proactivamente a incidentes y mantener el alto estándar de disponibilidad exigido por Santo Pegasus.

## 9. Gestión de Base de Datos y Migrations

La evolución del esquema de bases de datos relacionales en **Santo Pegasus** debe ser estrictamente controlada, versionada y automatizada. Está terminantemente prohibida la alteración manual de estructuras de datos, como la creación de tablas o modificación de columnas, así como la ejecución de scripts DDL directamente en entornos de producción por medio de clientes SQL. Para garantizar que el estado de la base de datos evolucione de forma predecible, auditable y sincronizada con la versión de la aplicación, adoptamos las siguientes directrices:

- **Herramientas de Migration:** Utilizamos obligatoriamente herramientas como **Flyway** o **Liquibase**. Estas herramientas permiten que cada alteración en la base de datos sea tratada como un artefacto de software, versionado en Git junto al código fuente.
- **Implantabilidad y Rollback:** Cada script de migración debe ser idempotente y probado exhaustivamente en entornos de homologación. Se recomienda el uso de **Flyway** por su simplicidad al gestionar el historial a través de una tabla de metadatos, garantizando la consistencia del esquema en todos los nodos del clúster.
- **Compatibilidad con Arquitectura Hexagonal:** Siguiendo nuestros patrones de desacoplamiento, la lógica de migración debe residir en la capa de infraestructura, garantizando que el núcleo del negocio no dependa de detalles de implementación de la base de datos (como el motor **PostgreSQL**).
- **Entornos Gestionados:** En infraestructuras en la nube (cloud), como **AWS RDS** u **Oracle Autonomous Database**, el control a través de migrations es vital para mantener la paridad entre entornos de desarrollo local y producción, facilitando la automatización vía pipelines de CI/CD.

Al mantener un registro histórico de todas las alteraciones, aseguramos la trazabilidad necesaria para auditorías y la resiliencia operativa indispensable para nuestros sistemas de misión crítica.

## 10. Estrategia y Cobertura de Pruebas

La calidad del código debe ser asegurada por un enfoque multifacético. Aunque la cobertura mínima del 80% en pruebas unitarias es obligatoria en la etapa de *Code Review*, nuestra ingeniería exige el mantenimiento de una pirámide de pruebas equilibrada:

- **Pruebas Unitarias:** Enfoque en la lógica de negocio aislada en *Services* y utilitarios, utilizando *mocks* para dependencias externas vía Mockito.
- **Pruebas de Integración:** Obligatorias para funcionalidades críticas y comunicaciones con bases de datos. Recomendamos el uso de **Testcontainers** para garantizar el aislamiento y la paridad con el entorno productivo.
- **Pruebas de Contrato:** Implementadas en APIs vitales para prevenir rupturas de compatibilidad inesperadas con sistemas consumidores y garantizar la estabilidad del ecosistema de microservicios.

## 11. CI/CD y Proceso de Deploy

El ciclo de vida de nuestras aplicaciones es automatizado a través de pipelines de Integración y Entrega Continuas (CI/CD). Tras la aprobación y el *merge* de un *Pull Request*, el proceso sigue directrices rigurosas:

- **Contenedorización:** El uso de **Docker** es obligatorio para asegurar la inmutabilidad del artefacto, garantizando que el mismo código validado en *Staging* (homologación) sea promovido a *Producción* sin variaciones.
- **Estrategias de Disponibilidad:** Para evitar cualquier indisponibilidad durante las actualizaciones, utilizamos orquestadores que soportan **Blue-Green** o **Canary Releases**, permitiendo reversiones rápidas en caso de anomalías.
- **Automatización de Infraestructura:** En entornos como AWS RDS, las alteraciones de la base de datos se integran vía herramientas de *migration* (Flyway/Liquibase) dentro del propio pipeline.

# Santo Pegasus Soluciones: Guía Oficial de Ingeniería Front-end

---

**Versión del Documento: 1.0.0**

**Última Actualización: Octubre de 2025**

**Departamento: Ingeniería de Software / Chapter de Front-end**

---

## Tabla de Contenidos

1. Introducción y Filosofía del Front-end en Santo Pegasus
2. Principios Fundamentales de Ingeniería
3. Stack Tecnológico Estándar
4. Arquitectura de Componentes
5. Gestión del Estado
6. Consumo de APIs
7. Estándares de Código
8. Estilo y Design System
9. Gestión de Formularios
10. Estrategia y Cobertura de Pruebas
11. Proceso de Code Review
12. Control de Versiones
13. Performance y Web Vitals
14. Seguridad en el Front-end
15. CI/CD Front-end

---

## 1. Introducción y Filosofía del Front-end en Santo Pegasus

### 1.1 Presentación Institucional

**Bienvenido a la Guía Oficial de Ingeniería Front-end de Santo Pegasus Soluciones. Este documento constituye la fuente de la verdad para todos los desarrolladores que integran el Chapter de Front-end, y su consulta es de carácter obligatorio para cualquier proyecto nuevo o refactorización significativa de interfaces de usuario.**

**Esta guía fue concebida para existir en perfecta coherencia con la Guía Oficial de Ingeniería Back-end (versión 2.4.0, Octubre de 2025), con la cual comparte filosofía, valores de ingeniería y estándares de proceso. El lector que ya domine el documento de back-end encontrará en estas páginas un lenguaje familiar, aunque aplicado a las particularidades y desafíos propios de la capa de presentación.**

Santo Pegasus Soluciones construye productos digitales de alta complejidad, cuyas interfaces de usuario son el principal punto de contacto entre la tecnología que desarrollamos y las personas que la utilizan. La excelencia en el front-end no es un ornamento: es un requisito de negocio.

## **1.2 Filosofía del Front-end**

**En Santo Pegasus, entendemos el front-end como una disciplina de ingeniería de software completa, sujeta a los mismos criterios de rigor técnico, legibilidad, seguridad y sostenibilidad que cualquier otra capa del sistema. Rechazamos la concepción que trata el código de interfaz como un conjunto de scripts auxiliares; en cambio, lo tratamos como un sistema distribuido de componentes que debe ser diseñado, probado, versionado y mantenido con la misma disciplina que cualquier microservicio de nuestro ecosistema back-end.**

Los tres pilares filosóficos que guían todas nuestras decisiones de front-end son:

- **Experiencia del Usuario como Métrica Técnica:** La performance percibida por el usuario final —medida a través de Web Vitals— es un criterio de aceptación técnica, no una preocupación exclusiva del área de producto.
- **Componentes como Contratos:** Cada componente expone una interfaz pública (sus props) que debe ser tan cuidadosamente diseñada como los endpoints de una API REST. Un componente bien diseñado es predecible, reutilizable y testeable de forma aislada.
- **Seguridad en la Capa de Presentación:** El front-end no es un entorno confiable. Todo dato que proviene del servidor debe ser tratado con desconfianza; todo dato introducido por el usuario debe ser validado y saneado. La seguridad es una responsabilidad compartida entre todas las capas del sistema.

## **1.3 Alcance y Aplicabilidad**

**Las directrices de este documento son de aplicación obligatoria para:**

- Todos los proyectos nuevos iniciados a partir de Octubre de 2025.
- Cualquier refactorización que afecte más del 30% de la base de código de un proyecto existente.
- Todo desarrollador del Chapter de Front-end, independientemente de su nivel de seniority.

**Su aplicación es fuertemente recomendada para proyectos legacy en mantenimiento activo, donde la adopción gradual es preferible a la no adopción.**

## 1.4 Actualización y Revisión

*Este documento es un artefacto vivo. Cualquier miembro del Chapter de Front-end puede proponer modificaciones a través de un Pull Request dirigido al repositorio `eng-guidelines`, siguiendo el proceso de Code Review estándar descrito en la Sección 11. Las actualizaciones deben ser aprobadas por al menos dos miembros Senior del Chapter y por el Tech Lead responsable.*

---

## 2. Principios Fundamentales de Ingeniería

Todo desarrollador del Chapter de Front-end debe tener los siguientes principios profundamente internalizados. Su aplicación no es opcional: es la base sobre la cual se construye código sostenible, escalable y colaborativo.

### 2.1 SOLID Aplicado al Front-end

Los principios SOLID, aunque originados en el contexto de la orientación a objetos, poseen una traducción directa y poderosa en el desarrollo de interfaces modernas basadas en componentes.

#### #### S — Single Responsibility Principle (SRP)

Un componente debe tener una única razón para cambiar. En la práctica, esto significa que un componente no debe simultáneamente obtener datos de la API, transformarlos, manejar el estado de carga y renderizar la interfaz. Cada una de estas responsabilidades debe estar delegada a la capa correspondiente: los hooks de datos, los transformadores de estado y los componentes de presentación.

#### **Incorrecto:**

```
``tsx
// ProductCard hace demasiado
const ProductCard = ({ productId }: { productId: string }) => {
 const [product, setProduct] = useState(null);
 const [loading, setLoading] = useState(true);
 useEffect(() => {
 fetch(`/api/products/${productId}`)
 .then(res => res.json())
 .then(data => {
 setProduct(data);
 setLoading(false);
 });
 }, [productId]);
```

```

 if (loading) return <Spinner />;
 return <div>{product?.name}</div>;
 };
 ...

```

### Correcto:

```

``tsx
// Responsabilidades separadas
// hook: useProduct.ts
export const useProduct = (productId: string) =>
 useQuery({ queryKey: ['product', productId], queryFn: () => fetchProduct(productId) });
// componente de presentación: ProductCard.tsx
export const ProductCard = ({ product }: { product: Product }) => (
 <div>{product.name}</div>
);
// contenedor: ProductCardContainer.tsx
export const ProductCardContainer = ({ productId }: { productId: string }) => {
 const { data, isLoading } = useProduct(productId);
 if (isLoading) return <Spinner />;
 return <ProductCard product={data!} />;
};
...

```

### #### O — Open/Closed Principle (OCP)

**Los componentes deben estar abiertos para extensión y cerrados para modificación. En React, esto se implementa a través de composición y del patrón de props como punto de extensión, evitando modificar el cuerpo interno de un componente para agregar nuevos comportamientos.**

### #### L — Liskov Substitution Principle (LSP)

*Un componente especializado debe poder sustituir a su componente base sin alterar el comportamiento esperado por el sistema. En términos prácticos, si un `PrimaryButton` extiende a `Button`, debe respetar y soportar todas las props del componente padre.*

### #### I — Interface Segregation Principle (ISP)

*Las interfaces de props de los componentes deben ser específicas y cohesivas. Un componente no debe estar obligado a recibir props que no utiliza. Prefiera la composición de interfaces pequeñas sobre la creación de un único objeto de props monolítico.*

### #### D — Dependency Inversion Principle (DIP)

Los componentes de alto nivel no deben depender de implementaciones concretas. En React, esto se expresa a través de hooks de abstracción (por ejemplo, `useUserRepository`) que pueden ser fácilmente sustituidos por mocks en los tests, sin que el componente conozca la fuente real de los datos.

## 2.2 DRY — Don't Repeat Yourself

La duplicación de código en el front-end es uno de los principales generadores de deuda técnica. La creación de un nuevo componente, hook o función utilitaria debe siempre ir precedida de una búsqueda en la base de código existente. Cuando se identifique lógica duplicada en dos o más lugares, la refactorización hacia una abstracción compartida es obligatoria.

El directorio `src/components/ui/` (componentes del Design System) y `src/hooks/` son los repositorios canónicos de código reutilizable del proyecto.

## 2.3 KISS — Keep It Simple, Stupid

La complejidad innecesaria es el mayor enemigo de la mantenibilidad. Ante la elección entre una solución simple y una solución "elegante" pero compleja, Santo Pegasus elige siempre la simplicidad. El código que cualquier desarrollador de nivel Pleno puede leer y entender en menos de cinco minutos es preferible al código que requiere conocimiento especializado de patrones avanzados.

## 2.4 YAGNI — You Aren't Gonna Need It

No construya abstracciones, componentes genéricos o sistemas de configuración para casos de uso hipotéticos que aún no existen. La arquitectura debe responder a necesidades reales y presentes. La sobre-ingeniería prematura es tan perjudicial como la ausencia de arquitectura. Los componentes deben ser generalizados únicamente cuando la necesidad de reutilización se haya demostrado en la práctica.

---

## 3. Stack Tecnológico Estándar

La coherencia del stack tecnológico es fundamental para reducir la fricción entre equipos, facilitar la rotación de desarrolladores entre proyectos y garantizar que las soluciones de CI/CD sean aplicables transversalmente.

### 3.1 Tabla del Stack Principal

Categoría	Tecnología	Versión Mínima	Justificativa
Lenguaje	TypeScript	5.0+	Tipado estático, integración con APIs back-end



Framework UI	React	18+	Concurrent features, ecosistema maduro
Meta-framework	Next.js	14+	SSR/SSG, App Router, RSC
Bundler	Vite	5+	Build rápido para SPA, HMR eficiente
Estado Global	Zustand	4+	Ligereza, sin boilerplate
Estado del Servidor	TanStack Query	5+	Cache, revalidación, sincronización
Formularios	React Hook Form	7+	Performance, integración con Zod
Validación	Zod	3+	Schemas tipados, integración con RHF
HTTP Client	Axios	1.6+	Interceptores, manejo centralizado
Estilos	Tailwind CSS	3+	Utility-first, consistencia de tokens
Componentes Base	shadcn/ui	Latest	Accesible, headless, personalizable
Testing Unitario	Vitest	1+	Nativo Vite, API compatible con Jest
Testing Componentes	Testing Library	14+	Enfoque en comportamiento de usuario
Testing E2E	Playwright	1.40+	Multi-browser, CI-ready
Linting	ESLint	8+	Calidad y consistencia de código
Formateo	Prettier	3+	Formato uniforme, sin debates

### 3.2 React 18+ y Concurrent Features

El uso de React 18+ es obligatorio en todos los proyectos nuevos. Las siguientes características deben ser adoptadas de forma apropiada:

- **`Suspense`**: Utilizado para manejar estados de carga de componentes asíncronos, en combinación con TanStack Query.
- **`useTransition` y `startTransition`**: Adoptados para marcar actualizaciones de estado no urgentes, manteniendo la interfaz responsiva durante renderizados costosos.

- **Server Components (Next.js):** En proyectos Next.js, se debe privilegiar el uso de React Server Components para contenido estático o fetching inicial de datos, reduciendo el JavaScript enviado al cliente.

### 3.3 Next.js para Proyectos con SSR/SSG

Next.js es el meta-framework obligatorio cuando el proyecto requiera:

- **SEO: Páginas de marketing, catálogos de productos, blogs institucionales.**
- **Performance de carga inicial: Aplicaciones con usuarios en redes lentas o dispositivos de bajo rendimiento.**
- **Server-Side Rendering: Datos que deben ser pre-renderizados en el servidor por razones de seguridad o consistencia.**

Para aplicaciones puramente internas (dashboards administrativos, herramientas de gestión sin requisitos de SEO), el uso de Vite + React como SPA es la opción estándar y preferida por su simplicidad operativa.

### 3.4 Vite como Bundler para SPAs

La configuración de Vite debe mantenerse en el archivo `vite.config.ts` en la raíz del proyecto. Las customizaciones deben ser documentadas con comentarios que expliquen la justificativa técnica de cada opción no estándar.

```
``typescript
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
 plugins: [react()],
 resolve: {
 alias: {
 '@': path.resolve(__dirname, './src'),
 },
 },
 build: {
 // Code splitting automático por ruta
 rollupOptions: {
 output: {
 manualChunks: {
 vendor: ['react', 'react-dom'],
 query: ['@tanstack/react-query'],
 },
 },
 },
 },
});
```

```

 },
 },
 },
 },
});
...

```

## 4. Arquitectura de Componentes

### 4.1 Atomic Design

**Santo Pegasus adopta la metodología Atomic Design como sistema de clasificación y organización de componentes.** Esta metodología, propuesta por Brad Frost, establece una jerarquía clara que facilita la reutilización, la consistencia visual y la comunicación entre desarrolladores y diseñadores.

#### #### 4.1.1 Atoms (Átomos)

Son los bloques de construcción más elementales de la interfaz. No pueden descomponerse en componentes más pequeños sin perder su significado funcional. Los átomos no contienen lógica de negocio ni estado derivado de APIs.

**Ejemplos: `Button`, `Input`, `Label`, `Icon`, `Badge`, `Avatar`, `Spinner`.**

```

```tsx
// src/components/atoms/Button/Button.tsx
interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: 'primary' | 'secondary' | 'destructive' | 'ghost';
  size?: 'sm' | 'md' | 'lg';
  isLoading?: boolean;
}
export const Button = ({
  variant = 'primary',
  size = 'md',
  isLoading = false,
  children,
  disabled,
  ...props
}: ButtonProps) => (
  <button
    className={cn(buttonVariants({ variant, size }))}
    disabled={disabled || isLoading}
```

```

    aria-busy={isLoading}
    {...props}
  >
    {isLoading ? <Spinner size="sm" /> : children}
  </button>
);
...

```

4.1.2 Molecules (Moléculas)

Son combinaciones de dos o más átomos que forman una unidad funcional con un propósito único y bien definido. Las moléculas pueden contener estado local simple (por ejemplo, el estado de foco de un campo de búsqueda).

Ejemplos: ``SearchField`` (Input + Icon + Button), ``FormField`` (Label + Input + ErrorMessage), ``UserAvatarWithName`` (Avatar + Text).

4.1.3 Organisms (Organismos)

Son secciones complejas y autónomas de la interfaz, formadas por la composición de moléculas y átomos. Los organismos pueden contener lógica de presentación más compleja y representan regiones reconocibles de la UI. Sin embargo, no deben realizar llamadas directas a APIs; esa responsabilidad pertenece a sus contenedores.

Ejemplos: ``ProductTable``, ``NavigationHeader``, ``UserProfileCard``, ``CheckoutSummary``.

4.1.4 Templates (Plantillas)

Definen la estructura y el layout de una página, estableciendo dónde se posicionarán los organismos, moléculas y átomos. Los templates trabajan con datos de marcador de posición (placeholder), son agnósticos del contenido real y representan el esqueleto visual de una ruta.

Ejemplos: ``DashboardTemplate``, ``AuthTemplate``, ``TwoColumnLayout``.

4.1.5 Pages (Páginas)

Son la instanciación concreta de un Template con datos reales. Las Pages son la capa de composición final, donde se conectan los contenedores de datos con los componentes de presentación. En proyectos Next.js, corresponden a los archivos dentro del directorio ``app/`` o ``pages/``.

4.2 Componentes de Presentación vs. Contenedores

La separación entre componentes de presentación (dumb components) y contenedores (smart components) es una directriz de arquitectura de aplicación obligatoria en Santo Pegasus.

Característica	Componente de Presentación	Contenedor
Responsabilidad	Renderizar UI a partir de props	Obtener datos, manejar estado
Llamadas a API	Nunca	Vía hooks de datos

Estado global	No accede	Accede vía Zustand/Query
Testabilidad	Alta (solo recibe props)	Media (requiere mocks)
Reutilización	Alta	Baja (acoplado al dominio)
Sufijo de archivo	`ProductCard.tsx`	`ProductCardContainer.tsx`

5. Gestión del Estado

5.1 Taxonomía del Estado

Antes de elegir una solución de estado, es obligatorio clasificar la naturaleza del estado a gestionar:

Tipo de Estado	Descripción	Solución Recomendada
Estado Local	UI efímera (modal abierto, tab activo)	`useState`, `useReducer`
Estado del Servidor	Datos provenientes de APIs	TanStack Query
Estado Global del Cliente	Auth, preferencias, tema, carrito	Zustand
Estado de URL	Filtros, paginación, búsqueda	`useSearchParams`
Estado de Formulario	Inputs, validaciones, envío	React Hook Form

5.2 Zustand para Estado Global del Cliente

Zustand es la solución estándar para estado global que no proviene del servidor. Su adopción está justificada por su mínimo boilerplate, ausencia de providers anidados y excelente integración con TypeScript.

5.2.1 Reglas de Uso de Zustand

- 1. Una store por dominio de negocio:** No se debe crear una única store global monolítica. Cada dominio (autenticación, UI global, carrito, notificaciones) debe tener su propia store.
- 2. Interfaz tipada obligatoria:** Toda store debe definir explícitamente su interfaz TypeScript.
- 3. Acciones junto al estado:** Las funciones que modifican el estado deben definirse dentro de la propia store.
- 4. Selectores para prevenir re-renders:** Siempre utilice selectores específicos al consumir la store.

```
``typescript
// src/store/auth.store.ts
interface AuthState {
```

```

user: User | null;
accessToken: string | null;
isAuthenticated: boolean;
setUser: (user: User, token: string) => void;
clearAuth: () => void;
}

export const useAuthStore = create<AuthState>((set) => ({
  user: null,
  accessToken: null,
  isAuthenticated: false,
  setUser: (user, accessToken) =>
    set({ user, accessToken, isAuthenticated: true }),
  clearAuth: () =>
    set({ user: null, accessToken: null, isAuthenticated: false }),
}));

// Uso con selector (previene re-renders innecesarios)
const user = useAuthStore((state) => state.user);
...

```

5.3 TanStack Query para Estado del Servidor

TanStack Query (anteriormente React Query) es la solución obligatoria para todo estado que proviene de APIs externas. Su adopción elimina la necesidad de `useEffect` para fetch de datos y proporciona cache, revalidación, estados de carga y error de forma automática.

5.3.1 Reglas de Uso de TanStack Query

1. Nunca usar `useEffect` para fetch de datos: Todo consumo de API debe realizarse a través de `useQuery` o `useMutation`.

2. Query Keys estructuradas: Las query keys deben seguir una estructura jerárquica consistente que refleje el dominio y los parámetros.

3. Hooks de dominio: Los calls a `useQuery` / `useMutation` deben encapsularse en hooks de dominio, no usarse directamente en los componentes.

```

``typescript
// src/hooks/queries/useProducts.ts
// Convención de query keys: [entidad, operación, ...parámetros]
export const productKeys = {
  all: ['products'] as const,
  lists: () => [...productKeys.all, 'list'] as const,
  list: (filters: ProductFilters) => [...productKeys.lists(), filters] as const,

```

```

    detail: (id: string) => [...productKeys.all, 'detail', id] as const,
  };
export const useProducts = (filters: ProductFilters) =>
  useQuery({
    queryKey: productKeys.list(filters),
    queryFn: () => productService.getAll(filters),
    staleTime: 5 * 60 * 1000, // 5 minutos
  });
export const useProduct = (id: string) =>
  useQuery({
    queryKey: productKeys.detail(id),
    queryFn: () => productService.getById(id),
    enabled: !!id,
  });
export const useCreateProduct = () => {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: productService.create,
    onSuccess: () => {
      // Invalida el cache de la lista tras crear
      queryClient.invalidateQueries({ queryKey: productKeys.lists() });
    },
  });
};
...
---
```

6. Consumo de APIs

6.1 Cliente HTTP Centralizado

Todo consumo de las APIs de Santo Pegasus debe realizarse a través de una instancia centralizada de Axios, configurada con interceptores para el manejo automático de autenticación, refresh de tokens y errores globales. Queda terminantemente prohibido instanciar Axios directamente en componentes o hooks de dominio.

6.2 Configuración del Cliente Axios

```

``typescript
// src/lib/api/client.ts
```

```

import axios, { AxiosInstance, InternalAxiosRequestConfig } from 'axios';
import { useAuthStore } from '@store/auth.store';
const apiClient: AxiosInstance = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 15000,
  headers: {
    'Content-Type': 'application/json',
  },
  // Las cookies httpOnly son enviadas automáticamente
  withCredentials: true,
});
// Interceptor de Request: inyecta el token JWT
apiClient.interceptors.request.use(
  (config: InternalAxiosRequestConfig) => {
    const token = useAuthStore.getState().accessToken;
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);
// Interceptor de Response: manejo de errores y refresh token
apiClient.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
      try {
        // Refresh token via cookie httpOnly — no expone el token en JS
        const { data } = await axios.post('/auth/refresh', {}, { withCredentials: true });
        useAuthStore.getState().setUser(data.user, data.accessToken);
        originalRequest.headers.Authorization = `Bearer ${data.accessToken}`;
        return apiClient(originalRequest);
      } catch {
        useAuthStore.getState().clearAuth();
      }
    }
  }
);

```



```

    window.location.href = '/login';
  }
}
// Manejo global de errores por código HTTP
handleHttpError(error.response?.status);
return Promise.reject(error);
}
);
...

```

6.3 Tipado Fuerte de Requests y Responses

Todo servicio de dominio debe tipar explícitamente sus parámetros de entrada y sus respuestas. Nunca se debe utilizar el tipo `any` en operaciones de red.

```

``typescript
// src/lib/api/services/product.service.ts
import { apiClient } from '@lib/api/client';
import type { Product, CreateProductDto, ProductFilters, PaginatedResponse } from
'@/types/product.types';

export const productService = {
  getAll: async (filters: ProductFilters): Promise<PaginatedResponse<Product>> => {
    const { data } = await apiClient.get<PaginatedResponse<Product>>('/products', {
      params: filters,
    });
    return data;
  },
  getById: async (id: string): Promise<Product> => {
    const { data } = await apiClient.get<Product>(`/products/${id}`);
    return data;
  },
  create: async (payload: CreateProductDto): Promise<Product> => {
    const { data } = await apiClient.post<Product>('/products', payload);
    return data;
  },
};
...

```

6.4 Manejo Global de Errores HTTP

La función `handleHttpError` debe cubrir los códigos de respuesta estándar y disparar notificaciones globales usando el sistema de toast del Design System.

```
``typescript
// src/lib/api/error-handler.ts
import { toast } from '@components/ui/toast';
const HTTP_ERROR_MESSAGES: Record<number, string> = {
  400: 'Solicitud inválida. Verifique los datos enviados.',
  403: 'Usted no tiene permisos para realizar esta acción.',
  404: 'El recurso solicitado no fue encontrado.',
  409: 'Conflicto: el recurso ya existe o está en uso.',
  422: 'Los datos enviados no pudieron ser procesados.',
  500: 'Error interno del servidor. Contacte al soporte técnico.',
  503: 'Servicio temporalmente no disponible. Intente nuevamente.',
};
export const handleHttpError = (status?: number): void => {
  if (!status) return;
  const message = HTTP_ERROR_MESSAGES[status] ?? 'Ocurrió un error inesperado.';
  toast.error(message);
};
...
---
```

7. Estándares de Código

7.1 Nomenclatura de Archivos y Directorios

La consistencia en la nomenclatura es obligatoria y será verificada en el proceso de Code Review.

Tipo de Artefacto	Convención	Ejemplo
Componentes React	PascalCase	<code>ProductCard.tsx</code>
Custom Hooks	camelCase con prefijo <code>use</code>	<code>useProductList.ts</code>
Funciones utilitarias	camelCase	<code>formatCurrency.ts</code>
Servicios de API	camelCase con sufijo <code>.service</code>	<code>product.service.ts</code>
Stores Zustand	camelCase con sufijo <code>.store</code>	<code>auth.store.ts</code>
Tipos e Interfaces	PascalCase con sufijo <code>.types</code>	<code>product.types.ts</code>
Constantes	SCREAMING_SNAKE_CASE	<code>API_ENDPOINTS.ts</code>

Archivos de Test	Mismo nombre + `.test` o `.spec`	`ProductCard.test.tsx`
Directorios	kebab-case	`product-card/`

7.2 Estructura de Carpetas del Proyecto

...

src/

app/ # Rutas (Next.js App Router) o pages (Vite)

components/

atoms/ # Componentes elementales del Design System

molecules/ # Composiciones de átomos

organisms/ # Secciones complejas de UI

templates/ # Layouts de página

ui/ # Re-exportaciones de shadcn/ui

hooks/

queries/ # Hooks de TanStack Query por dominio

mutations/ # Hooks de useMutation por dominio

lib/

api/ # Cliente Axios, servicios, error-handler

utils/ # Funciones utilitarias puras

store/ # Stores de Zustand

types/ # Interfaces y tipos globales de TypeScript

styles/ # Configuraciones globales de Tailwind y CSS

tests/

unit/ # Tests unitarios de utils y hooks

components/ # Tests de componentes con Testing Library

e2e/ # Tests Playwright

constants/ # Constantes globales de la aplicación

...

7.3 ESLint — Configuración Obligatoria

El proyecto debe incluir una configuración ESLint que extienda las siguientes reglas base:

```
``json
```

```
{
```

```
"extends": [
```

```
"eslint:recommended",
```

```
"plugin:@typescript-eslint/recommended",
```

```
"plugin:react-hooks/recommended",
```

```

    "plugin:jsx-a11y/recommended",
    "plugin:import/typescript",
    "prettier"
  ],
  "rules": {
    "@typescript-eslint/no-explicit-any": "error",
    "@typescript-eslint/no-unused-vars": "error",
    "react-hooks/exhaustive-deps": "error",
    "import/order": ["error", { "newlines-between": "always" }],
    "no-console": ["warn", { "allow": ["error"] }]
  }
}
...

```

7.4 Prettier — Configuración Estándar

```

```.json
{
 "semi": true,
 "trailingComma": "all",
 "singleQuote": true,
 "printWidth": 100,
 "tabWidth": 2,
 "arrowParens": "always"
}
...

```

*El formateo automático con Prettier debe ejecutarse como pre-commit hook mediante Husky + lint-staged, garantizando que ningún código mal formateado llegue al repositorio.*

---

## 8. Estilo y Design System

### 8.1 Tailwind CSS como Sistema de Tokens

*Tailwind CSS es el sistema de estilos estándar de Santo Pegasus. Su filosofía utility-first elimina los problemas de especificidad de CSS, facilita la revisión de estilos en Code Review y garantiza la consistencia visual a través de tokens predefinidos.*

La personalización del tema debe realizarse exclusivamente en `tailwind.config.ts`, donde se definen los tokens de diseño del proyecto.

```

```.typescript

```

```

// tailwind.config.ts
import type { Config } from 'tailwindcss';

const config: Config = {
  content: ['./src/**/*.ts', './src/**/*.tsx'],
  theme: {
    extend: {
      colors: {
        brand: {
          50: '#f0f4ff',
          500: '#3b5bdb',
          900: '#1a237e',
        },
        neutral: {
          50: '#f8f9fa',
          500: '#868e96',
          900: '#212529',
        },
        success: '#2f9e44',
        warning: '#f59f00',
        error: '#e03131',
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
        mono: ['JetBrains Mono', 'monospace'],
      },
      spacing: {
        '4.5': '1.125rem',
        '18': '4.5rem',
      },
    },
  },
  plugins: [require('tailwindcss-animate')],
};

export default config;

```

8.2 shadcn/ui como Biblioteca de Componentes Base

shadcn/ui es la biblioteca de componentes base estándar de Santo Pegasus. Sus componentes son instalados directamente en el repositorio del proyecto (no como dependencia de node_modules), lo que permite su personalización completa sin conflictos de versioning.

8.2.1 Reglas de Uso de shadcn/ui

- 1. Los componentes de shadcn/ui deben residir en `src/components/ui/` y no deben ser modificados directamente. En su lugar, créese un componente envolvente en la capa de átomos que extienda el comportamiento deseado.
- 2. Todo componente de shadcn/ui utilizado debe tener su accesibilidad verificada conforme a la Sección 8.3.

8.3 Accesibilidad — WCAG 2.1 Nivel AA

La conformidad con el estándar WCAG 2.1 Nivel AA es un requisito no negociable en Santo Pegasus. Las siguientes directrices son de aplicación obligatoria:

Criterio	Requisito	Herramienta de Verificación
Contraste de Color	Ratio mínimo 4.5:1 (texto normal), 3:1 (texto grande)	Figma Contrast Plugin, axe DevTools
Navegación por Teclado	Todos los elementos interactivos deben ser accesibles vía Tab	Prueba manual + `eslint-plugin-jsx-a11y`
Textos Alternativos	`alt` descriptivo en todas las imágenes significativas	`eslint-plugin-jsx-a11y`
Labels en Formularios	Cada campo debe tener un ` <label>` asociado</label>	`eslint-plugin-jsx-a11y`
Roles ARIA	Uso correcto de `role`, `aria-label`, `aria-describedby`	axe DevTools, revisión en PR
Gestión de Foco	Modales y drawers deben atrapar el foco correctamente	Prueba manual
Skip Links	Páginas con navegación deben incluir "Saltar al contenido"	Revisión en PR

La regla `plugin:jsx-a11y/recommended` en ESLint automatiza la detección de la mayoría de estas violaciones en tiempo de desarrollo.

9. Gestión de Formularios

9.1 React Hook Form + Zod

La gestión de formularios en Santo Pegasus se basa en la combinación de React Hook Form para el control del estado del formulario y Zod para la definición del esquema de validación. Esta combinación garantiza una única fuente de verdad para la lógica de validación, que además genera tipos TypeScript automáticamente.

9.2 Patrón de Implementación

```
``typescript
// src/types/product.types.ts
import { z } from 'zod';
export const createProductSchema = z.object({
  name: z.string()
    .min(3, 'El nombre debe tener al menos 3 caracteres.')
    .max(100, 'El nombre no puede superar 100 caracteres.'),
  price: z.number()
    .positive('El precio debe ser un valor positivo.')
    .multipleOf(0.01, 'El precio debe tener máximo dos decimales.'),
  categoryId: z.string().uuid('Seleccione una categoría válida.'),
  description: z.string().max(500).optional(),
});
// El tipo se infiere automáticamente del schema
export type CreateProductDto = z.infer<typeof createProductSchema>;
...

``tsx
// src/components/organisms/ProductForm/ProductForm.tsx
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { createProductSchema, CreateProductDto } from '@types/product.types';
import { useCreateProduct } from '@hooks/mutations/useCreateProduct';
export const ProductForm = () => {
  const { mutate: createProduct, isPending } = useCreateProduct();
  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm<CreateProductDto>({
    resolver: zodResolver(createProductSchema),
  });
```

```

const onSubmit = (data: CreateProductDto) => createProduct(data);
return (
  <form onSubmit={handleSubmit(onSubmit)} noValidate>
    <FormField
      label="Nombre del Producto"
      error={errors.name?.message}
    >
      <Input
        {...register('name')}
        aria-describedby={errors.name ? 'name-error' : undefined}
      />
    </FormField>
    <Button type="submit" isLoading={isPending}>
      Crear Producto
    </Button>
  </form>
);
};
...

```

9.3 Reglas de Validación

1. Validación en el cliente es una mejora de UX, no de seguridad. Los schemas Zod del front-end deben ser equivalentes a las validaciones del back-end (Bean Validation en Java), pero nunca los reemplazan.
2. Mensajes de error en español y orientados al usuario, no al desarrollador.
3. `noValidate` en todos los formularios: Deshabilitar la validación nativa del navegador para mantener control total sobre el feedback de error.

10. Estrategia y Cobertura de Pruebas

La calidad del código de front-end debe ser asegurada por un enfoque multifacético que cubra la lógica de negocio, el comportamiento de los componentes y los flujos críticos del usuario.

10.1 Cobertura Mínima Obligatoria

La cobertura mínima de 80% en pruebas unitarias es obligatoria y verificada automáticamente en el pipeline de CI/CD. El build falla si la cobertura cae por debajo de este umbral.

10.2 Pirámide de Pruebas

10.2.1 Pruebas Unitarias — Vitest

Enfocadas en lógica pura: funciones utilitarias, transformadores de datos, schemas Zod y hooks de negocio aislados de la UI.

```
``typescript
// src/lib/utils/formatCurrency.test.ts
import { describe, it, expect } from 'vitest';
import { formatCurrency } from './formatCurrency';
describe('formatCurrency', () => {
  it('formatea correctamente un valor positivo en BRL', () => {
    expect(formatCurrency(1999.99, 'BRL')).toBe('R$ 1.999,99');
  });
  it('formatea cero como R$ 0,00', () => {
    expect(formatCurrency(0, 'BRL')).toBe('R$ 0,00');
  });
  it('lanza error con valor negativo', () => {
    expect(() => formatCurrency(-1, 'BRL')).toThrow('Valor inválido');
  });
});
``
```

10.2.2 Pruebas de Componentes — Testing Library

Enfocadas en el comportamiento del usuario, no en los detalles de implementación. Las pruebas deben consultar elementos por roles ARIA, labels y texto visible, jamás por selectores CSS o IDs internos.

```
``tsx
// src/components/organisms/ProductForm/ProductForm.test.tsx
import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { ProductForm } from './ProductForm';
describe('ProductForm', () => {
  it('muestra error de validación al enviar nombre vacío', async () => {
    render(<ProductForm />);
    await userEvent.click(screen.getByRole('button', { name: /crear producto/i }));
    expect(await screen.findByText(/al menos 3 caracteres/i)).toBeInTheDocument();
  });
  it('llama a onSubmit con datos válidos', async () => {
    const mockCreate = vi.fn();
    render(<ProductForm onSubmit={mockCreate} />);
    await userEvent.type(screen.getByLabelText(/nombre/i), 'Producto de prueba');
```

```

    await userEvent.click(screen.getByRole('button', { name: /crear producto/i }));
    await waitFor(() => expect(mockCreate).toHaveBeenCalledTimes());
  });
});
...

```

10.2.3 Pruebas E2E — Playwright

Obligatorias para los flujos críticos del negocio. Ejecutadas en el pipeline de CI contra un ambiente de Staging.

Flujos que requieren cobertura E2E obligatoria:

- Autenticación completa (login, refresh, logout)
- Flujo de checkout / acción principal del negocio
- Operaciones CRUD del recurso central del sistema
- Recuperación de contraseña

```

``typescript
// src/tests/e2e/auth.spec.ts
import { test, expect } from '@playwright/test';
test.describe('Autenticación', () => {
  test('usuario puede iniciar sesión con credenciales válidas', async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Correo electrónico').fill('usuario@santopegasus.com');
    await page.getByLabel('Contraseña').fill('Senha@Segura123');
    await page.getByRole('button', { name: /iniciar sesión/i }).click();
    await expect(page).toHaveURL('/dashboard');
    await expect(page.getByRole('heading', { name: /bienvenido/i })).toBeVisible();
  });
});
...

```

10.3 Configuración de Vitest

```

``typescript
// vitest.config.ts
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,

```

```

environment: 'jsdom',
setupFiles: './src/tests/setup.ts',
coverage: {
  provider: 'v8',
  thresholds: {
    lines: 80,
    functions: 80,
    branches: 80,
    statements: 80,
  },
  exclude: ['src/components/ui/', 'src/types/'],
},
},
});
...
---
```

11. Proceso de Code Review

11.1 Reglas para el Autor del PR

1. **Tamaño máximo del PR:** Todo Pull Request debe tener un máximo de 400 líneas de código modificado, excluyendo archivos de configuración, tipos generados automáticamente y assets. PRs que excedan este límite deben ser divididos en unidades lógicas más pequeñas.
2. **Auto-revisión obligatoria:** El autor debe revisar su propio PR en la interfaz de GitHub/GitLab antes de solicitar revisión, asegurándose de que no haya código de debug, `console.log` o `TODO` no documentados.
3. **Template del PR:** Todo PR debe completar el template oficial (ver 11.3).
4. **CI verde:** No se permite solicitar revisión con el pipeline de CI fallando.
5. **Screenshots o videos:** Toda alteración visual debe incluir capturas de pantalla comparativas (antes/después) o una grabación de la interacción.

11.2 Reglas para el Revisor

1. **Aprobaciones requeridas:** Todo Pull Request exige al menos 2 aprobaciones de miembros Senior o Semi-Senior antes del merge, siguiendo el mismo criterio establecido en la Guía de Ingeniería Back-end.

2. Tono y Etiqueta: La comunicación debe estar siempre pautada por la empatía y la educación. Los comentarios deben estar orientados exclusivamente a la mejora del código. Se recomienda el uso de preguntas sugestivas: "¿Consideraste usar ``useMemo`` aquí para evitar el recálculo?" en lugar de imperativos como "Usa ``useMemo``".

3. Criterios de Revisión Front-end: El revisor debe validar:

Criterio	Descripción
Correctitud	El componente se comporta como se espera en todos los estados (loading, error, vacío, datos)
Accesibilidad	Uso correcto de roles ARIA, labels, contraste y navegación por teclado
Performance	Ausencia de renders innecesarios, uso correcto de <code>`useMemo`/`useCallback`</code>
Seguridad	Sin datos sensibles en <code>localStorage</code> , sin XSS, sin <code>`dangerouslySetInnerHTML`</code> no sanitizado
Tipado	Ausencia de <code>`any`</code> , tipos explícitos en todas las funciones públicas
Pruebas	Cobertura adecuada de los casos feliz, error y borde
Guía de Ingeniería	Conformidad con las directrices de este documento

11.3 Template Oficial del PR

```markdown

**Descripción**

<!-- ¿Qué cambios fueron realizados en este PR? -->

**Motivación**

<!-- ¿Por qué estos cambios son necesarios? Referencia al ticket/issue. -->

Closes: #[número del issue]

**Cómo Probar**

<!-- Instrucciones paso a paso para validar los cambios manualmente. -->

- 1.
- 2.
- 3.

**Screenshots**

<!-- Para cambios visuales, incluir capturas antes/después. -->

| Antes | Después |
|-------|---------|
| img   | img     |

## Checklist del Autor

- ☐ Código revisado por el propio autor antes de solicitar review
- ☐ Pruebas unitarias/componentes añadidas o actualizadas
- ☐ Cobertura de pruebas  $\geq 80\%$
- ☐ Accesibilidad verificada (teclado + contraste)
- ☐ Sin ``console.log`` ni código de debug
- ☐ Sin ``any`` en TypeScript
- ☐ Pipeline de CI pasando (lint + tests + build)
- ☐ PR tiene menos de 400 líneas modificadas
- ☐ Screenshots incluidos (si aplica)

...

---

## 12. Control de Versiones

### 12.1 GitFlow

Santo Pegasus adopta el modelo GitFlow de forma idéntica a lo establecido en la Guía de Ingeniería Back-end, garantizando la coherencia del proceso de desarrollo entre todas las capas del sistema.

| Rama                    | Propósito                           | Merge Origin                                     | Merge Target                                 |
|-------------------------|-------------------------------------|--------------------------------------------------|----------------------------------------------|
| <code>`main`</code>     | Código en producción                | <code>`release/`</code> , <code>`hotfix/`</code> | —                                            |
| <code>`develop`</code>  | Integración de features             | <code>`feature/`</code> , <code>`fix/`</code>    | <code>`release/`</code>                      |
| <code>`feature/`</code> | Nuevas funcionalidades              | <code>`develop`</code>                           | <code>`develop`</code>                       |
| <code>`fix/`</code>     | Correcciones no urgentes            | <code>`develop`</code>                           | <code>`develop`</code>                       |
| <code>`release/`</code> | Preparación para deploy             | <code>`develop`</code>                           | <code>`main`</code> + <code>`develop`</code> |
| <code>`hotfix/`</code>  | Correcciones urgentes en producción | <code>`main`</code>                              | <code>`main`</code> + <code>`develop`</code> |

### 12.2 Conventional Commits

El uso de **Conventional Commits** es obligatorio en todos los proyectos del Chapter de Front-end, siguiendo la misma especificación adoptada en el back-end. El formato es validado automáticamente por Commitlint configurado como commit-msg hook en Husky.

```
...

<tipo>(<escopo>): <descripción corta en imperativo>
[cuerpo opcional]
[footer opcional: BREAKING CHANGE, Closes #123]
...
```

**Tipos permitidos:**

| Tipo       | Uso                                              |
|------------|--------------------------------------------------|
| `feat`     | Nueva funcionalidad visible para el usuario      |
| `fix`      | Corrección de bug                                |
| `refactor` | Refactorización sin cambio de comportamiento     |
| `style`    | Cambios de formato/estilos sin impacto en lógica |
| `test`     | Adición o corrección de pruebas                  |
| `chore`    | Tareas de mantenimiento (deps, config)           |
| `docs`     | Documentación                                    |
| `perf`     | Mejoras de performance                           |
| `ci`       | Cambios en pipelines de CI/CD                    |

**Ejemplos:**

```
...

feat(product): añade filtro por categoría en listado de productos
fix(auth): corrige loop infinito en refresh token cuando el server retorna 401
perf/dashboard): implementa lazy loading en tabla de transacciones
test(product-form): añade tests para estado de error en formulario de creación
...

```

### 13. Performance y Web Vitals

#### 13.1 Web Vitals como Criterios de Calidad

Las métricas Core Web Vitals de Google son criterios de aceptación técnica en Santo Pegasus. Ninguna funcionalidad puede ser desplegada a producción si degrada significativamente estas métricas por debajo de los umbrales establecidos.

| Métrica                        | Descripción                                     | Umbral "Good" | Umbral Máximo |
|--------------------------------|-------------------------------------------------|---------------|---------------|
| LCP (Largest Contentful Paint) | Tiempo de carga del elemento más grande visible | $\leq 2.5s$   | $\leq 4.0s$   |
| FID (First Input Delay) / INP  | Latencia de respuesta a la primera interacción  | $\leq 100ms$  | $\leq 300ms$  |
| CLS (Cumulative Layout Shift)  | Estabilidad visual del layout durante la carga  | $\leq 0.1$    | $\leq 0.25$   |
| TTFB (Time to First Byte)      | Tiempo de respuesta del servidor                | $\leq 800ms$  | $\leq 1.8s$   |
| FCP (First Contentful Paint)   | Primer renderizado de contenido                 | $\leq 1.8s$   | $\leq 3.0s$   |

**Las métricas deben ser monitoreadas en producción utilizando herramientas como Vercel Analytics, Google Search Console o Datadog RUM.**

## 13.2 Lazy Loading y Code Splitting

**Code splitting es obligatorio para rutas de la aplicación. Ninguna ruta debe ser incluida en el bundle principal (initial chunk).**

```
``tsx
// Correcto: lazy loading por ruta
import { lazy, Suspense } from 'react';
const ProductsPage = lazy(() => import('@app/products/ProductsPage'));
const DashboardPage = lazy(() => import('@app/dashboard/DashboardPage'));
const SettingsPage = lazy(() => import('@app/settings/SettingsPage'));
export const AppRouter = () => (
 <Suspense fallback={<PageLoadingSpinner />}>
 <Routes>
 <Route path="/products" element={<ProductsPage />} />
 <Route path="/dashboard" element={<DashboardPage />} />
 <Route path="/settings" element={<SettingsPage />} />
 </Routes>
 </Suspense>
);
...

```

## 13.3 Optimización de Imágenes

- **Formato:** Usar siempre WebP como formato primario, con fallback para JPEG/PNG.

- **Dimensiones:** Nunca servir imágenes con dimensiones mayores que su tamaño de display.
- `loading="lazy"`: Obligatorio en todas las imágenes que no estén en el viewport inicial.
- `width` y `height`` explícitos: Obligatorio para prevenir CLS durante la carga.
- **Next.js Image:** En proyectos Next.js, el componente `<Image />` es obligatorio. Queda prohibido el uso de la etiqueta `<img>` nativa para imágenes de contenido.

### 13.4 Memoización y Prevención de Re-renders

El uso de `React.memo``, `useMemo`` y `useCallback`` debe ser justificado y medido, no aplicado de forma prematura. La regla es: primero identificar el problema con las DevTools de React Profiler, luego aplicar la memoización.

```
``tsx
// Uso justificado: componente hijo que recibe callback y re-renderiza en cascada
const ProductRow = React.memo(({ product, onSelect }: ProductRowProps) => (
 <tr onClick={() => onSelect(product.id)}>
 <td>{product.name}</td>
 </tr>
));
// En el padre, el callback debe ser estabilizado
const handleSelect = useCallback((id: string) => {
 setSelectedProductId(id);
}, []); // sin dependencias, siempre la misma referencia

```

## 14. Seguridad en el Front-end

### 14.1 Filosofía de Seguridad

El front-end opera en un entorno inherentemente no confiable: el navegador del usuario. Toda la lógica de autorización y validación de datos sensibles debe residir en el back-end. El front-end implementa medidas de seguridad como defensa en profundidad, nunca como única línea de protección.

### 14.2 Protección contra XSS (Cross-Site Scripting)

Práctica	Regla
<code>dangerouslySetInnerHTML`</code>	Prohibido salvo en casos excepcionales documentados con sanitización previa vía DOMPurify
Contenido dinámico del usuario	Siempre renderizado como texto ( <code>{variable}`</code> ), nunca como HTML



URLs dinámicas	Validar que comienzan con `https://` antes de usarlas en `href` o `src`
Cabecera CSP	El servidor debe enviar una Content Security Policy estricta

```

`tsx
// Prohibido: vulnerable a XSS
<div dangerouslySetInnerHTML={{ __html: userProvidedContent }} />
// Correcto: con sanitización
import DOMPurify from 'dompurify';
const safeHtml = DOMPurify.sanitize(userProvidedContent);
<div dangerouslySetInnerHTML={{ __html: safeHtml }} />
// Mejor opción: renderizar como texto siempre que sea posible
<div>{userProvidedContent}</div>
...

```

### 14.3 Gestión Segura de Tokens de Autenticación

El almacenamiento de tokens de autenticación sigue una política estricta en Santo Pegasus:

Método de Almacenamiento	Uso	Justificativa
`httpOnly` Cookie	Refresh Token	Inaccesible para JavaScript; protege contra XSS
Memoria (Zustand store)	Access Token (corta duración)	No persiste entre sesiones; limpiado al cerrar tab
`localStorage`	Prohibido para tokens	Accesible por cualquier script; vulnerable a XSS
`sessionStorage`	Prohibido para tokens sensibles	Accesible por JavaScript; inseguro contra XSS

### 14.4 Protección contra CSRF

- Las cookies de sesión deben ser emitidas por el back-end con los atributos `SameSite=Strict` o `SameSite=Lax` y `Secure`.
- Para operaciones mutantes (POST, PUT, DELETE), el back-end debe validar el origen de la petición.
- En formularios que requieran protección adicional, se debe usar el patrón de CSRF Token generado por el servidor.

### 14.5 Principio de Mínimo Privilegio en el Front-end

- No exponer información sensible en URLs: IDs de usuarios, tokens o datos de sesión no deben aparecer en parámetros de query string.

- **Feature flags por rol:** Los elementos de UI para acciones privilegiadas deben ocultarse para usuarios sin el rol correspondiente. Sin embargo, la autorización real siempre debe verificarse en el back-end.
- **Logging seguro:** Nunca registrar en ``console.error`` información sensible como tokens, datos personales o respuestas completas de la API.

---

## 15. CI/CD Front-end

### 15.1 Filosofía del Pipeline

El pipeline de CI/CD del front-end debe garantizar que ningún código que falle en lint, pruebas o build llegue a producción. La automatización es la primera línea de control de calidad, complementando el proceso de Code Review humano.

### 15.2 Etapas del Pipeline

El pipeline de CI/CD se compone de las siguientes etapas ejecutadas secuencialmente:

...

```
LINT & ► UNIT & ► BUILD ► E2E en ► DEPLOY
TYPECHECK COMPONENT DOCKER STAGING PROD
 TESTS
```

...

### 15.3 Pipeline GitHub Actions — Definición Completa

```yaml

.github/workflows/ci.yml

```
name: CI/CD — Front-end
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [develop]
jobs:
  lint-and-typecheck:
    name: Lint & TypeCheck
```

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4

with:

node-version: '20'

cache: 'npm'

- run: npm ci
- run: npm run lint
- run: npm run typecheck

test:

name: Unit & Component Tests

needs: lint-and-typecheck

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4

with:

node-version: '20'

cache: 'npm'

- run: npm ci
- run: npm run test:coverage
- name: Upload Coverage Report
- uses: codecov/codecov-action@v4

build:

name: Docker Build

needs: test

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Build Docker Image
- run: docker build -t santopegasus/\${{ github.event.repository.name }}:\${{ github.sha }} .
- name: Push to Registry
- if: github.ref == 'refs/heads/develop' || github.ref == 'refs/heads/main'
- run: |
- docker push santopegasus/\${{ github.event.repository.name }}:\${{ github.sha }}

e2e:

name: E2E Tests (Playwright)

needs: build

runs-on: ubuntu-latest

if: github.ref == 'refs/heads/develop' || github.ref == 'refs/heads/main'

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
with: { node-version: '20', cache: 'npm' }
- run: npm ci
- run: npx playwright install --with-deps
- run: npm run test:e2e

env:

PLAYWRIGHT_BASE_URL: \${ secrets.STAGING_URL }

- uses: actions/upload-artifact@v4

if: failure()

with:

name: playwright-report

path: playwright-report/

deploy-staging:

name: Deploy to Staging

needs: e2e

if: github.ref == 'refs/heads/develop'

runs-on: ubuntu-latest

steps:

- name: Deploy to Staging Environment
run: echo "Deploy automatizado al ambiente de Staging"

deploy-production:

name: Deploy to Production

needs: e2e

if: github.ref == 'refs/heads/main'

runs-on: ubuntu-latest

environment: production

steps:

- name: Deploy to Production
run: echo "Deploy a Producción con aprobación manual requerida"

...

15.4 Dockerfile para Front-end

El uso de Docker es obligatorio para garantizar la inmutabilidad del artefacto entre los ambientes de Staging y Producción, siguiendo el mismo principio establecido en la Guía de Ingeniería Back-end.

```
```dockerfile
```

### **Dockerfile — Multi-stage build para SPA con Vite**

```
FROM node:20-alpine AS builder
```

```
WORKDIR /app
```

```
COPY package.json ./
```

```
RUN npm ci --frozen-lockfile
```

```
COPY . .
```

```
RUN npm run build
```

### **Stage de producción con Nginx**

```
FROM nginx:1.25-alpine AS production
```

### **Copia la configuración de Nginx para SPA (manejo de rutas client-side)**

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

### **Copia el build generado por Vite**

```
COPY --from=builder /app/dist /usr/share/nginx/html
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
...
```

```
```nginx
```

nginx.conf — Configuración para SPA React

```
server {
```

```
    listen 80;
```

```
    root /usr/share/nginx/html;
```

```
    index index.html;
```

```
    # Redirecciona todas las rutas al index.html (client-side routing)
```

```
    location / {
```

```
        try_files $uri $uri/ /index.html;
```

```

}
# Cabeceras de seguridad
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline';" always;

# Cache de assets estáticos (Vite genera hashes en los nombres)
location ~ \.(js|css|png|jpg|jpeg|webp|svg|ico|woff2)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
}
...

```

15.5 Variables de Entorno en el Front-end

Las variables de entorno en proyectos Vite deben seguir el prefijo ``VITE_`` para ser expuestas al cliente. En proyectos Next.js, el prefijo es ``NEXT_PUBLIC_``.

Reglas obligatorias:

- **Nunca incluir secretos en variables de entorno del front-end. Toda variable ``VITE_`` o ``NEXT_PUBLIC_`` es embebida en el bundle y visible para cualquier usuario.**
- El archivo ``.env.local`` debe estar obligatoriamente listado en ``.gitignore``.
- Las variables de entorno de Staging y Producción deben ser gestionadas en la plataforma de CI/CD (GitHub Actions Secrets, AWS Secrets Manager), nunca en archivos commiteados al repositorio.

```
```bash
```

## **.env.example — Archivo de referencia (commitado al repositorio)**

```
VITE_API_BASE_URL=https://api.santopegasus.com
```

```
VITE_APP_ENV=production
```

```
VITE_SENTRY_DSN=
```

## **.env.local — Configuración local del desarrollador (en .gitignore)**

```
VITE_API_BASE_URL=http://localhost:8080
```

```
VITE_APP_ENV=development
```

...

---

## Apéndice A — Checklist de Nuevo Proyecto

Utilice este checklist al iniciar un nuevo proyecto front-end en Santo Pegasus:

- ☐ Stack tecnológico configurado conforme a la Sección 3
- ☐ Estructura de carpetas conforme a la Sección 7.2
- ☐ ESLint y Prettier configurados con las reglas estándar (Sección 7.3 y 7.4)
- ☐ Husky + lint-staged + Commitlint configurados
- ☐ Cliente Axios centralizado con interceptores (Sección 6.2)
- ☐ TanStack Query Provider configurado en la raíz de la aplicación
- ☐ Stores Zustand organizados por dominio (Sección 5.2)
- ☐ Tailwind CSS con tokens de diseño personalizados (Sección 8.1)
- ☐ shadcn/ui instalado y componentes base en `src/components/ui/`
- ☐ Vitest configurado con umbral de cobertura del 80% (Sección 10.3)
- ☐ Playwright configurado para E2E
- ☐ Dockerfile multi-stage creado (Sección 15.4)
- ☐ Pipeline de CI/CD configurado (Sección 15.3)
- ☐ Variables de entorno documentadas en `.env.example`
- ☐ `README.md` con instrucciones de setup local

---

## Apéndice B — Glosario

Término	Definición
Atomic Design	Metodología de organización de componentes en átomos, moléculas, organismos, templates y páginas
Bundle	Archivo JavaScript resultante del proceso de build, enviado al navegador
CLS	Cumulative Layout Shift — métrica de estabilidad visual del layout
Code Splitting	Técnica de división del bundle en chunks cargados bajo demanda
CSP	Content Security Policy — política de seguridad HTTP que restringe recursos cargables
DTO	Data Transfer Object — objeto que define la forma de los datos en la comunicación con la API
E2E	End-to-End — pruebas que simulan el flujo completo del usuario en el navegador real

FCP	First Contentful Paint — primer momento en que el browser renderiza contenido
FID/INP	First Input Delay / Interaction to Next Paint — métricas de responsividad a la interacción
HMR	Hot Module Replacement — actualización de módulos en tiempo real durante el desarrollo
httpOnly Cookie	Cookie inaccesible para JavaScript, protegida contra XSS
JWT	JSON Web Token — estándar de token de autenticación stateless
LCP	Largest Contentful Paint — métrica de velocidad de carga del contenido principal
RSC	React Server Components — componentes renderizados en el servidor sin JS en el cliente
SSG	Static Site Generation — pre-renderización de páginas en build time
SSR	Server-Side Rendering — renderización de páginas en el servidor por request
XSS	Cross-Site Scripting — vulnerabilidad de inyección de scripts maliciosos

---

*Santo Pegasus Soluciones — Guía Oficial de Ingeniería Front-end v1.0.0 — Octubre 2025*

*Departamento de Ingeniería de Software / Chapter de Front-end*

*Este documento es revisado periódicamente. Consulte siempre la versión más reciente en el repositorio ``eng-guidelines``.*



# Manual de Onboarding para Nuevos Desarrolladores

## Santo Pegasus Soluciones

---

> **Versión del Documento:** 1.0.0

> **Fecha de Publicación:** Junio 2026

> **Departamento Responsable:** People & Engineering — Chapter de Back-end y Front-end

> **Clasificación:** Interno — Confidencial

> **Próxima Revisión:** Diciembre 2026

---

## TABLA DE CONTENIDOS

1. Bienvenida Institucional y Cultura de la Empresa
2. Estructura del Equipo
3. Día 1 — Accesos y Cuentas
4. Configuración del Entorno Local — Back-end
5. Configuración del Entorno Local — Front-end
6. Guía de Git y Flujo de Trabajo
7. Primeras Tareas Sugeridas — Plan 30/60/90 Días
8. Herramientas y Sistemas Internos
9. Proceso de Code Review
10. Beneficios y Políticas de RRHH
11. Seguridad desde el Primer Día
12. Checklist de Onboarding — Semana 1
13. Contactos Útiles
14. Preguntas Frecuentes del Onboarding
15. Disposiciones Finales y Revisión del Documento

---

---

## SECCIÓN 1 — BIENVENIDA INSTITUCIONAL Y CULTURA DE LA EMPRESA

### 1.1 Mensaje de Bienvenida

**Bienvenido o bienvenida a Santo Pegasus Soluciones.**

Si estás leyendo estas líneas, es porque pasaste por un proceso selectivo riguroso, demostraste tus capacidades técnicas y — lo que es igualmente importante — conectaste con los valores y la forma de trabajar de nuestro equipo. Eso no es un detalle menor. En Santo Pegasus, contratamos personas, no solo perfiles técnicos.

**Este manual fue diseñado para que tu primera semana sea productiva, clara y acogedora. Nuestro objetivo es que al finalizar el séptimo día ya tengas tu entorno configurado, hayas conocido a tu equipo, comprendas nuestro flujo de trabajo y estés listo para comenzar a contribuir con código real. No te vamos a dejar solo: tendrás un buddy asignado (un desarrollador Senior o Pleno) que acompañará tus primeros 30 días.**

**Leé este documento con calma, seguí los pasos en orden y, ante cualquier duda, recordá: preguntar no es señal de debilidad, es señal de inteligencia.**

¡Bienvenido al equipo!

— La Dirección de Ingeniería, Santo Pegasus Soluciones

---

## 1.2 Nuestra Misión

**> "Construir soluciones de software que transformen negocios reales, con código que perdure, equipos que crezcan y tecnología que respete a las personas."**

Santo Pegasus Soluciones es una empresa de tecnología especializada en el desarrollo de productos digitales, plataformas SaaS e integraciones complejas para clientes de diferentes industrias. Nuestra fortaleza está en la combinación de excelencia técnica con un enfoque humano: entendemos el negocio de nuestros clientes antes de escribir la primera línea de código.

---

## 1.3 Nuestros Valores

Estos no son palabras en una pared. Son los criterios reales con los que tomamos decisiones, evaluamos el trabajo y construimos la cultura del día a día:

#	Valor	Significado Práctico
1	Artesanía del Código	Escribimos código como si el próximo desarrollador que lo lea fuera alguien que sabemos que nos va a pedir explicaciones. Claridad antes que astucia.
2	Honestidad Técnica	Si algo no está bien hecho, lo decimos. Si cometemos un error, lo admitimos. No existe el "está más o menos funcionando" en producción.

3	Aprendizaje Continuo	La tecnología cambia. Nuestra curiosidad no puede parar. Invertimos en cursos, certificaciones y tiempo de estudio como parte del trabajo.
4	Colaboración sin Ego	El código no tiene dueño. Las ideas se defienden con argumentos, no con jerarquía. Un Junior puede tener razón frente a un Senior, y eso está bien.
5	Entrega con Responsabilidad	Cumplir los plazos importa. Pero cumplirlos sin comprometer la calidad importa más. Si hay un conflicto, comunicamos con anticipación.
6	Diversidad e Inclusión	Construimos mejores productos cuando nuestro equipo refleja la diversidad del mundo real. Todo tipo de discriminación es inaceptable.
7	Impacto sobre Volumen	Preferimos 10 Pull Requests bien hechos a 30 apresurados. La métrica que importa es el valor entregado, no las líneas escritas.

---

## 1.4 Cómo Trabajamos

Santo Pegasus opera en un modelo híbrido. La mayoría del equipo trabaja de manera remota, con encuentros presenciales opcionales (o requeridos en casos específicos de dinámicas de equipo o retros trimestrales). Las políticas exactas de home office se detallan en la Sección 10.

Nuestro flujo de trabajo está basado en metodología ágil (Scrum adaptado):

- Sprints de 2 semanas con planning, daily, refinement y retrospectiva.
- Daily asíncrona por Slack (canal del squad) cuando no hay reunión sincrónica agendada.
- Revisión de sprint con demostración de las funcionalidades entregadas, abierta a todos los miembros del equipo.
- Retrospectiva con espacio seguro para mejoras, frustraciones y celebraciones.

La comunicación interna ocurre principalmente por Slack. Las decisiones técnicas se documentan en Confluence. El tracking de trabajo está en Jira. El código vive en GitHub (organización privada).

---

## 1.5 Nuestra Pila Tecnológica Principal

Para que tengas el mapa completo desde el primer día:

Área	Tecnologías
------	-------------

Back-end	Java 17+, Spring Boot 3+, Spring Security, Spring Data JPA
Front-end	React 18+, TypeScript, Vite, Tailwind CSS
Bases de datos	PostgreSQL, MongoDB, Redis
Bases Vectoriales	Pinecone, Qdrant
Inteligencia Artificial	LangChain (RAG), integración con LLMs externos
Infraestructura Cloud	AWS (RDS, SES, SQS, Secrets Manager, EC2/ECS)
Contenedorización	Docker, Docker Compose
CI/CD	GitHub Actions
Observabilidad	SLF4J/Logback, Prometheus, Datadog
Calidad de Código	SonarQube, JUnit 5, Mockito, Testcontainers
Control de Versiones	Git, GitFlow, Conventional Commits

---

---

## SECCIÓN 2 — ESTRUCTURA DEL EQUIPO

### 2.1 Organización por Chapters

**Santo Pegasus organiza a sus desarrolladores en Chapters (agrupaciones por especialidad técnica) y en Squads (equipos multidisciplinarios por producto/cliente). Esta estructura permite que cada persona tenga una comunidad técnica de referencia (el Chapter) y a la vez trabaje en contextos de negocio concretos (el Squad).**

...

Santo Pegasus Soluciones

Chapter de Back-end

- Tech Lead de Back-end
- Developers Senior (Back-end)
- Developers Pleno / Semi-Senior (Back-end)
- Developers Junior (Back-end)

Chapter de Front-end

- Tech Lead de Front-end
- Developers Senior (Front-end)

Developers Pleno / Semi-Senior (Front-end)  
Developers Junior (Front-end)

Chapter de DevOps / SRE  
Tech Lead de DevOps  
Engineers Senior (DevOps)  
Engineers Pleno (DevOps)

Área de People & Talent  
People Business Partner  
Talent Acquisition

...

---

## 2.2 Roles y Niveles

Rol	Descripción General
Junior	En formación. Trabaja con supervisión directa del buddy o Tech Lead. Foco en aprender el flujo de trabajo, las convenciones del equipo y entregar tareas bien definidas.
Pleno (Semi-Senior)	Trabaja con autonomía en features medianas. Participa activamente en Code Reviews. Capaz de resolver problemas sin supervisión constante, pero consulta en decisiones de arquitectura.
Senior	Diseña soluciones, toma decisiones técnicas, mentoriza Juniors y Plenos, lidera reviews. Referente técnico en su Chapter.
Tech Lead	Responsable técnico de un Chapter. Define estándares, valida arquitecturas, coordina con Product Managers y Tech Leads de otros Chapters. Participa en entrevistas técnicas.

---

## 2.3 Squads — División por Productos

**Los squads son equipos de entrega. Cada squad tiene un Product Manager (PM), al menos un Tech Lead (puede ser compartido entre squads pequeños), desarrolladores de back-end, front-end y acceso a DevOps cuando es necesario.**

Squad	Área de Negocio	Tech Stack Principal
Squad Pegasus Core	Plataforma central / API Gateway	Java + Spring Boot + PostgreSQL
Squad Phoenix	Portal del cliente / Dashboard	React 18+ + TypeScript

Squad Hermes	Integraciones y mensajería	SQS + Spring Boot + Event-Driven
Squad Athena	Módulo de IA / RAG	LangChain + Pinecone/Qdrant + Python adapter
Squad Atlas	Infraestructura y DevOps	AWS + Docker + GitHub Actions

> **Nota para nuevos integrantes:** Al incorporarte, tu Tech Lead te informará en cuál squad estarás asignado durante el onboarding y cuál será tu squad definitivo.

---

---

## SECCIÓN 3 — DÍA 1 — ACCESOS Y CUENTAS

### 3.1 Antes de Llegar: Preparación Previa

El equipo de IT/Soporte y tu Tech Lead deben haber iniciado el proceso de creación de accesos con al menos 48 horas de anticipación a tu primer día. Si llegás al Día 1 sin correo corporativo, contactá a People (ver Sección 13).

Tu correo corporativo tendrá el formato:

...

nombre.apellido@santopegasus.com

...

---

### 3.2 Tabla Completa de Accesos — Día 1

Sistema	Para qué sirve	A quién solicitarlo	Plazo esperado
Correo corporativo (Google Workspace)	Comunicación, calendario, Drive	IT/Soporte	Antes del Día 1
Slack	Comunicación interna del equipo	IT/Soporte	Antes del Día 1
GitHub (organización privada)	Repositorios de código fuente	Tech Lead de tu Chapter	Día 1 — primeras 2h
Jira	Gestión de tareas y sprints	Tech Lead o PM del squad	Día 1
Confluence	Documentación interna	Tech Lead o PM del squad	Día 1
AWS Console (IAM User)	Acceso a servicios cloud (Read-only inicial)	Tech Lead de DevOps	Día 1-2
Datadog	Observabilidad, logs y métricas	Tech Lead de DevOps	Día 1-2

SonarQube	Análisis de calidad de código	IT/Soporte	Día 1–2
VPN Corporativa	Acceso seguro a recursos internos	IT/Soporte	Día 1
1Password (Gestor de contraseñas)	Gestión segura de credenciales	IT/Soporte	Día 1
Figma	Diseño de interfaces (Front-end)	Tech Lead de Front-end	Día 2–3

---

### 3.3 Procedimiento para Solicitar Accesos

**1. Envió un correo a [it-support@santopegasus.com](mailto:it-support@santopegasus.com) con el asunto: `[ONBOARDING] Solicitud de accesos — [Tu Nombre] — [Fecha de Ingreso]`**

2. En el cuerpo del correo, incluí: tu nombre completo, tu rol, el nombre de tu Tech Lead y el squad al que fuiste asignado.

3. IT/Soporte responderá con un ticket de seguimiento en un plazo máximo de 4 horas hábiles.

4. Para accesos que dependen del Tech Lead (GitHub, AWS), pedíselo directamente por Slack en el mismo Día 1.

**> ⚠ Importante: Nunca compartas tus credenciales con otros miembros del equipo, ni siquiera con tu Tech Lead. Si necesitás compartir acceso temporalmente, usá roles y permisos, nunca tu usuario personal. Ver Sección 11.**

---

### 3.4 Canales de Slack a los que Unirte el Día 1

Una vez en Slack, unite inmediatamente a los siguientes canales:

Canal	Propósito
`#general`	Comunicaciones generales de la empresa
`#back-end`	Discusiones técnicas del Chapter de Back-end
`#front-end`	Discusiones técnicas del Chapter de Front-end
`#devops`	Infraestructura, deploys, alertas de CI/CD
`#incidents`	Canal de incidentes en producción (solo lectura inicial)
`#code-reviews`	Notificaciones de Pull Requests abiertos
`#aprendizaje`	Recursos, cursos, artículos y certifications
`#random`	Conversaciones informales y cultura del equipo

## SECCIÓN 4 — CONFIGURACIÓN DEL ENTORNO LOCAL — BACK-END

> Esta sección es para desarrolladores del Chapter de Back-end. Si sos del Chapter de Front-end, podés saltar directamente a la Sección 5.

### 4.1 Requisitos Previos del Sistema

Antes de comenzar, asegurate de tener:

- **Sistema operativo: macOS 13+, Ubuntu 22.04+ o Windows 11 con WSL2 (recomendado)**
- **Al menos 16 GB de RAM y 50 GB de espacio libre en disco**
- Conexión a la VPN corporativa activa

### 4.2 Instalación de JDK 17 con SDKMAN

Usamos **SDKMAN** para gestionar versiones del JDK. Esto permite que diferentes proyectos usen diferentes versiones de Java sin conflictos.

#### Paso 1 — Instalar SDKMAN:

```
``bash
curl -s "https://get.sdkman.io" | bash
source "$HOME/.sdkman/bin/sdkman-init.sh"
...`
```

#### Paso 2 — Verificar instalación:

```
``bash
sdk version
...`
```

#### Paso 3 — Instalar Java 17 (distribución Temurin de Eclipse):

```
``bash
sdk install java 17.0.11-tem
sdk default java 17.0.11-tem
...`
```

#### Paso 4 — Verificar la versión activa:

```
``bash
java -version`
```



# Salida esperada: openjdk version "17.0.11"

## 2024-04-16

...

---

### 4.3 Instalación y Configuración de IntelliJ IDEA

Usamos IntelliJ IDEA (Community o Ultimate — la empresa provee licencias Ultimate para todos los desarrolladores back-end).

#### Paso 1 — Instalar IntelliJ IDEA:

Descargá desde: <https://www.jetbrains.com/idea/download/>

#### Paso 2 — Plugins obligatorios a instalar:

Plugin	Propósito
SonarLint	Análisis de calidad de código en tiempo real
Lombok	Soporte para anotaciones Lombok en Spring Boot
MapStruct Support	Soporte para MapStruct en conversión de DTOs
Docker	Gestión de contenedores desde el IDE
GitToolBox	Información de Git inline en el editor
.env files support	Soporte para archivos .env

#### Paso 3 — Configuración del Code Style:

...

File → Settings → Editor → Code Style → Java

→ Importar el archivo santo-pegasus-codestyle.xml

(disponible en el repositorio interno: /docs/ide/santo-pegasus-codestyle.xml)

...

#### Paso 4 — Configurar el JDK en IntelliJ:

...

File → Project Structure → SDK → Add SDK → JDK

→ Apuntar a la ruta de SDKMAN: ~/.sdkman/candidates/java/current

...

---

### 4.4 Instalación de Docker Desktop

Usamos Docker obligatoriamente para levantar el ambiente local con todas las dependencias (base de datos, Redis, etc.).

```
```bash
macOS (con Homebrew)
```

```
brew install --cask docker
Ubuntu
```

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
Verificar instalación
```

```
docker --version
docker compose version
```
```

> En macOS y Windows, iniciá Docker Desktop como aplicación antes de ejecutar cualquier comando `docker`.

---

## 4.5 Clonar el Repositorio y Configurar el Proyecto

**Paso 1 — Clonar el repositorio principal:**

```
```bash
git clone git@github.com:santo-pegasus/[nombre-del-proyecto].git
cd [nombre-del-proyecto]
```
```

> **Necesitás tener tu SSH Key configurada en tu cuenta de GitHub. Si no la tenés:**

```
> ```bash
> ssh-keygen -t ed25519 -C "nombre.apellido@santopegasus.com"
> cat ~/.ssh/id_ed25519.pub
> # Copiá el output y agregalo en GitHub → Settings → SSH Keys
> ```
```

**Paso 2 — Crear el archivo `.env` local:**

```
```bash
cp .env.example .env
```
```

**Paso 3 — Completar las variables obligatorias en el `.env`:**

| Variable | Descripción | Ejemplo de Valor |
|----------|-------------|------------------|
|----------|-------------|------------------|

|               |                                            |                                                |
|---------------|--------------------------------------------|------------------------------------------------|
| `DB_URL`      | URL de conexión a PostgreSQL local         | `jdbc:postgresql://localhost:5432/pegasus_dev` |
| `DB_USERNAME` | Usuario de la base de datos local          | `pegasus_user`                                 |
| `DB_PASSWORD` | Contraseña de la base de datos local       | `dev_password_local`                           |
| `JWT_SECRET`  | Secret para firma de tokens JWT (solo dev) | `my-local-dev-secret-key-256bits`              |
| `AWS_REGION`  | Región AWS configurada                     | `us-east-1`                                    |
| `REDIS_HOST`  | Host del Redis local                       | `localhost`                                    |
| `REDIS_PORT`  | Puerto del Redis                           | `6379`                                         |
| `APP_PORT`    | Puerto en el que corre la aplicación       | `8080`                                         |

> **⚠ El archivo `.env` NUNCA debe ser commiteado al repositorio. Ya está listado en el `.gitignore`. Si por error lo commiteas, notificará inmediatamente al Tech Lead. Ver Sección 11.**

---

#### 4.6 Levantar el Ambiente Local con Docker Compose

```
```bash
```

Levantar todas las dependencias (PostgreSQL, Redis, etc.)

```
docker compose up -d
```

Verificar que los contenedores están corriendo

```
docker ps
```

Salida esperada:

CONTAINER ID	IMAGE	STATUS	PORTS
--------------	-------	--------	-------

abc123def456	postgres:15	Up 2 minutes	0.0.0.0:5432->5432/tcp
--------------	-------------	--------------	------------------------

789ghi101jkl	redis:7	Up 2 minutes	0.0.0.0:6379->6379/tcp
--------------	---------	--------------	------------------------

...

4.7 Ejecutar el Proyecto por Primera Vez

```bash

### Opción 1 — Con Maven Wrapper (recomendado)

./mvnw spring-boot:run

### Opción 2 — Desde IntelliJ IDEA

**Abrir la clase principal:**

**src/main/java/.../Application.java**

**Click derecho → Run 'Application'**

**Verificar que la aplicación levantó correctamente**

curl http://localhost:8080/actuator/health

**Respuesta esperada: {"status":"UP"}**

...

---

#### 4.8 Ejecutar las Pruebas

```bash

Ejecutar todas las pruebas

./mvnw test

Ejecutar solo las pruebas unitarias

./mvnw test -Dtest="UnitTest"

Ejecutar solo las pruebas de integración (requiere Docker corriendo)

./mvnw test -Dtest="IntegrationTest"

Verificar el reporte de cobertura (generado en target/site/jacoco/index.html)

```
./mvnw verify
```

```
open target/site/jacoco/index.html
```

```
...
```

> **Recordá: la cobertura mínima obligatoria es del 80% en pruebas unitarias para aprobar un Code Review. [Source: Guía Oficial de Ingeniería Back-end.pdf, Página 12]**

```
---
```

```
---
```

SECCIÓN 5 — CONFIGURACIÓN DEL ENTORNO LOCAL — FRONT-END

> Esta sección es para desarrolladores del Chapter de Front-end.

5.1 Requisitos Previos del Sistema

- **Sistema operativo: macOS 13+, Ubuntu 22.04+ o Windows 11 con WSL2**
- **Al menos 8 GB de RAM y 20 GB de espacio libre**
- **Conexión a la VPN corporativa activa**

```
---
```

5.2 Instalación de Node.js 20+ con NVM

Usamos NVM (Node Version Manager) para gestionar versiones de Node.js, de la misma manera que usamos SDKMAN para Java.

Paso 1 — Instalar NVM:

```
```bash
```

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

## Recargar el shell

```
source ~/.bashrc # o ~/.zshrc en macOS con Zsh
```

```
...
```

#### Paso 2 — Instalar y usar Node.js 20:

```
```bash
```

```
nvm install 20
```

```
nvm use 20
```

```
nvm alias default 20
```

```
...
```

Paso 3 — Verificar instalación:

```
```bash
node --version # v20.x.x
npm --version # 10.x.x
```

---
```

5.3 Visual Studio Code — Instalación y Extensiones

Usamos VS Code como editor estándar para el Chapter de Front-end.

Descargá desde: <https://code.visualstudio.com/>

Extensiones obligatorias:

| Extensión | ID en Marketplace | Propósito |
|---------------------------|--|--|
| ESLint | <code>`dbaeumer.vscode-eslint`</code> | Linting de JavaScript/TypeScript |
| Prettier | <code>`esbenp.prettier-vscode`</code> | Formateo automático de código |
| GitLens | <code>`eamodio.gitlens`</code> | Información de Git inline en el editor |
| TypeScript Hero | <code>`rbbbit.typescript-hero`</code> | Organización de imports TypeScript |
| Tailwind CSS IntelliSense | <code>`bradlc.vscode-tailwindcss`</code> | Autocompletado para Tailwind |
| Auto Rename Tag | <code>`formulahendry.auto-rename-tag`</code> | Renombrado automático de tags JSX |
| Error Lens | <code>`usernamehd.error-lens`</code> | Visualización inline de errores |
| SonarLint | <code>`sonarsource.sonarlint-vscode`</code> | Calidad de código en tiempo real |

Configuración de VS Code (settings.json):

```
```json
{
 "editor.formatOnSave": true,
 "editor.defaultFormatter": "esbenp.prettier-vscode",
 "editor.codeActionsOnSave": {
 "source.fixAll.eslint": true
 },
 "typescript.preferences.importModuleSpecifier": "relative"
}
```

---
```

5.4 Clonar el Repositorio Front-end

```
```bash
git clone git@github.com:santo-pegasus/[nombre-del-proyecto]-frontend.git
cd [nombre-del-proyecto]-frontend
```
```

5.5 Configurar el Archivo `.env`

```
```bash
cp .env.example .env
```
```

Variables obligatorias para el front-end:

| Variable | Descripción | Ejemplo |
|---------------------|------------------------------|-----------------------------|
| `VITE_API_BASE_URL` | URL de la API back-end local | `http://localhost:8080/api` |
| `VITE_APP_NAME` | Nombre de la aplicación | `Pegasus Portal` |
| `VITE_ENVIRONMENT` | Entorno actual | `development` |

> **▲ En proyectos Vite, solo las variables con prefijo `VITE\_` son expuestas al cliente. Nunca pongas secrets en variables front-end. El `.env` ya está en el `.gitignore`.**

5.6 Instalar Dependencias y Ejecutar el Proyecto

```
```bash
```

### Instalar dependencias

```
npm install
```

### Iniciar el servidor de desarrollo

```
npm run dev
```

**La aplicación estará disponible en:**

**http://localhost:5173**

### Ejecutar pruebas unitarias

```
npm run test
```

# Ejecutar pruebas con cobertura

```
npm run test:coverage
```

## Build para producción (solo para verificar que el build no falla)

```
npm run build
```

...

---

---

## SECCIÓN 6 — GUÍA DE GIT Y FLUJO DE TRABAJO

### 6.1 Modelo de Branching — GitFlow

Usamos GitFlow como estrategia de branching. Las ramas principales son:

Rama	Propósito	Reglas
`main`	Código en producción	Nunca se commitea directamente. Solo recibe merges desde `release/` o `hotfix/`
`develop`	Integración de features en desarrollo	Base para abrir features. Recibe merges desde `feature/`
`feature/`	Desarrollo de nuevas funcionalidades	Se crea desde `develop`. Se mergea de vuelta a `develop`
`release/`	Preparación para producción	Se crea desde `develop`. Se mergea a `main` y `develop`
`hotfix/`	Correcciones urgentes en producción	Se crea desde `main`. Se mergea a `main` y `develop`

---

### 6.2 Nomenclatura de Branches

El patrón obligatorio para nombrar una branch es:

...

[tipo]/[JIRA-TICKET]-[descripcion-corta-en-kebab-case]

...

**Tipos válidos:**

Tipo	Cuándo usarlo
`feature`	Nueva funcionalidad



`fix`	Corrección de bug
`hotfix`	Corrección urgente en producción
`refactor`	Refactorización de código existente
`docs`	Actualización de documentación
`test`	Adición o corrección de pruebas
`chore`	Tareas de mantenimiento (actualizaciones de deps, etc.)

#### Ejemplos correctos:

```

`bash
feature/PEG-142-cadaastro-de-usuarios
fix/PEG-201-correccion-validacion-email
refactor/PEG-315-extraer-logica-autenticacion
hotfix/PEG-999-error-critico-login-produccion
`

```

#### Cómo crear la branch:

```

`bash

```

## Asegurarse de estar en develop actualizado

```

git checkout develop

```

```

git pull origin develop

```

## Crear y cambiar a la nueva branch

```

git checkout -b feature/PEG-142-cadaastro-de-usuarios

```

```

`

```

```

```

### 6.3 Conventional Commits — Mensajes de Commit Semánticos

Usamos Conventional Commits para todos los mensajes de commit. Esto permite la generación automática de changelogs y una historia de Git legible.

#### Formato:

```

`

```

```

<tipo>(<scope>): <descripción corta en minúsculas>

```

```

[cuerpo opcional — explicación más detallada]

```

```

[footer opcional — referencias a tickets]

```

```

`

```

**Tipos válidos:**

Tipo	Significado
`feat`	Nueva funcionalidad
`fix`	Corrección de bug
`refactor`	Refactorización sin cambio de funcionalidad
`test`	Adición o corrección de pruebas
`docs`	Documentación
`style`	Formateo, espacios (sin cambios lógicos)
`chore`	Mantenimiento, actualizaciones
`perf`	Mejoras de rendimiento
`ci`	Cambios en CI/CD

**Ejemplos correctos:**

```
```bash
```

Feature simple

git commit -m "feat(auth): agregar validación de token JWT en el endpoint de login"

Fix con referencia a ticket

git commit -m "fix(usuarios): corregir validación de email duplicado

El servicio no estaba verificando correctamente los emails en mayúsculas.

Closes PEG-201"

Refactor

git commit -m "refactor(pagos): extraer lógica de cálculo de impuestos a servicio dedicado"

```
```
```

**Ejemplos incorrectos (no hacer esto):**

```
```bash
```

Demasiado vago

git commit -m "fix bug"

Sin tipo

```
git commit -m "agrego validación"
```

En mayúsculas

```
git commit -m "FEAT: Nueva funcionalidad"
```

```
...
```

```
---
```

6.4 Proceso de Push y Pull Request

Paso 1 — Asegurarte de que las pruebas pasen localmente:

```
```bash
```

```
./mvnw test # Back-end
```

```
npm run test # Front-end
```

```
```
```

Paso 2 — Push de tu branch:

```
```bash
```

```
git push origin feature/PEG-142-cadastro-de-usuarios
```

```
```
```

Paso 3 — Abrir el Pull Request en GitHub:

- Ir al repositorio en GitHub
- **Hacer click en "Compare & pull request"**
- Completar el template obligatorio del PR:
 - **¿Qué se hizo? — Descripción clara de los cambios**
 - **¿Por qué se hizo? — Contexto de negocio o técnico**
 - **¿Cómo probar? — Pasos detallados para reproducir y verificar los cambios**
 - **Screenshots (si aplica, especialmente en front-end)**
 - **Checklist de calidad (pruebas, documentación, etc.)**

Paso 4 — Solicitar revisores:

- Asignar al menos 2 revisores Senior o Semi-Senior (Pleno)
- Notificar en el canal `#code-reviews` de Slack con el link al PR

> Todo PR en Santo Pegasus exige al menos 2 aprobaciones de miembros Senior o Semi-Senior (Pleno) antes del merge. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

```
---
```

6.5 Responder Reviews

- **Respondé todos los comentarios antes de solicitar una nueva revisión.**

- Si estás de acuerdo con un comentario, hacé el cambio y respondé con `Corregido en el commit abc1234`.
- Si no estás de acuerdo, explicá tu razonamiento con respeto y datos técnicos. El debate técnico es bienvenido.
- Nunca hagas merge sin tener todas las aprobaciones requeridas.

SECCIÓN 7 — PRIMERAS TAREAS SUGERIDAS — PLAN 30/60/90 DÍAS

7.1 Para Desarrolladores Junior

| Período | Objetivos | Indicadores de Éxito |
|------------------|---|--|
| Primeros 30 días | Completar el onboarding. Tener el entorno funcionando. Leer la Guía de Ingeniería Back-end o Front-end completa. Resolver 2–3 tickets de tipo `good-first-issue` en Jira con apoyo del buddy. Participar en al menos 1 Code Review como observador. | PR mergeado. Entorno configurado y funcional. Participación activa en dailies. |
| 30–60 días | Entregar features pequeñas de forma autónoma (con revisión del buddy). Escribir pruebas unitarias para el código propio. Aprender a leer y usar el Swagger de las APIs internas. Participar activamente en Code Reviews. | Cobertura de pruebas $\geq 80\%$ en código propio. Al menos 3 PRs mergeados sin errores graves. |
| 60–90 días | Contribuir en refinements con preguntas técnicas. Proponer mejoras en el código del equipo. Tomar ownership de al menos 1 feature completa de principio a fin. Dar su primer Code Review como revisor (con guía del buddy). | Feature completa entregada. Primer review dado. Participación en retros con sugerencias propias. |

7.2 Para Desarrolladores Pleno (Semi-Senior)

| Período | Objetivos | Indicadores de Éxito |
|------------------|---|--|
| Primeros 30 días | Onboarding técnico completo. Entender la arquitectura del squad. Tomar ownership de features de mediana complejidad. Realizar Code Reviews activos. | Al menos 5 PRs mergeados. Al menos 3 Code Reviews realizados con comentarios de calidad. |

| | | |
|------------|---|--|
| 30–60 días | Liderar técnicamente una feature del sprint. Proponer mejoras de arquitectura o proceso en el squad. Mentorizar a un Junior si hay en el squad. | Feature liderada entregada. Propuesta técnica documentada en Confluence. |
| 60–90 días | Tomar decisiones técnicas en su área con autonomía. Participar en definición de criterios de aceptación en refinements. Proponer y liderar una iniciativa de deuda técnica. | Decisión técnica documentada. Iniciativa de deuda técnica con PR abierto o completado. |

7.3 Para Desarrolladores Senior

| Período | Objetivos | Indicadores de Éxito |
|------------------|---|---|
| Primeros 30 días | Onboarding completo. Entender la arquitectura global (no solo del squad). Revisar la Guía de Ingeniería y proponer mejoras si las identifica. Contribuir con reviews de alta calidad desde la primera semana. | ADR (Architecture Decision Record) de una decisión tomada. Reviews reconocidos por el equipo. |
| 30–60 días | Identificar y documentar áreas de mejora técnica. Liderar una iniciativa técnica transversal. Comenzar a mentorizar activamente a Juniors o Plenos. | Documento técnico en Confluence. Al menos 2 Juniors/Plenos con sesión de mentoring. |
| 60–90 días | Participar en entrevistas técnicas de nuevos candidatos. Proponer cambios en la Guía de Ingeniería. Tener visibilidad clara sobre el roadmap técnico del squad. | Participación en al menos 1 entrevista. Pull Request de actualización de la Guía de Ingeniería. |

SECCIÓN 8 — HERRAMIENTAS Y SISTEMAS INTERNOS

8.1 Jira — Gestión de Tareas

URL: <https://santopegasus.atlassian.net/jira>

Cómo rastrear tus tickets:

1. Al inicio de cada sprint, los tickets asignados a vos aparecen en tu Board del Squad.
2. Cada ticket tiene un ID único: ej. `PEG-142`. Ese ID se usa en branches y commits.
3. Los estados de un ticket son: To Do → In Progress → In Review → Done.
4. Cuando comenzás a trabajar en un ticket, movelo a In Progress inmediatamente.
5. Cuando abrís un PR, movelo a In Review y agregá el link al PR en el ticket de Jira.
6. Solo movelo a Done cuando el PR fue mergeado.

Tipos de tickets que vas a encontrar:

| Tipo | Descripción |
|------------------|---|
| Story | Funcionalidad desde la perspectiva del usuario |
| Task | Tarea técnica sin valor directo de negocio |
| Bug | Error a corregir |
| Spike | Investigación técnica o exploración |
| good-first-issue | Tickets etiquetados especialmente para nuevos integrantes |

8.2 Confluence — Documentación Interna

URL: <https://santopegasus.atlassian.net/confluence>

Espacios principales:

| Espacio | Qué encontrarás |
|----------------------------------|--|
| Engineering / Back-end | Guías técnicas, ADRs, arquitectura de sistemas |
| Engineering / Front-end | Guías de componentes, design system, convenciones |
| Engineering / DevOps | Procedimientos de deploy, runbooks de incidentes |
| Squads | Documentación específica de cada squad y producto |
| People & Onboarding | Este manual, políticas de RRHH, beneficios |
| Decisiones de Arquitectura (ADR) | Registro histórico de decisiones técnicas relevantes |

> Buena práctica: Ante cualquier duda técnica recurrente, buscá primero en Confluence antes de preguntar en Slack. Si la respuesta no está, agregala vos mismo después de encontrarla — así mejoramos la documentación colectivamente.

8.3 Slack — Comunicación Interna

Canales obligatorios (ya listados en Sección 3.4) — Guía de etiqueta:

- Preferí los canales públicos a los mensajes directos para temas técnicos. Así el conocimiento queda disponible para todos.
- Usá hilos (threads) para responder a mensajes específicos. Evitá el ruido en el canal principal.
- Para menciones urgentes, usá `@nombre` o `@here` con criterio (no abusos del `@channel`).

- El canal `#incidents` tiene alertas automáticas de Datadog y PagerDuty. No envíes mensajes en este canal salvo que estés informando o coordinando un incidente activo.

8.4 Swagger — APIs Internas

Todas las APIs de Santo Pegasus están documentadas usando la especificación OpenAPI (Swagger), generada automáticamente desde el código con `springdoc-openapi`.

Acceso al Swagger en ambiente local:

...

<http://localhost:8080/swagger-ui/index.html>

...

Acceso al Swagger en ambiente de Staging:

...

<https://api-staging.santopegasus.com/swagger-ui/index.html>

...

> El Swagger de producción está protegido y requiere autenticación. Consultá con tu Tech Lead para obtener las credenciales de acceso al Swagger de Staging.

Cómo usar el Swagger:

1. Expandí el endpoint que querés explorar.
2. **Hacé click en "Try it out" para ejecutar llamadas reales.**
3. Completá los parámetros requeridos.
4. **Hacé click en "Execute" y analizá la respuesta.**

SECCIÓN 9 — PROCESO DE CODE REVIEW

9.1 Filosofía del Code Review en Santo Pegasus

El Code Review no es un filtro de control de calidad punitivo. Es una oportunidad de aprendizaje colectivo, mejora del código y diseminación de conocimiento. Todos — sin importar el nivel — dan y reciben reviews.

> La comunicación en los reviews debe estar siempre pautada por la empatía y la educación. El enfoque de la revisión debe ser exclusivamente la mejora del código, nunca una crítica al autor. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

9.2 Cómo Participar en Reviews desde el Día 1

Como autor del PR:

- Completá el template del PR con detalle: qué se hizo, por qué se hizo, cómo probar.

- Hacé PRs pequeños y focalizados. Un PR que toca 15 archivos no relacionados es difícil de revisar.
- Respondé todos los comentarios antes de pedir re-revisión.
- No hagas merge sin las 2 aprobaciones requeridas.

Como revisor:

- Aceptá la revisión dentro de 24 horas hábiles de ser asignado.
- Leé el contexto del ticket de Jira antes de revisar el código.
- Priorizá comentarios sobre: legibilidad, seguridad, rendimiento y conformidad con la Guía de Ingeniería.

9.3 Etiqueta — Cómo Dar Feedback

Recomendado — Preguntas sugestivas:

...

"¿Qué te parece extraer esta lógica a un método privado para mejorar la legibilidad?"

"Noté que este método hace más de una cosa. ¿Consideraste separar las responsabilidades?"

"¿Hay alguna razón para no usar Optional aquí en lugar del null check manual?"

...

No recomendado — Imperativos sin contexto:

...

"Cambia esto."

"Esto está mal."

"No hagas así."

...

> Se recomienda el uso de preguntas sugestivas para incentivar la reflexión del desarrollador, como: "¿Qué te parece extraer esto a un método privado?", evitando imperativos como "Cambia esto a un método privado". [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

9.4 Criterios Técnicos de Revisión

El revisor debe validar:

| Criterio | Qué verificar |
|-------------|---|
| Legibilidad | El código es fácil de entender por alguien que no lo escribió |
| Seguridad | No hay hardcode de credenciales, validación de entradas presente, no se expone información sensible |
| Rendimiento | No hay consultas N+1, no hay operaciones bloqueantes innecesarias |

| | |
|--------------|--|
| Pruebas | La cobertura mínima del 80% está cumplida; los casos edge están cubiertos |
| Conformidad | El código sigue las directrices de la Guía de Ingeniería Back-end o Front-end |
| Arquitectura | Las responsabilidades están en las capas correctas (Controller, Service, Repository) |

> El revisor actúa como un guardián de la base de código, debiendo validar minuciosamente la legibilidad, la seguridad, el rendimiento y si la implementación respeta rigurosamente todas las directrices establecidas en la Guía de Ingeniería Back-end. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

SECCIÓN 10 — BENEFICIOS Y POLÍTICAS DE RRHH

10.1 Beneficios Activos

| Beneficio | Detalle |
|----------------------------|--|
| Vale Transporte | Cubierto al 100% para días de trabajo presencial, proporcional a los días en oficina |
| Vale Alimentación / Comida | Tarjeta mensual cargada el primer día hábil del mes, valor definido según nivel de seniority |
| Seguro de Salud | Plan médico corporativo, cobertura desde el primer mes de trabajo |
| Seguro Dental | Incluido en el plan de salud corporativo |
| Horario Flexible | Core hours de 10:00 a 17:00 (zona horaria del equipo). El resto del horario es flexible |
| Home Office | Modelo híbrido. La asistencia presencial es opcional salvo eventos específicos del equipo |
| Cursos y Certificaciones | Presupuesto anual por persona para capacitaciones y certificaciones técnicas. Consultá con People el monto vigente |
| Día de Cumpleaños Libre | El día de tu cumpleaños (o el día hábil más cercano) es libre remunerado |
| Política de Vacaciones | Según legislación local + días adicionales por antigüedad. Coordinar con 30 días de anticipación |

10.2 Cursos y Certificaciones Patrocinados

Santo Pegasus invierte activamente en el crecimiento técnico de su equipo. Las certificaciones priorizadas son:

- **AWS Certified Developer – Associate**
- **AWS Certified Solutions Architect**
- **Oracle Certified Professional: Java SE 17 Developer**
- **Professional Scrum Developer (PSD)**
- **Kubernetes Application Developer (CKAD)**

Para solicitar el patrocinio de un curso o certificación:

1. Conversarlo con tu Tech Lead.
2. Completar el formulario en Confluence: `People → Beneficios → Solicitud de Capacitación`.
3. Aguardar aprobación del People BP (plazo: 5 días hábiles).

10.3 Política de Desconexión

Santo Pegasus respeta el tiempo fuera del trabajo. No se esperan respuestas de Slack fuera del horario laboral, salvo que seas el responsable de guardia de on-call (rotación voluntaria con compensación adicional). Los incidentes de producción tienen un proceso de escalada definido — nadie debe estar disponible 24/7 sin acuerdo previo.

SECCIÓN 11 — SEGURIDAD DESDE EL PRIMER DÍA

11.1 Gestión de Contraseñas

Usamos 1Password como gestor de contraseñas corporativo. Todas las contraseñas relacionadas con sistemas de la empresa deben estar almacenadas ahí.

Reglas básicas:

- Nunca reutilices contraseñas entre sistemas.
- Todas las contraseñas corporativas deben tener al menos 20 caracteres, generadas por 1Password.
- **Activa autenticación de dos factores (2FA) en todos los sistemas que lo permitan: GitHub, AWS Console, Jira, Confluence.**
- **Nunca compartas tu contraseña con nadie, ni con tu Tech Lead, ni con IT/Soporte. Si necesitas dar acceso, usá roles y permisos del sistema correspondiente.**

11.2 Credenciales en Código — Regla de Oro

> **NUNCA** haga **hardcode** (escribir directamente en el código) de contraseñas, claves de API, URLs de base de datos o tokens. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

En entornos de producción y homologación, es obligatorio el uso de soluciones de gestión de secretos como HashiCorp Vault, AWS Secrets Manager o Google Cloud Secret Manager. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 8]

Checklist de seguridad antes de hacer un commit:

- [] ¿El archivo `.env` está en el `.gitignore`?
- [] ¿No hay ningún secret, password o API key en el código fuente?
- [] ¿Las variables sensibles están referenciadas por nombre de variable de entorno?
- [] ¿El archivo `.env.example` tiene los nombres de las variables pero sin valores reales?

11.3 Uso Correcto del `.gitignore`

El repositorio ya tiene un `.gitignore` pre-configurado. Nunca lo modifique para incluir archivos que contienen secretos. Los archivos que siempre deben estar en el `.gitignore`:

```.gitignore`

## Variables de entorno locales

`.env`

`.env.local`

`.env..local`

## Archivos de configuración con secrets

`application-local.yml`

`application-local.properties`

## Directorios de build

`target/`

`build/`

`dist/`

`node_modules/`

## Archivos de IDE

`.idea/`

`.iml`

`.vscode/settings.json`

...

---

## 11.4 Escaneo Automático de Secrets

Usamos herramientas de escaneo automático como gitleaks o trufflehog en nuestros pipelines de CI/CD para detectar y bloquear commits que contengan cadenas de texto que se asemejen a secretos. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 9]

Si el pipeline bloquea tu PR por detección de un posible secret:

1. **No intentes bypassar el control.**
2. Revisá el commit indicado en el error.
3. Si es un falso positivo, notificá al Tech Lead de DevOps.
4. **Si es un secret real, notificá inmediatamente al Tech Lead y a IT/Soporte para iniciar el proceso de rotación de credenciales.**

---

## 11.5 Reporte de Incidentes de Seguridad

Si detectás o sospechás de un incidente de seguridad (secret expuesto, acceso no autorizado, comportamiento anómalo):

1. **No lo resuelvas solo y en silencio.**
2. Notificá inmediatamente en `#incidents` con el tag `@security-lead`.
3. Contactá a tu Tech Lead directamente por Slack o teléfono.
4. Documentá lo que viste, cuándo lo viste y qué acciones tomaste.
5. No borres evidencia. No modifiques logs.

> El principio de privilegio mínimo debe ser aplicado, garantizando que cada microservicio tenga acceso solo a los secretos estrictamente necesarios para su operación. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 9]

---

---

# SECCIÓN 12 — CHECKLIST DE ONBOARDING — SEMANA 1

Este checklist debe ser completado conjuntamente por el nuevo desarrollador y su Tech Lead antes del fin de la primera semana. Guardalo en Confluence en tu página personal de onboarding.

---

## 12.1 Accesos y Cuentas

- [ ] Correo corporativo (@santopegasus.com) funcionando
- [ ] Slack instalado y canales obligatorios unidos
- [ ] Acceso a GitHub (organización privada) confirmado

- [ ] Acceso a Jira funcionando, squad asignado visible
- [ ] Acceso a Confluence funcionando
- [ ] VPN corporativa instalada y probada
- [ ] 1Password corporativo configurado y contraseñas migradas
- [ ] 2FA activado en GitHub, Google Workspace y AWS Console
- [ ] Acceso a AWS Console (Read-only) confirmado
- [ ] Acceso a Datadog confirmado
- [ ] Acceso a SonarQube confirmado

---

## 12.2 Entorno de Desarrollo — Back-end

- [ ] SDKMAN instalado y Java 17 activo (``java -version``)
- [ ] IntelliJ IDEA instalado con plugins obligatorios
- [ ] Docker Desktop instalado y corriendo (``docker ps``)
- [ ] Repositorio clonado correctamente con SSH
- [ ] Archivo ``.env`` creado con todas las variables obligatorias
- [ ] ``docker compose up -d`` ejecutado sin errores
- [ ] Aplicación corriendo en localhost:8080 (``actuator/health`` retorna UP)
- [ ] ``./mvnw test`` ejecutado sin fallos
- [ ] Swagger local accesible en localhost:8080/swagger-ui/index.html

---

## 12.3 Entorno de Desarrollo — Front-end

- [ ] NVM instalado y Node.js 20 activo (``node --version``)
- [ ] VS Code instalado con extensiones obligatorias
- [ ] Repositorio front-end clonado correctamente con SSH
- [ ] Archivo ``.env`` front-end creado con variables obligatorias
- [ ] ``npm install`` ejecutado sin errores
- [ ] ``npm run dev`` corriendo en localhost:5173
- [ ] ``npm run test`` ejecutado sin fallos

---

## 12.4 Flujo de Trabajo

- [ ] SSH Key configurada en GitHub
- [ ] Primer commit de prueba realizado siguiendo Conventional Commits
- [ ] Branch creada siguiendo el patrón de nomenclatura correcto
- [ ] Pull Request de prueba abierto con template completo
- [ ] Al menos 1 Code Review observado

- ☐ Daily del squad participado al menos 1 vez

---

## 12.5 Cultura y Conocimiento

- ☐ Guía de Ingeniería Back-end o Front-end leída completamente
- ☐ Sesión de bienvenida con Tech Lead realizada (mínimo 1h)
- ☐ Presentación al squad en la daily o reunión de equipo
- ☐ Al menos 1 sesión de pair programming con el buddy agendada
- ☐ Plan 30/60/90 días revisado y acordado con Tech Lead
- ☐ Página personal de onboarding creada en Confluence

---

## 12.6 Firma de Confirmación

| Rol                     | Nombre | Firma | Fecha |
|-------------------------|--------|-------|-------|
| Nuevo Desarrollador     |        |       |       |
| Tech Lead               |        |       |       |
| People Business Partner |        |       |       |

---

---

# SECCIÓN 13 — CONTACTOS ÚTILES

## 13.1 Tech Leads por Área

| Área                     | Nombre                 | Slack               | Email                        |
|--------------------------|------------------------|---------------------|------------------------------|
| Tech Lead — Back-end     | A confirmar por People | @tech-lead-backend  | tl-backend@santopegasus.com  |
| Tech Lead — Front-end    | A confirmar por People | @tech-lead-frontend | tl-frontend@santopegasus.com |
| Tech Lead — DevOps / SRE | A confirmar por People | @tech-lead-devops   | tl-devops@santopegasus.com   |
| Dirección de Ingeniería  | A confirmar por People | @eng-director       | engineering@santopegasus.com |

> Los nombres específicos de los Tech Leads son comunicados en el correo de bienvenida enviado por People antes del Día 1.

---

## 13.2 Equipo de People & Talent

| Rol                     | Contacto                | Canal                 |
|-------------------------|-------------------------|-----------------------|
| People Business Partner | people@santopegasus.com | `#people-hr` en Slack |
| Talent Acquisition      | talent@santopegasus.com | `#people-hr` en Slack |

## 13.3 Equipo de IT / Soporte

| Canal                        | Propósito                                        | Tiempo de Respuesta |
|------------------------------|--------------------------------------------------|---------------------|
| it-support@santopegasus.com  | Solicitud de accesos, hardware, incidentes de IT | 4h hábiles          |
| `#it-support` en Slack       | Consultas rápidas de IT                          | 2h hábiles          |
| Ticket en Jira (proyecto IT) | Solicitudes formales con seguimiento             | Según prioridad     |

## 13.4 Contactos de Emergencia

| Situación                              | A quién contactar                  | Cómo                                           |
|----------------------------------------|------------------------------------|------------------------------------------------|
| Incidente de producción                | Tech Lead del squad + `#incidents` | Slack                                          |
| Incidente de seguridad                 | Tech Lead de DevOps + IT/Soporte   | Slack + Email                                  |
| Problema de accesos urgente            | IT/Soporte                         | Email + Slack                                  |
| Situación personal (salud, emergencia) | People BP                          | Email o teléfono directo (comunicado en Día 1) |

## SECCIÓN 14 — PREGUNTAS FRECUENTES DEL ONBOARDING

**P: ¿Cuánto tiempo tengo para hacer el onboarding antes de que se espere que sea plenamente productivo?**

R: El plan 30/60/90 días es una guía de rampaje progresiva. No se espera que seas plenamente productivo al finalizar la Semana 1. El objetivo de la primera semana es tener el entorno configurado y conocer el equipo. La productividad plena se espera aproximadamente al mes 3, según tu nivel de seniority.

---

**P: ¿Puedo hacer preguntas "básicas" sin sentirme mal?**

R: Absolutamente sí. Tenés un buddy asignado exactamente para eso. En Santo Pegasus, preguntar es una señal de inteligencia, no de ignorancia. Si una pregunta te parece básica, es probable que también le sirva a otra persona del equipo, así que preguntala en el canal correspondiente en Slack.

---

**P: ¿Qué pasa si rompo algo en el ambiente local?**

R: El ambiente local es exactamente para eso: experimentar, romper y aprender. Nada en tu entorno local afecta producción. Si tenés dudas, tu buddy puede ayudarte a reconstruirlo desde cero con ``docker compose down -v && docker compose up -d``.

---

**P: ¿Puedo pushear código directamente a `develop` o `main`?**

R: No. Las ramas ``develop`` y ``main`` están protegidas y solo reciben código a través de Pull Requests aprobados. Cualquier intento de push directo será rechazado por GitHub.

---

**P: ¿En qué nivel de inglés debo estar para trabajar aquí?**

R: La comunicación interna del equipo es principalmente en español. La documentación técnica (Guías de Ingeniería, ADRs) puede estar en inglés o español según quién la escribió. El código, los comentarios, los commits y los nombres de variables deben estar en inglés. Un nivel de lectura y escritura técnica básica es suficiente para comenzar.

---

**P: ¿Cuándo puedo solicitar el presupuesto de cursos y certificaciones?**

R: El presupuesto está disponible desde el primer mes. Sin embargo, recomendamos que en los primeros 90 días te enfoques en el onboarding y el aprendizaje del contexto del producto. Después de ese período, conversá con tu Tech Lead y People sobre las certificaciones más alineadas con tu plan de carrera.

---

**P: ¿Cómo sé si mi Pull Request fue revisado?**

R: Recibirás una notificación de GitHub por correo electrónico y también en el canal ``#code-reviews`` de Slack. También podés configurar las notificaciones de GitHub directamente en Slack usando la integración de GitHub disponible en el workspace.

---

**P: ¿Qué es un ADR y cómo lo creo?**



R: ADR significa Architecture Decision Record. Es un documento breve que registra una decisión técnica importante: el contexto, las opciones consideradas, la decisión tomada y sus consecuencias. El template está disponible en Confluence: `Engineering → Decisiones de Arquitectura → Template ADR`. Pedile a tu Tech Lead que revise el primero que escribas.

---

P: ¿Puedo usar `System.out.println()` para debug?

R: No en código que va al repositorio. La utilización de `System.out.println()` está estrictamente prohibida en Santo Pegasus. Usamos SLF4J como interfaz de abstracción con Logback como framework de implementación. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 9]

---

P: ¿Cómo reporto un bug que encontré en producción?

R: Abrí un ticket en Jira del tipo Bug en el proyecto del squad correspondiente. Si el bug es crítico y está afectando usuarios en producción ahora mismo, notificará primero en `#incidents` en Slack con una descripción breve y luego creá el ticket.

---

---

## SECCIÓN 15 — DISPOSICIONES FINALES Y REVISIÓN DEL DOCUMENTO

### 15.1 Vigencia del Documento

Este manual entra en vigencia a partir de su fecha de publicación (Junio 2026) y aplica a todos los desarrolladores que se incorporen a Santo Pegasus Soluciones a partir de esa fecha.

---

### 15.2 Responsabilidades

| Rol                             | Responsabilidad                                                                                                                         |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| People & Talent                 | Distribuir este manual a cada nuevo integrante antes del Día 1. Mantener la Sección 10 actualizada.                                     |
| Tech Leads (todos los Chapters) | Asegurar que el checklist de la Sección 12 sea completado en la primera semana. Asignar un buddy a cada nuevo integrante.               |
| Dirección de Ingeniería         | Aprobar las revisiones del documento. Mantener la Sección 1 (Cultura) y Sección 2 (Estructura) alineadas con la realidad de la empresa. |
| IT / Soporte                    | Garantizar que los accesos listados en la Sección 3 estén disponibles en los plazos indicados.                                          |

---

## 15.3 Ciclo de Revisión

**Este documento es revisado formalmente cada 6 meses (próxima revisión: Diciembre 2026). Las actualizaciones pueden ocurrir de forma extraordinaria cuando:**

- Hay cambios significativos en el stack tecnológico.
- Se incorporan o eliminan herramientas del flujo de trabajo.
- Hay cambios en la estructura del equipo o en los beneficios de RRHH.
- Se reciben sugerencias fundamentadas de miembros del equipo.

---

## 15.4 Cómo Contribuir a Este Documento

**Este manual es un documento vivo. Si durante tu onboarding identificás información desactualizada, un paso que falta o una mejora posible:**

1. Hacé un comentario directamente en la página de Confluence.
2. O enviá tu sugerencia al canal `#onboarding-feedback` en Slack.
3. O abrí un ticket en Jira del tipo Task en el proyecto People con el label `docs-improvement`.

**Las contribuciones de nuevos integrantes son especialmente valiosas porque vienen de la perspectiva de alguien que acaba de pasar por el proceso. Tu mirada fresca importa.**

---

## 15.5 Agradecimiento Final

Construir software de calidad es un deporte de equipo. Este manual representa el esfuerzo colectivo de todos los que pasaron por Santo Pegasus antes que vos, documentando lo que les habría gustado saber en su primer día.

Ahora sos parte de ese equipo. En algún momento, quizás seas vos quien ayude a escribir la próxima versión de este documento para darle la bienvenida a alguien nuevo.

**¡Bienvenido a Santo Pegasus Soluciones. El código que escribas hoy importa.**

---

---

**> Versión: 1.0.0 | Última Actualización: Junio 2026 | Próxima Revisión: Diciembre 2026**

**> Departamento Responsable: People & Engineering — Santo Pegasus Soluciones**

**> Clasificación: Interno — Confidencial**

**> Aprobado por: Dirección de Ingeniería y People Business Partner**

---

*Este documento fue elaborado con base en la Guía Oficial de Ingeniería Back-end de Santo Pegasus Soluciones, Versión 2.4.0, Octubre 2025, Departamento de Ingeniería de Software / Chapter de Back-end.*

# PROTOCOLO DE RESPUESTA A INCIDENTES Y POST-MORTEM

## Santo Pegasus Soluciones — Site Reliability Engineering (SRE)

---

**Versión del Documento:** 1.0.0

**Última Actualización:** Junio de 2026

**Departamento:** Ingeniería de Software / Chapter de SRE

**Clasificación:** Documento Interno — Uso Restringido al Equipo de Ingeniería y Dirección

**Propietario:** Chapter Lead de SRE — Santo Pegasus Soluciones

---

> *"La confiabilidad no es una característica opcional. Es el fundamento sobre el cual todos los demás atributos de un sistema son evaluados."*

> — Principio SRE, Santo Pegasus Soluciones

---

## TABLA DE CONTENIDOS

1. Introducción y Filosofía SRE en Santo Pegasus
2. Definiciones Fundamentales: Incidente, Problema y Cambio Planificado
3. Clasificación de Severidad de Incidentes
4. Roles y Responsabilidades Durante un Incidente
5. Proceso de Detección y Alerta
6. Proceso Paso a Paso de Respuesta al Incidente
7. Procedimiento de Rollback en Docker y AWS ECS
8. Comunicación Durante el Incidente
9. Criterios de Resolución y Cierre del Incidente
10. Plantilla Oficial de Post-Mortem
11. Proceso de Revisión y Archivo del Post-Mortem
12. Métricas de Confiabilidad: SLI, SLO y SLA
13. Error Budget: Definición, Cálculo y Política de Agotamiento
14. Simulacros de Incidentes — GameDay
15. Disposiciones Finales, Vigencia y Revisión del Protocolo

---

## 1. INTRODUCCIÓN Y FILOSOFÍA SRE EN SANTO PEGASUS

## 1.1 Propósito del Documento

Este documento establece el Protocolo Oficial de Respuesta a Incidentes y Post-Mortems de Santo Pegasus Soluciones. Su objetivo es garantizar que cualquier degradación o interrupción de los servicios en producción sea detectada, gestionada, mitigada y documentada de forma estructurada, eficiente y reproducible.

La alineación con este protocolo es obligatoria para todos los ingenieros de back-end, DevOps, SRE, líderes técnicos y gestores de producto que operen o supervisen sistemas en ambientes productivos. El cumplimiento de estas directrices asegura que Santo Pegasus mantenga los compromisos de disponibilidad acordados con sus clientes y preserve la integridad operativa de todos sus productos, incluido el sistema Agendio (plataforma de agendamiento de consultas médicas) y demás microservicios críticos.

## 1.2 La Filosofía SRE en Santo Pegasus Soluciones

Site Reliability Engineering (SRE) es la disciplina que combina principios de ingeniería de software con operaciones de infraestructura para construir sistemas escalables, confiables y gestionables. En Santo Pegasus, esta disciplina no es responsabilidad exclusiva de un equipo aislado: es una responsabilidad compartida entre todos los desarrolladores, arquitectos, gestores de producto y ejecutivos que participan en el ciclo de vida del software.

Los principios que guían nuestra práctica SRE son:

### #### 1.2.1 Confiabilidad como Responsabilidad Compartida

Todo ingeniero que escribe código es también responsable por su comportamiento en producción. En Santo Pegasus, quien desarrolla una funcionalidad también participa de su operación, monitoreo y respuesta ante fallos. No existe una separación rígida entre "quienes construyen" y "quienes operan": todos son corresponsables por la salud de los sistemas.

Esto implica que:

- **Los equipos de desarrollo participan activamente en las rotaciones de on-call.**
- Las métricas de confiabilidad (SLIs y SLOs) son métricas de producto, no solo métricas de infraestructura.
- **Las decisiones de velocidad de entrega (cadencia de deploys, volumen de features) están directamente influenciadas por el saldo del Error Budget.**

### #### 1.2.2 Blameless Culture (Cultura Sin Culpa)

Uno de los pilares más críticos de nuestra filosofía SRE es la cultura blameless (sin culpa). En Santo Pegasus, cuando un incidente ocurre, el foco del análisis es siempre el sistema, nunca el individuo. Los incidentes son síntomas de fragilidades sistémicas, no de incompetencias personales.

Esta cultura garantiza que:

- Los ingenieros reporten incidentes con honestidad y detalle, sin temor a represalias.

- Los Post-Mortems sean documentos técnicos de aprendizaje, no instrumentos disciplinarios.
- Las causas raíz sean identificadas en procesos, herramientas y arquitecturas, promoviendo mejoras reales y duraderas.
- El conocimiento generado por cada incidente sea compartido ampliamente con todo el equipo de ingeniería.

#### #### 1.2.3 Automatización como Mecanismo de Confiabilidad

En Santo Pegasus, el trabajo manual repetitivo es un riesgo operacional. Toda tarea que pueda automatizarse debe ser automatizada. Nuestros pipelines CI/CD, estrategias de deploy Blue-Green y Canary Releases, y procesos de rollback automatizado existen precisamente para reducir la dependencia de intervenciones humanas en situaciones de alta presión.

#### #### 1.2.4 Confiabilidad y Velocidad en Equilibrio

**La confiabilidad no debe ser un freno a la innovación, sino un marco que permite innovar de forma segura y sostenida. El concepto de Error Budget (detallado en la Sección 13) es el mecanismo que Santo Pegasus utiliza para equilibrar la velocidad de entrega de nuevas funcionalidades con la necesidad de mantener la estabilidad de los sistemas existentes.**

---

## 2. DEFINICIONES FUNDAMENTALES

Para garantizar un lenguaje común entre todos los equipos, este protocolo adopta las siguientes definiciones operativas:

### 2.1 Incidente

**Un incidente es cualquier evento no planificado que cause o amenace con causar degradación o interrupción de uno o más servicios en producción, con impacto real o potencial sobre los usuarios finales o sobre los SLOs establecidos.**

#### Ejemplos:

- El servicio Agendio presenta error 503 para el 60% de las requisiciones de agendamiento.
- La latencia P99 del endpoint `/consultas/agendar` supera 2000ms por más de 5 minutos.
- Una migración de base de datos en AWS RDS provoca bloqueo de tablas críticas.
- El pipeline de notificaciones por e-mail via AWS SES deja de procesar mensajes de la cola SQS.

### 2.2 Problema

**Un problema es la causa raíz subyacente de uno o más incidentes. Un mismo problema puede manifestarse como múltiples incidentes recurrentes hasta que su causa sea identificada y eliminada.**

**Distinción clave:** El incidente es el síntoma observable; el problema es la causa sistémica. La gestión de incidentes trata el síntoma para restaurar el servicio. La gestión de problemas investiga y elimina la causa para evitar recurrencia.

**Ejemplo:**

- **Incidentes:** El servicio de notificaciones falló tres veces en un mes.
- **Problema:** Un mecanismo de retry sin backoff exponencial provoca tormentas de mensajes en SQS, saturando el consumidor.

2.3 Cambio Planificado

Un cambio planificado es cualquier alteración deliberada en los sistemas de producción ejecutada de forma coordinada, comunicada con anticipación y dentro de una ventana de mantenimiento autorizada. Los cambios planificados no se clasifican como incidentes aunque causen indisponibilidad temporal y programada.

**Requisitos para un cambio planificado válido:**

- Aprobación del Incident Commander de turno y del líder técnico del servicio afectado.
- Comunicación previa de al menos 48 horas para clientes y stakeholders internos.
- Plan de rollback documentado y validado.
- Ventana de mantenimiento definida y acordada.

---

3. CLASIFICACIÓN DE SEVERIDAD DE INCIDENTES

3.1 Tabla de Severidades

| Nivel | Nombre  | Descripción del Impacto                                                                                              | Ejemplos Concretos                                                                                                                                      | SLA de Respuesta | SLA de Actualización |
|-------|---------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------|
| SEV-1 | Crítico | Sistema completamente caído. Impacto total en producción. Todos los clientes afectados. Revenue en riesgo inmediato. | Agendio inaccessible, API Gateway caída, base de datos de producción inalcanzable, fallo total de autenticación.                                        | 15 minutos       | Cada 30 minutos      |
| SEV-2 | Alto    | Degradación severa. Funcionalidad crítica no disponible para parte significativa de los clientes.                    | Agendamiento de consultas falla para 40% de los usuarios, notificaciones por e-mail (SES) completamente detenidas, latencia P99 > 5 segundos sostenida. | 30 minutos       | Cada 1 hora          |

|       |       |                                                                                                                 |                                                                                                                                                                                                          |                   |              |
|-------|-------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|--------------|
| SEV-3 | Medio | Degradación leve. Funcionalidad no crítica afectada. Existe workaround disponible para los usuarios.            | Reportes de histórico con lentitud, búsqueda de disponibilidad retornando resultados desactualizados, dashboard administrativo con visualización parcial.                                                | 4 horas           | Cada 4 horas |
| SEV-4 | Bajo  | Anomalía detectada en métricas o logs. Sin impacto visible al usuario final. Requiere investigación preventiva. | Aumento inusual en tasa de errores 4xx sin impacto funcional, métrica de CPU por encima del umbral de alerta pero dentro del límite operativo, log de warning recurrente no relacionado a flujo crítico. | Próximo día hábil | Al cierre    |

### 3.2 Criterios de Escalada de Severidad

La severidad de un incidente puede y debe ser revisada durante su ciclo de vida. Los criterios para escalar (aumentar la severidad) son:

- El impacto se expande a un mayor número de usuarios o servicios.
- El workaround disponible deja de funcionar.
- El tiempo de resolución supera el doble del SLA de respuesta de la severidad actual.
- Hay evidencia de impacto económico o contractual.

Los criterios para desescalar (reducir la severidad) son:

- La mitigación aplicada reduce significativamente el impacto.
- Un workaround efectivo queda disponible para todos los usuarios afectados.
- Las métricas de SLO retornan a niveles normales.

### 3.3 Autoridad para Declarar y Modificar la Severidad

El Incident Commander (IC) es la única figura con autoridad para declarar formalmente un incidente y definir o modificar su nivel de severidad. En ausencia del IC principal, el Technical Lead de turno asume esta responsabilidad hasta que el IC sea asignado.

---

## 4. ROLES Y RESPONSABILIDADES DURANTE UN INCIDENTE

## 4.1 Descripción de Roles

### #### 4.1.1 Incident Commander (IC)

**El Incident Commander es la autoridad máxima durante un incidente activo. Su función no es resolver técnicamente el problema, sino coordinar al equipo, garantizar el flujo de información y tomar decisiones de alto nivel.**

#### **Responsabilidades:**

- Declarar formalmente el incidente y su severidad.
- Asignar los demás roles (Communications Lead, Technical Lead, SMEs).
- Abrir el canal de War Room en Slack y convocar a los participantes necesarios.
- Mantener el timeline del incidente actualizado en tiempo real.
- Tomar la decisión final sobre rollback, mitigación o continuación del diagnóstico.
- Comunicar el cierre del incidente y garantizar la apertura del proceso de Post-Mortem.
- Asegurar que nadie en la War Room actúe sin coordinación.

**Perfil: Senior Engineer o Tech Lead con experiencia en el ecosistema productivo de Santo Pegasus. Debe haber completado al menos un ciclo de shadow (observación) del rol antes de asumir de forma independiente.**

### #### 4.1.2 Communications Lead

**El Communications Lead es el responsable de toda la comunicación externa e interna durante el incidente, liberando al IC para enfocarse en la coordinación técnica.**

#### **Responsabilidades:**

- Publicar actualizaciones de estado en el canal `#incidents` de Slack según los SLAs de actualización.
- Redactar y enviar comunicados externos a clientes afectados (vía e-mail o status page).
- Comunicar el estado del incidente a la dirección y stakeholders ejecutivos.
- Gestionar la status page pública (cuando aplique).
- Documentar en el canal de War Room todas las acciones tomadas y sus timestamps.
- Coordinar con el equipo de Soporte al Cliente cuando sea necesario.

**Perfil: Puede ser un engineer senior, un product manager técnico o un líder de equipo. Debe tener habilidades claras de comunicación escrita y comprensión suficiente del impacto técnico para traducirlo a lenguaje ejecutivo y orientado al cliente.**

### #### 4.1.3 Technical Lead

**El Technical Lead lidera la investigación técnica del incidente. Es quien dirige activamente el diagnóstico, propone hipótesis, coordina a los SMEs y ejecuta o autoriza las acciones de mitigación y rollback.**

#### **Responsabilidades:**

- Liderar el diagnóstico técnico del incidente en la War Room.



- Formular y priorizar hipótesis de causa raíz.
- Coordinar a los SMEs en sus áreas de especialidad.
- Proponer y ejecutar acciones de mitigación al IC para aprobación.
- Autorizar y supervisar la ejecución de procedimientos de rollback.
- Verificar que el sistema retorna a estado estable tras la mitigación.
- Documentar las hipótesis descartadas y confirmadas para el Post-Mortem.

**Perfil: Engineer senior o pleno con dominio técnico del servicio afectado. Idealmente el responsable técnico del microservicio en cuestión.**

#### #### 4.1.4 Subject Matter Expert (SME)

**Los SMEs son especialistas convocados según la naturaleza técnica del incidente. Pueden ser incorporados al equipo de respuesta en cualquier momento del ciclo de vida del incidente.**

#### Responsabilidades:

- Responder a consultas técnicas específicas dentro de su dominio de conocimiento.
- Ejecutar tareas técnicas delegadas por el Technical Lead.
- Proporcionar contexto sobre sistemas, integraciones o configuraciones específicas.
- Participar activamente en el análisis de causa raíz durante el Post-Mortem.

#### Ejemplos de SMEs en Santo Pegasus:

- Especialista en AWS RDS y PostgreSQL (incidentes de base de datos).
- Especialista en AWS ECS y Docker (incidentes de infraestructura y deploy).
- Especialista en AWS SQS/SES (incidentes de mensajería y notificaciones).
- Especialista en Spring Security y JWT (incidentes de autenticación y autorización).

### 4.2 Matriz de Responsabilidades (RACI)

| Actividad                | Incident Commander | Communications Lead | Technical Lead | SME |
|--------------------------|--------------------|---------------------|----------------|-----|
| Declarar el incidente    | R/A                | I                   | C              | I   |
| Definir la severidad     | R/A                | I                   | C              | C   |
| Abrir War Room           | R/A                | R                   | I              | I   |
| Comunicación externa     | A                  | R                   | I              | —   |
| Comunicación a dirección | A                  | R                   | I              | —   |
| Diagnóstico técnico      | I                  | —                   | R/A            | R   |
| Ejecutar rollback        | A                  | I                   | R              | C   |
| Actualizar timeline      | A                  | R                   | C              | —   |

|                     |     |   |   |   |
|---------------------|-----|---|---|---|
| Declarar resolución | R/A | I | C | I |
| Iniciar Post-Mortem | R/A | I | R | I |

R = Responsable / A = Aprobador / C = Consultado / I = Informado

---

## 5. PROCESO DE DETECCIÓN Y ALERTA

### 5.1 Fuentes de Detección

**Santo Pegasus opera un sistema de observabilidad en capas, conforme a los tres pilares definidos en la Guía Oficial de Ingeniería Back-end: logs, métricas y rastreo. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 10]**

#### #### 5.1.1 Métricas — Prometheus y Datadog

**Las métricas de rendimiento y salud son expuestas mediante Spring Boot Actuator y Micrometer, con exportación nativa hacia Prometheus. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 10] Datadog actúa como plataforma centralizada de visualización, correlación y alerting.**

**Tipos de alertas configuradas:**

| Métrica                     | Umbral de Alerta (Warning) | Umbral de Alerta (Critical) | Severidad Sugerida |
|-----------------------------|----------------------------|-----------------------------|--------------------|
| Disponibilidad del servicio | < 99.95% en 5 min          | < 99.5% en 5 min            | SEV-2 / SEV-1      |
| Latencia P99 del endpoint   | > 300ms en 10 min          | > 500ms en 5 min            | SEV-3 / SEV-2      |
| Tasa de error HTTP 5xx      | > 0.05% en 5 min           | > 0.1% en 5 min             | SEV-3 / SEV-2      |
| CPU de instancia ECS        | > 75% por 10 min           | > 90% por 5 min             | SEV-3 / SEV-1      |
| Profundidad de cola SQS     | > 1.000 mensajes           | > 5.000 mensajes            | SEV-3 / SEV-2      |
| Conexiones disponibles RDS  | < 20% del pool             | < 5% del pool               | SEV-2 / SEV-1      |
| Health check del contenedor | Falla 1 vez                | Falla 3 veces consecutivas  | SEV-3 / SEV-1      |

Los endpoints de health check deben reflejar el estado real de las dependencias críticas (bases de datos, brokers de mensajes), permitiendo que el orquestador de contenedores tome decisiones precisas sobre el ciclo de vida de las instancias. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 11]

#### #### 5.1.2 Logs — SLF4J y Logback

El sistema de logging estandarizado en SLF4J (Simple Logging Facade for Java) con Logback como framework de implementación nativo de Spring Boot [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 9] genera alertas automáticas cuando:

- El volumen de logs en nivel ERROR supera el umbral configurado en Datadog.
- Patrones de error específicos (como `NullPointerException` en flujos críticos, o errores de conexión con RDS) son detectados por reglas de parsing.
- Stacktraces de excepciones de negocio personalizadas (`ResourceNotFoundException`, `BusinessRuleException`) aparecer en frecuencia anómala.

**Nota: El uso de `System.out.println()` está estrictamente prohibido en todos los servicios de Santo Pegasus. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 9] Toda la instrumentación de logs debe pasar por SLF4J.**

#### #### 5.1.3 Rastreo Distribuido (Distributed Tracing)

En la arquitectura de microservicios, es obligatorio implementar soluciones de rastreo para propagar el Trace ID y el Span ID entre las peticiones. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 10] Estas trazas permiten identificar cuellos de botella entre servicios y correlacionar eventos de múltiples microservicios durante un incidente.

#### #### 5.1.4 Reporte Manual por Usuarios o Soporte

Un incidente también puede ser detectado a través de:

- Reporte directo de usuarios finales al equipo de Soporte al Cliente.
- Alerta interna comunicada por un desarrollador que observa comportamiento anómalo.
- Alerta de un cliente corporativo vía canal dedicado de comunicación.

Todo reporte manual debe ser inmediatamente evaluado por el IC de turno para determinar si justifica la declaración formal de un incidente.

## 5.2 Canales de Comunicación de Incidentes

| Canal                 | Propósito                                                                         | Audiencia                                                |
|-----------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------|
| `#incidents`          | Canal general de monitoreo y declaración de todos los incidentes                  | Todos los ingenieros, PMs, dirección técnica             |
| `#sev1-war-room`      | War Room exclusiva para SEV-1. Canal dedicado, criado en el momento del incidente | IC, Communications Lead, Technical Lead, SMEs convocados |
| `#sev2-war-room`      | War Room para SEV-2                                                               | IC, Communications Lead, Technical Lead, SMEs            |
| `#incidents-resolved` | Registro histórico de incidentes resueltos y links a Post-Mortems                 | Todos los ingenieros                                     |
| PagerDuty             | Alertas automáticas de on-call, escalada y asignación de roles                    | Engineer on-call, IC de turno                            |

## 5.3 Política de On-Call y PagerDuty

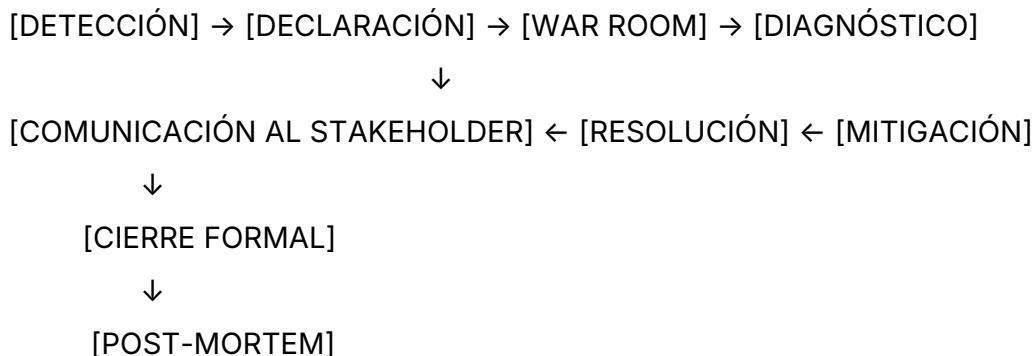
- Todos los ingenieros Senior y Semi-Senior (Pleno) participan obligatoriamente en la rotación de on-call.
- La rotación ocurre semanalmente, con un Primary On-Call y un Secondary On-Call (backup).
- Las alertas SEV-1 y SEV-2 activan notificación inmediata via PagerDuty para el Primary On-Call.
- Si el Primary no reconoce la alerta en 5 minutos, el Secondary es notificado automáticamente.
- Si el Secondary no responde en 5 minutos adicionales, el IC de plantón y el Tech Lead de la semana son notificados.
- El ingeniero on-call tiene hasta 15 minutos para declarar formalmente el incidente o descartarlo con justificación documentada en `#incidents`.

---

## 6. PROCESO PASO A PASO DE RESPUESTA AL INCIDENTE

### 6.1 Diagrama del Flujo de Respuesta

...



...

### 6.2 Fase 1 — Detección (T+0)

**Responsable:** Engineer On-Call / Sistema automatizado

#### Checklist:

- [ ] Alerta recibida via PagerDuty o Datadog.
- [ ] Engineer on-call reconoce la alerta en PagerDuty (botón "Acknowledge").
- [ ] Verificar en Datadog/Prometheus si la alerta es legítima o falso positivo.
- [ ] Consultar logs en Datadog (filtro por nivel ERROR o WARN en el servicio afectado).
- [ ] Verificar Trace ID en la solución de rastreo distribuido para identificar el microservicio de origen.
- [ ] Consultar el health check del servicio en AWS ECS.

- [ ] Si confirmado: proceder a Fase 2. Si falso positivo: documentar en `#incidents` y cerrar alerta.

## 6.3 Fase 2 — Declaración del Incidente (T+0 a T+5 minutos)

**Responsable: Engineer On-Call → Incident Commander**

**Checklist:**

- [ ] Engineer on-call publica mensaje inicial en `#incidents` (ver template en Sección 8.1).
- [ ] IC de turno recibe notificación via PagerDuty y asume el comando.
- [ ] IC declara formalmente el incidente con severidad inicial (puede ser revisada).
- [ ] IC crea el canal de War Room correspondiente en Slack (`#sev1-war-room` o `#sev2-war-room`).
- [ ] IC asigna formalmente los roles: Communications Lead, Technical Lead, SMEs necesarios.
- [ ] IC abre el documento de timeline del incidente (Google Doc o Confluence) y comparte el link en el canal de War Room.
- [ ] IC publica la declaración formal en el canal de War Room (ver template en Sección 8.2).

## 6.4 Fase 3 — Guerra Room y Diagnóstico (T+5 a T+variable)

**Responsable: Technical Lead + SMEs**

**Checklist de Diagnóstico:**

- [ ] Technical Lead formula hipótesis iniciales basadas en la alerta y los síntomas observados.
- [ ] Revisar el historial de deploys recientes: ¿hubo un deploy en las últimas 2 horas?
- [ ] Revisar el historial de cambios de configuración en AWS ECS, RDS o variables de entorno.
- [ ] Analizar métricas de Prometheus: latencia, tasa de error, CPU, memoria, conexiones RDS.
- [ ] Analizar logs estructurados en Datadog: filtrar por nivel ERROR, buscar stacktraces relevantes.
- [ ] Utilizar el Trace ID para rastrear el flujo completo de una requisición fallida entre microservicios.
- [ ] Revisar el estado de la cola SQS: mensajes en DLQ (Dead Letter Queue), profundidad de cola.
- [ ] Verificar el estado del servicio AWS RDS: conexiones activas, queries de larga duración, locks.
- [ ] Consultar el Health Check de los contenedores en AWS ECS.
- [ ] Cada hipótesis descartada debe ser documentada en el timeline con timestamp.
- [ ] Technical Lead reporta el estado del diagnóstico al IC a cada actualización significativa.

## 6.5 Fase 4 — Mitigación (T+variable)

**Responsable: Technical Lead (con aprobación del IC)**

**La mitigación es la acción que restaura el servicio al usuario, aunque no necesariamente elimine la causa raíz. El objetivo de la mitigación es reducir el impacto en el menor tiempo posible.**

**Acciones de mitigación comunes:**

| Causa Identificada                      | Acción de Mitigación                                             | Ejecutor                    |
|-----------------------------------------|------------------------------------------------------------------|-----------------------------|
| Deploy defectuoso                       | Rollback inmediato (ver Sección 7)                               | Technical Lead + SME DevOps |
| Esgotamiento del pool de conexiones RDS | Reiniciar instancias ECS, aumentar pool temporalmente            | SME RDS                     |
| Cola SQS con mensajes venenosos         | Mover mensajes problemáticos a DLQ, reiniciar consumidor         | SME Backend                 |
| Tormenta de retries                     | Activar Circuit Breaker, aumentar visibilidad de mensajes en SQS | Technical Lead              |
| Instancia ECS defectuosa                | Forzar nuevo deployment para reemplazar la tarea                 | SME DevOps                  |
| Leak de memoria en el contenedor        | Reiniciar el servicio, escalar horizontalmente para ganar tiempo | SME DevOps                  |

**Regla de oro de la mitigación:**

*> Primero estabilizar. Después entender. La presión para encontrar la causa raíz no debe retardar la restauración del servicio.*

- ☐ Technical Lead propone la acción de mitigación al IC.
- ☐ IC aprueba la acción.
- ☐ SME o Technical Lead ejecuta la acción.
- ☐ **Monitorear las métricas en Datadog/Prometheus durante los 5 minutos siguientes a la mitigación.**
- ☐ Confirmar que el impacto se redujo o eliminó.
- ☐ Communications Lead publica actualización en `#incidents` y para clientes (si aplica).

## 6.6 Fase 5 — Resolución (T+variable)

**Responsable: Technical Lead → IC**

**La resolución ocurre cuando:**

1. El servicio retorna a operación normal, dentro de los SLOs establecidos.
2. El impacto sobre los usuarios ha cesado completamente.

**3. Las métricas de SLI se estabilizan dentro de los umbrales normales por al menos 10 minutos consecutivos.**

- ☐ Technical Lead informa al IC que las condiciones de resolución han sido cumplidas.
- ☐ IC valida las métricas en Datadog y Prometheus.

- [ ] IC declara formalmente la resolución del incidente.
- [ ] Communications Lead publica comunicado de resolución (ver template Sección 8.4).

## 6.7 Fase 6 — Comunicación al Stakeholder y Cierre Formal

### Responsable: IC + Communications Lead

- [ ] Communications Lead envía comunicado final a todos los públicos afectados.
- [ ] IC registra el cierre formal del incidente en el sistema de tracking (Jira, PagerDuty).
- [ ] IC anuncia en `#incidents` el cierre del incidente y el link del timeline completo.
- [ ] IC designa el responsable del Post-Mortem (normalmente el Technical Lead del incidente).
- [ ] IC define el plazo para la entrega del rascunho del Post-Mortem (máximo 72 horas para SEV-1 y SEV-2; 5 días hábiles para SEV-3 y SEV-4).

---

## 7. PROCEDIMIENTO DE ROLLBACK EN DOCKER Y AWS ECS

### 7.1 Filosofía del Rollback

El rollback es la acción de reversión de un deploy defectuoso hacia la última versión estable conocida del servicio. En Santo Pegasus, el uso de Docker es obligatorio para asegurar la inmutabilidad del artefacto, garantizando que el mismo código validado en Staging (homologación) sea promovido a Producción sin variaciones. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 12]

Esta inmutabilidad hace del rollback una operación rápida y segura: revertir significa simplemente redirigir el orquestador para servir la imagen Docker de la versión anterior, que ya fue validada en Staging.

### 7.2 Estrategias de Deploy y su Impacto en el Rollback

Santo Pegasus utiliza orquestadores que soportan Blue-Green o Canary Releases, permitiendo reversiones rápidas en caso de anomalías. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 12]

#### #### 7.2.1 Rollback en Blue-Green Deployment

En Blue-Green, dos ambientes idénticos (Blue = versión estable, Green = nueva versión) son mantenidos en paralelo. El rollback consiste en redirigir el tráfico del Load Balancer del ambiente Green de vuelta al ambiente Blue.

```bash

ROLLBACK BLUE-GREEN — AWS ECS + Application Load Balancer

Paso 1: Identificar el ARN del Target Group azul (versión estable)

```
aws elbv2 describe-target-groups \  
  --names "santo-pegasus-blue-tg" \  
  --query "TargetGroups[0].TargetGroupArn" \  
  --output text
```

Paso 2: Redirigir el listener del Load Balancer al Target Group azul

```
aws elbv2 modify-listener \  
  --listener-arn <ARN_DO_LISTENER> \  
  --default-actions Type=forward,TargetGroupArn=<ARN_DO_TARGET_GROUP_BLUE>
```

Paso 3: Verificar que el tráfico está siendo dirigido al Target Group correcto

```
aws elbv2 describe-listeners \  
  --listener-arns <ARN_DO_LISTENER> \  
  --query "Listeners[0].DefaultActions"
```

Paso 4: Monitorear las métricas de health check del Target Group azul

```
aws elbv2 describe-target-health \  
  --target-group-arn <ARN_DO_TARGET_GROUP_BLUE>  
...
```

7.2.2 Rollback en Canary Release

En Canary, la nueva versión recibe un porcentaje pequeño del tráfico (ej. 10%). El rollback consiste en redirigir el 100% del tráfico de vuelta a la versión estable.

```
```bash
```

## ROLLBACK CANARY — Redirigir 100% del tráfico a la versión estable



## Paso 1: Modificar las reglas del listener para 100% en versión estable

```
aws elbv2 modify-rule \
 --rule-arn <ARN_DA_REGRA_CANARY> \
 --actions '[
 {
 "Type": "forward",
 "ForwardConfig": {
 "TargetGroups": [
 {"TargetGroupArn": "<ARN_STABLE_TG>", "Weight": 100},
 {"TargetGroupArn": "<ARN_CANARY_TG>", "Weight": 0}
]
 }
 }
]'
```

## Paso 2: Confirmar la distribución de tráfico

```
aws elbv2 describe-rules \
 --rule-arns <ARN_DA_REGRA_CANARY>
...
```

#### 7.2.3 Rollback Directo en AWS ECS (Force New Deployment)

Cuando es necesario revertir la imagen Docker de un servicio ECS directamente:

```
```bash
```

ROLLBACK DIRECTO DE IMAGEN EN AWS ECS

Paso 1: Listar las revisiones recientes de la Task Definition del servicio

```
aws ecs list-task-definitions \  
  --family-prefix "santo-pegasus-agendio" \  
  --sort DESC \  
  --max-items 5
```

Paso 2: Identificar la última revisión ESTABLE (ej: revisión N-1)

Supongamos que la revisión actual defectuosa es :42 y la estable es :41

Paso 3: Actualizar el servicio ECS para usar la revisión estable

```
aws ecs update-service \  
  --cluster "santo-pegasus-production" \  
  --service "agendio-service" \  
  --task-definition "santo-pegasus-agendio:41" \  
  --force-new-deployment
```

Paso 4: Monitorear el estado del deployment

```
aws ecs describe-services \  
  --cluster "santo-pegasus-production" \  
  --services "agendio-service" \  
  --query "services[0].deployments"
```

Paso 5: Verificar que las nuevas tareas están en estado RUNNING

```
aws ecs list-tasks \  
  --cluster "santo-pegasus-production" \  
  --service-name "agendio-service" \  
  --desired-status RUNNING
```

Paso 6: Verificar health checks de los contenedores

```
aws ecs describe-tasks \  
  --cluster "santo-pegasus-production" \  
  --tasks <TASK_ARN_1> <TASK_ARN_2>  
...
```

7.3 Checklist Completo de Rollback

- [] IC autoriza formalmente el inicio del procedimiento de rollback.
- [] SME DevOps o Technical Lead identifica la versión estable anterior (imagen Docker o Task Definition revision).

- [] Verificar que la versión estable no contiene la causa del incidente actual (consultar el changelog del deploy).
- [] Verificar si hay migraciones de base de datos pendientes que puedan ser afectadas por el rollback.
- [] **Si sí: evaluar rollback de la migración con Flyway o Liquibase antes de revertir la aplicación.**
- [] Si no: proceder directamente con el rollback de la imagen.
- [] Ejecutar el procedimiento de rollback (Blue-Green, Canary o Force New Deployment según el contexto).
- [] **Monitorear métricas en Datadog durante los 10 minutos siguientes.**
- [] Confirmar que el servicio está healthy y las métricas de SLI están dentro de los umbrales normales.
- [] Documentar el rollback en el timeline del incidente con timestamp, versión revertida y versión de destino.
- [] Communications Lead publica actualización informando la acción tomada.

7.4 Rollback de Migraciones de Base de Datos

La evolución del esquema de bases de datos relacionales en Santo Pegasus debe ser estrictamente controlada, versionada y automatizada. [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 11] Las herramientas Flyway o Liquibase se utilizan obligatoriamente.

```
```bash
```

# ROLLBACK DE MIGRACIÓN — FLYWAY

## Paso 1: Verificar el historial de migraciones

```
flyway -url=jdbc:postgresql://<RDS_HOST>:5432/<DB_NAME> \
 -user=<DB_USER> \
 -password=<DB_PASSWORD> \
 info
```

## Paso 2: Ejecutar el script de undo (si disponible — requiere Flyway Teams)

```
flyway -url=jdbc:postgresql://<RDS_HOST>:5432/<DB_NAME> \
 -user=<DB_USER> \
 -password=<DB_PASSWORD> \
 undo
```

**IMPORTANTE: Toda migración debe ser idempotente y probada**

**exhaustivamente en ambientes de homologación antes del deploy en producción.**

**[Source: Guía Oficial de Ingeniería Back-end.pdf, Page 11]**

...

**▲ Advertencia crítica:** La alteración manual de estructuras de datos, como la creación de tablas o modificación de columnas, así como la ejecución de scripts DDL directamente en entornos de producción por medio de clientes SQL, está terminantemente prohibida en Santo Pegasus.  
[Source: Guía Oficial de Ingeniería Back-end.pdf, Page 11]

---

## **8. COMUNICACIÓN DURANTE EL INCIDENTE**

### **8.1 Template — Alerta Inicial en `#incidents`**

...

[INCIDENTE DECLARADO] — SEV-**{NIVEL}**

Hora de Detección: **{HH:MM}** (UTC-3)

Servicio Afectado: **{NOMBRE DEL SERVICIO / MICROSERVICIO}**

Impacto: **{DESCRIPCIÓN BREVE DEL IMPACTO OBSERVABLE}**

Usuarios Afectados: **{ESTIMATIVA — Ej: ~40% de los usuarios del Agendio}**

Métricas: **{DESCRIPCIÓN BREVE — Ej: Latencia P99 = 2.300ms, Error rate = 3.2%}**

Incident Commander: **@{NOMBRE\_IC}**

Technical Lead: **@{NOMBRE\_TL}**

Communications Lead: **@{NOMBRE\_CL}**

War Room: **#{CANAL\_WAR\_ROOM}**

Timeline: **{LINK AL DOCUMENTO DE TIMELINE}**

Status: INVESTIGANDO

...

## 8.2 Template — Declaración Formal en el Canal de War Room

...

### DECLARACIÓN FORMAL DE INCIDENTE

ID del Incidente: INC-{ANO}-{NÚMERO SECUENCIAL}

Severidad: SEV-{NIVEL}

Hora de Declaración: {HH:MM} (UTC-3)

### DESCRIPCIÓN DEL SÍNTOMA:

{Descripción técnica detallada del comportamiento observado}

### SERVICIOS AFECTADOS:

- {Servicio 1}
- {Servicio 2}

### HIPÓTESIS INICIAL:

{Primera hipótesis técnica basada en los síntomas}

### ACCIONES EN CURSO:

- {Acción 1} — Responsable: @{NOMBRE}
- {Acción 2} — Responsable: @{NOMBRE}

PRÓXIMA ACTUALIZACIÓN: {HH:MM} (UTC-3)

...

## 8.3 Template — Actualización de Estado (a publicar en `#incidents`)

...

[ACTUALIZACIÓN SEV-{NIVEL}] INC-{ANO}-{NÚMERO} — {HH:MM}

Tiempo de Incidente: {DURACIÓN DESDE LA DECLARACIÓN}

Estado Actual: {INVESTIGANDO / MITIGANDO / MONITOREANDO}

### PROGRESO:

{Acción completada 1}

{Acción completada 2}

{Acción en curso}

### PRÓXIMOS PASOS:

→ {Próxima acción}

Próxima actualización en {X} minutos.

...

## 8.4 Template — Comunicado de Resolución

...

[INCIDENTE RESUELTO] INC-{ANO}-{NÚMERO} — {HH:MM}

Duración Total: {DURACIÓN}

Servicio: {NOMBRE DEL SERVICIO}

RESUMEN:

{Descripción breve del incidente y de la acción que lo resolvió}

ACCIÓN TOMADA:

{Descripción de la mitigación aplicada — Ej: Rollback a la versión 2.3.1 del servicio Agendio}

ESTADO ACTUAL:

Servicio operando normalmente

Métricas dentro de los parámetros normales

Sin impacto residual para los usuarios

PRÓXIMOS PASOS:

- Post-Mortem será publicado en hasta {PLAZO}
- Link del timeline completo: {LINK}

Agradecemos la paciencia y colaboración de todos.

...

## 8.5 Template — Comunicado Externo a Clientes Afectados

...

Asunto: [Actualización de Status] Intermittencia en {NOMBRE DEL SERVICIO} — Santo Pegasus

Estimado cliente,

Informamos que identificamos una intermitencia que afectó {DESCRIPCIÓN DEL SERVICIO AFECTADO} entre las {HORA DE INICIO} y las {HORA DE RESOLUCIÓN} (horario de Brasília).

IMPACTO IDENTIFICADO:

{Descripción clara y no técnica del impacto sobre el usuario}

ACCIONES TOMADAS:

Nuestro equipo de ingeniería identificó la causa del problema y aplicó las correcciones necesarias. El servicio fue restaurado a las {HORA DE RESOLUCIÓN} y opera normalmente desde entonces.

PRÓXIMOS PASOS:

Realizaremos un análisis detallado del ocurrido (Post-Mortem) e implementaremos mejoras para prevenir la recurrencia de este tipo de evento.

Pedimos disculpas por los inconvenientes causados. Nuestro equipo permanece

comprometido con la calidad y disponibilidad de los servicios.  
Para dudas adicionales, contáctenos en: sre@santopegasus.com.br  
Atenciosamente,  
Equipo de Ingeniería — Santo Pegasus Soluciones  
...

### 8.6 Template — Comunicado a la Dirección (Mensaje Ejecutivo)

...  
RESUMEN EJECUTIVO DE INCIDENTE — SEV-{NIVEL}

Para: Dirección / C-Level  
De: Chapter Lead SRE / Incident Commander  
Fecha: {FECHA}

RESUMEN EN UNA LÍNEA:  
{Una frase describiendo el incidente y su resolución}

- DATOS CLAVE:
- Inicio: {HORA DE INICIO}
  - Resolución: {HORA DE RESOLUCIÓN}
  - Duración Total: {DURACIÓN}
  - Clientes/Usuarios Afectados: {NÚMERO O PORCENTAJE ESTIMADO}
  - SLO Impactado: {SLO AFECTADO — Ej: Disponibilidad del Agendio}

IMPACTO ESTIMADO:  
• {Descripción del impacto de negocio si es conocido}

ACCIÓN TOMADA:  
{Descripción ejecutiva de la solución aplicada}

- PRÓXIMOS PASOS:
- Post-Mortem con análisis de causa raíz: {FECHA DE ENTREGA}
  - Reunión de revisión del Post-Mortem: {FECHA PROPUESTA}

...  
---

## 9. CRITERIOS DE RESOLUCIÓN Y CIERRE DEL INCIDENTE

### 9.1 Criterios Técnicos de Resolución

Un incidente es considerado resuelto cuando todas las condiciones a continuación son satisfechas simultáneamente:

Criterio	Condición de Satisfacción
----------	---------------------------

Disponibilidad del servicio	≥ 99.9% en los últimos 10 minutos consecutivos
Latencia P99	< 500ms sostenida por al menos 10 minutos
Tasa de error HTTP 5xx	< 0.1% por al menos 10 minutos
Health checks de ECS	Todos los contenedores en estado HEALTHY
Profundidad de cola SQS	Retornando a los niveles basales normales
Confirmación del Technical Lead	Technical Lead declara formalmente el servicio estable
Ausencia de nuevas alertas	Ninguna nueva alerta crítica en los últimos 10 minutos

## 9.2 Procedimiento de Cierre

- [ ] Technical Lead confirma al IC que todos los criterios técnicos fueron satisfechos.
- [ ] IC valida independientemente las métricas en Datadog y Prometheus.
- [ ] IC declara el cierre formal en el canal de War Room y en `#incidents`.
- [ ] Communications Lead envía comunicados de resolución (clientes, `#incidents`, dirección si aplica).
- [ ] IC registra el cierre en el sistema de tracking (Jira/PagerDuty) con timestamps completos.
- [ ] IC abre formalmente el ticket de Post-Mortem y asigna el responsable.
- [ ] El canal de War Room es archivado (no eliminado) para preservar el historial.

## 9.3 Periodo de Observación Post-Resolución

**Para SEV-1 y SEV-2, se recomienda un periodo de observación activa de 1 hora después de la declaración de resolución. Durante este periodo:**

- El engineer on-call mantiene monitoreo activo de las métricas.
- Se está preparado para re-escalar el incidente si ocurre una regresión.
- Si la regresión ocurre, se reabre el incidente existente (no se crea uno nuevo) y se reinicia el proceso desde la Fase 3.

---

# 10. PLANTILLA OFICIAL DE POST-MORTEM

## 10.1 Principios del Post-Mortem en Santo Pegasus

El Post-Mortem es el documento más importante que resulta de un incidente. Es la fuente de aprendizaje organizacional. Debe ser:

- **Blameless: Ningún individuo es culpado. El foco es el sistema.**
- **Honesto: Todos los errores y malas decisiones tomadas durante el incidente son documentados.**



- **Accionable:** Cada causa raíz identificada debe resultar en una o más acciones correctivas concretas.
- **Compartido:** Accesible a todo el equipo de ingeniería de Santo Pegasus.

---

**PLANTILLA DE POST-MORTEM — SANTO PEGASUS SOLUCIONES**

...

POST-MORTEM OFICIAL

Santo Pegasus Soluciones — Chapter de SRE

ID del Incidente: INC-{ANO}-{NÚMERO}  
Severidad: SEV-{NIVEL}  
Fecha del Incidente: {DD/MM/AAAA}  
Fecha del PM: {DD/MM/AAAA}  
Autor: {NOMBRE DEL AUTOR}  
Revisores: {NOMBRES DE LOS REVISORES}  
Status del PM: [BORRADOR / EN REVISIÓN / APROBADO]

SECCIÓN 1 — RESUMEN EJECUTIVO

{Párrafo de 3 a 5 líneas describiendo el incidente, su duración, el impacto principal y la acción que lo resolvió.  
Debe ser comprensible por un lector no técnico.}

Ejemplos:

"El día {FECHA}, el servicio Agendio presentó indisponibilidad total del endpoint de agendamiento de consultas por un período de {DURACIÓN}. El incidente fue causado por {CAUSA RAÍZ BREVE} y afectó aproximadamente {N} usuarios. El servicio fue restaurado mediante {ACCIÓN DE RESOLUCIÓN}."

SECCIÓN 2 — TIMELINE DEL INCIDENTE

Timestamp (UTC-3)	Evento	Responsable
HH:MM	Datadog detecta aumento en error rate > 0.1%	Sistema

HH:MM	PagerDuty notifica Engineer On-Call	Sistema
HH:MM	Engineer On-Call reconoce la alerta	{NOMBRE}
HH:MM	Incidente declarado — SEV-{N}	{IC}
HH:MM	War Room abierta en #{CANAL}	{IC}
HH:MM	Technical Lead asume el diagnóstico	{TL}
HH:MM	Hipótesis 1 formulada: {DESCRIPCIÓN}	{TL}
HH:MM	Hipótesis 1 descartada: {RAZÓN}	{TL/SME}
HH:MM	Hipótesis 2 formulada: {DESCRIPCIÓN}	{TL}
HH:MM	Causa raíz identificada: {DESCRIPCIÓN}	{TL/SME}
HH:MM	Rollback/Mitigación autorizado por IC	{IC}
HH:MM	Acción de mitigación ejecutada	{TL/SME}
HH:MM	Métricas retornan a niveles normales	Sistema
HH:MM	Incidente declarado RESUELTO	{IC}
HH:MM	Comunicado de resolución enviado a clientes	{CL}

Duración Total: {DURACIÓN} (desde la declaración hasta la resolución)

Duración del Impacto: {DURACIÓN} (desde el inicio real del problema)

## SECCIÓN 3 — IMPACTO

### 3.1 Usuarios Afectados

- Número estimado de usuarios impactados: {NÚMERO}
- Porcentaje de la base de usuarios: {PORCENTAJE}%
- Tipo de impacto: {Indisponibilidad total / Degradación / Error intermitente}
- Servicios afectados: {LISTA DE SERVICIOS}

### 3.2 Tiempo de Indisponibilidad

- Inicio del impacto real (detectado en logs/métricas): {TIMESTAMP}
- Fin del impacto: {TIMESTAMP}

- Duración total del impacto: {DURACIÓN}
- Impacto sobre el SLO de disponibilidad: {CONSUMO DE ERROR BUDGET}

### 3.3 Estimativa de Impacto de Negocio

- Agendamientos no realizados (si aplica): {NÚMERO ESTIMADO}
- Impacto financiero estimado: {VALOR o "En análisis"}
- Clientes corporativos afectados: {LISTA o "No aplica"}
- Tickets de soporte recibidos: {NÚMERO}

## SECCIÓN 4 — ANÁLISIS DE CAUSA RAÍZ — 5 WHYS

La metodología "5 Whys" es aplicada para identificar la causa raíz sistémica, más allá del síntoma superficial.

### SÍNTOMA OBSERVADO:

{Descripción del síntoma que desencadenó el incidente}

¿Por qué 1?

→ {Primera causa. Ej: "El servicio Agendio presentó error 503 para 100% de las requisiciones."}

¿Por qué 2?

→ {Segunda causa. Ej: "Porque el pool de conexiones con el RDS PostgreSQL fue agotado."}

¿Por qué 3?

→ {Tercera causa. Ej: "Porque un deploy de la versión 2.4.0 introdujo una query N+1 en el endpoint de agendamiento."}

¿Por qué 4?

→ {Cuarta causa. Ej: "Porque la query N+1 no fue detectada en Code Review ni en las pruebas de integración en Staging."}

¿Por qué 5?

→ {Quinta causa (raíz sistémica). Ej: "Porque no existe un proceso de pruebas de carga automatizado en el pipeline de CI/CD que simule el volumen de producción."}

### CAUSA RAÍZ IDENTIFICADA:

{Declaración formal de la causa raíz sistémica}

## SECCIÓN 5 — LO QUE FUNCIONÓ BIEN

{Lista de elementos del proceso, herramientas o decisiones que funcionaron según lo esperado durante el incidente.}

- {Ej: Las alertas de Datadog detectaron el problema antes que los usuarios reportaran.}
- {Ej: El proceso de rollback fue ejecutado sin errores en menos de 8 minutos.}
- {Ej: La comunicación entre IC, TL y SMEs fue fluida y clara.}

SECCIÓN 6 — LO QUE PUEDE MEJORAR

{Lista de gaps, fricciones o puntos de mejora identificados durante el incidente.}

- {Ej: La alerta de agotamiento del pool de conexiones RDS tardó 8 minutos para dispararse, extendiendo el impacto.}
- {Ej: El runbook de rollback no contemplaba el escenario de migración de base de datos activa.}
- {Ej: Ausencia de pruebas de carga en el pipeline de CI/CD.}

SECCIÓN 7 — ACCIONES CORRECTIVAS

ID	Acción	Tipo	Responsable	Fecha Límite	Status
AC1	{Descripción de la acción correctiva 1}	Prevención	{NOMBRE}	{FECHA}	Pendiente
AC2	{Descripción de la acción correctiva 2}	Detección	{NOMBRE}	{FECHA}	Pendiente
AC3	{Descripción de la acción correctiva 3}	Mitigación	{NOMBRE}	{FECHA}	Pendiente
AC4	{Descripción de la acción correctiva 4}	Proceso	{NOMBRE}	{FECHA}	Pendiente

Tipos de Acción:

- Prevención: Elimina la causa raíz para que el incidente no ocurra de nuevo.
- Detección: Mejora la capacidad de detectar el problema más rápidamente.
- Mitigación: Reduce el tiempo o el impacto de futuras ocurrencias.
- Proceso: Mejora el proceso de respuesta al incidente en sí.

SECCIÓN 8 — LECCIONES APRENDIDAS

{Párrafo o lista con los aprendizajes más importantes que este incidente deja para el equipo de ingeniería de Santo Pegasus.}

SECCIÓN 9 — REFERENCIAS Y EVIDENCIAS

- Link del canal de War Room en Slack: {URL}
- Link del dashboard de Datadog durante el incidente: {URL}
- Link del timeline completo (Google Doc/Confluence): {URL}
- Link del ticket en Jira/PagerDuty: {URL}
- Logs relevantes (extractos o links): {URL}
- Trace IDs relevantes: {IDs}

...  
---

11. PROCESO DE REVISIÓN Y ARCHIVO DEL POST-MORTEM

11.1 Plazos para Entrega del Post-Mortem

Severidad	Plazo para Borrador Inicial	Plazo para Revisión	Plazo para Aprobación Final
SEV-1	48 horas	72 horas	5 días hábiles
SEV-2	72 horas	5 días hábiles	7 días hábiles
SEV-3	5 días hábiles	7 días hábiles	10 días hábiles
SEV-4	7 días hábiles	10 días hábiles	15 días hábiles

11.2 Proceso de Revisión

Paso 1 — Borrador Inicial:

El Technical Lead del incidente redacta el borrador inicial del Post-Mortem, incluyendo el timeline completo y el análisis de causa raíz con la metodología de 5 Whys.

Paso 2 — Revisión Técnica:

El borrador es compartido con todos los participantes del incidente (IC, Communications Lead, SMEs) para validación factual. Cualquier participante puede añadir contexto, corregir informaciones o enriquecer el documento.

Paso 3 — Revisión del Chapter Lead SRE:

El Chapter Lead de SRE revisa el documento con los siguientes criterios:

- ¿La causa raíz está correctamente identificada?

- ¿Las acciones correctivas son realmente accionables y tienen responsables y fechas definidos?
- ¿El tono del documento es blameless?
- ¿Las lecciones aprendidas agregan valor real al conocimiento del equipo?

#### **Paso 4 — Reunión de Revisión del Post-Mortem:**

**Para SEV-1 y SEV-2, es obligatoria la realización de una reunión de revisión con duración máxima de 60 minutos, con la participación de:**

- IC del incidente
- Technical Lead del incidente
- Chapter Lead de SRE
- Al menos 1 engineer de otro equipo (perspectiva externa)
- Opcionalmente: Product Manager del servicio afectado

La reunión tiene como objetivos:

1. Revisar el timeline y garantizar que todos los hechos están correctos.
2. Discutir y validar la causa raíz identificada.
3. Revisar y comprometer a los responsables con las acciones correctivas.
4. Extraer lecciones aprendidas adicionales a través del diálogo colectivo.

#### **Paso 5 — Aprobación y Publicación:**

El Chapter Lead de SRE aprueba formalmente el Post-Mortem. El documento es entonces publicado en el repositorio oficial de Post-Mortems de Santo Pegasus (Confluence / Google Drive / Notion — según el sistema vigente) y el link es compartido en el canal `#incidents-resolved` de Slack.

### **11.3 Política de Archivo**

- Todos los Post-Mortems son archivados indefinidamente en el repositorio oficial.
- El repositorio es indexado por: ID del incidente, fecha, severidad, servicio afectado, causa raíz.
- El repositorio es de acceso read-only para toda la empresa y read-write solo para el Chapter de SRE.
- Trimestralmente, el Chapter Lead de SRE realiza una revisión del repositorio para extraer tendencias y patrones de incidentes recurrentes, alimentando el backlog de mejoras de confiabilidad.

### **11.4 Seguimiento de Acciones Correctivas**

- Todas las acciones correctivas del Post-Mortem son convertidas en tickets formales en Jira y asignadas al sprint correspondiente.
- El Chapter Lead de SRE es responsable por monitorear el cumplimiento de las fechas límite.

- **Acciones correctivas de SEV-1 tienen prioridad P1 en el backlog y no pueden ser postergadas sin aprobación explícita del Chapter Lead.**
- En la reunión de revisión de incidentes (mensual), el status de todas las acciones correctivas en curso es revisado.

---

## 12. MÉTRICAS DE CONFIABILIDAD: SLI, SLO Y SLA

### 12.1 Definiciones

**SLI (Service Level Indicator):** Métrica cuantitativa que mide un aspecto específico del comportamiento del servicio. Es el "termómetro" de la confiabilidad.

**SLO (Service Level Objective):** El objetivo interno de desempeño que Santo Pegasus se compromete a mantener para cada SLI. El SLO define el nivel de calidad esperado.

**SLA (Service Level Agreement):** El acuerdo formal con los clientes que define las garantías de disponibilidad y las consecuencias (compensaciones) en caso de incumplimiento. El SLA es siempre igual o menor que el SLO, para que haya margen de seguridad.

> Principio: SLO es la promesa interna. SLA es la promesa al cliente.

### 12.2 SLIs, SLOs y SLAs por Servicio

#### #### 12.2.1 Servicio Agendio (Agendamiento de Consultas)

SLI	Descripción	SLO (Objetivo Interno)	SLA (Compromiso con Cliente)
Disponibilidad	% de tiempo en que el servicio está operativo	≥ 99.95% mensual	≥ 99.9% mensual
Latencia P99	99º percentil de latencia de respuesta	< 400ms	< 500ms
Latencia P50	Mediana de latencia de respuesta	< 150ms	< 200ms
Tasa de Error	% de requisiciones que retornan 5xx	< 0.05%	< 0.1%
Tasa de Éxito de Agendamiento	% de intentos de agendamiento que finalizan con éxito	≥ 99.9%	≥ 99.5%

#### #### 12.2.2 Servicio de Notificaciones (AWS SES + SQS)

SLI	Descripción	SLO (Objetivo Interno)	SLA (Compromiso con Cliente)
Disponibilidad de Entrega	% de mensajes procesados exitosamente de la cola SQS	≥ 99.9% mensual	≥ 99.5% mensual

Latencia de Entrega de E-mail	Tiempo entre el evento y el envío del e-mail	< 60 segundos P99	< 120 segundos P99
Tasa de Mensaje en DLQ	% de mensajes que van a la Dead Letter Queue	< 0.1%	< 0.5%

#### #### 12.2.3 API Gateway / Servicios de Autenticación

SLI	Descripción	SLO (Objetivo Interno)	SLA (Compromiso con Cliente)
Disponibilidad	% de tiempo operativo del API Gateway	≥ 99.99% mensual	≥ 99.9% mensual
Latencia P99 de Autenticación	Latencia de validación de token JWT	< 100ms	< 200ms
Tasa de Error de Autenticación	% de fallos en la validación de tokens válidos	< 0.01%	< 0.1%

### 12.3 Medición y Reporte de SLIs

- Los SLIs son medidos continuamente por Prometheus y visualizados en dashboards dedicados en Datadog.
- Los dashboards de SLO deben estar disponibles en tiempo real para todos los ingenieros.
- Los reportes mensales de SLO son generados automáticamente y distribuidos al Chapter Lead de SRE y a los Product Managers de los servicios.
- Las violaciones de SLO son documentadas y analizadas en el proceso de revisión mensual de incidentes.

---

## 13. ERROR BUDGET: DEFINICIÓN, CÁLCULO Y POLÍTICA DE AGOTAMIENTO

### 13.1 ¿Qué es el Error Budget?

**El Error Budget (presupuesto de errores) es la cantidad máxima de indisponibilidad o degradación que Santo Pegasus puede "gastar" en un período determinado (normalmente mensual) sin violar el SLO establecido. Es el mecanismo que define el equilibrio entre la velocidad de entrega de nuevas funcionalidades y la necesidad de mantener la estabilidad del sistema.**

El Error Budget transforma la confiabilidad en un recurso gestionable: cuando el budget está disponible, la velocidad de deploy puede ser alta. Cuando el budget se agota, la prioridad se invierte completamente hacia la estabilidad.



## 13.2 Cálculo del Error Budget

### Fórmula Base:

...

Error Budget (%) = 100% - SLO (%)

...

### Para el servicio Agendio (SLO de Disponibilidad = 99.9%):

...

Error Budget Mensual (%) = 100% - 99.9% = 0.1%

Minutos en un mes (30 días) = 30 × 24 × 60 = 43.200 minutos

Error Budget Mensual (minutos) = 43.200 × 0.001 = 43,2 minutos

...

**Esto significa que el servicio Agendio puede estar indisponible por un máximo de 43 minutos por mes antes de que el SLO sea violado.**

### Cálculo del Error Budget consumido en un incidente:

...

Budget Consumido = Duración del Incidente × % de Usuarios Afectados

...

Ejemplo: Un incidente de 20 minutos que afecta al 100% de los usuarios consume 20 minutos del Error Budget. Un incidente de 60 minutos que afecta al 50% de los usuarios consume 30 minutos del Error Budget.

## 13.3 Dashboard de Error Budget

El dashboard de Error Budget en Datadog debe mostrar, en tiempo real:

- Error Budget total del mes (en minutos y porcentaje).
- Error Budget consumido hasta el momento.
- Error Budget restante.
- Tasa de consumo: si el ritmo actual se mantiene, ¿cuándo se agotará el budget?
- Tendencia comparativa con el mes anterior.

## 13.4 Política de Agotamiento del Error Budget

La política de Error Budget de Santo Pegasus establece tres niveles de respuesta según el porcentaje del budget consumido:

Budget Consumido	Estado	Acción Requerida
0% – 50%	Saludable	Operación normal. Deploy de nuevas features autorizado.

51% – 75%	Alerta	Engineering y Product son notificados. Revisión de estabilidad obligatoria antes de nuevos deploys de alto riesgo.
76% – 99%	Crítico	Engineering Lead y Product Lead deben revisar y autorizar explícitamente cada nuevo deploy. Prioridad del equipo se orienta hacia mejoras de estabilidad.
100% (Agotado)	Agotado	Pausa inmediata de deploys de nuevas funcionalidades. El equipo dirige el 100% de su capacidad a acciones de mejora de confiabilidad hasta la recuperación del budget en el próximo ciclo.

### 13.5 Política de Pausa de Features por Agotamiento del Error Budget

Quando el Error Budget de un servicio es completamente agotado, las siguientes medidas entran en vigor de forma automática:

**1. Freeze de deploys de features:** Ningún Pull Request de nueva funcionalidad puede ser mergeado o deployado en producción para el servicio afectado hasta el inicio del próximo ciclo mensual o hasta que se demuestre una mejora estructural en la confiabilidad del servicio.

**2. Priorización obligatoria de confiabilidad:** El 100% de la capacidad de desarrollo del equipo responsable por el servicio es redirigida a:

- Implementación de las acciones correctivas identificadas en Post-Mortems pendientes.
- Mejoras en la cobertura de pruebas (la cobertura mínima de 80% en pruebas unitarias es obligatoria en la etapa de Code Review [Source: Guía Oficial de Ingeniería Back-end.pdf, Page 12]).
- Mejoras en observabilidad, alerting y runbooks de respuesta.
- Refactoring de componentes frágiles identificados como contribuyentes a incidentes recurrentes.

**3. Revisión ejecutiva:** El Chapter Lead de SRE presenta a la dirección técnica un plan de recuperación de confiabilidad con métricas, acciones y plazos definidos.

**4. Excepción de Hotfixes:** Deploys de correcciones críticas de seguridad o de bugs que en sí mismos mejoren la confiabilidad del sistema pueden ser autorizados por el Chapter Lead de SRE de forma individual, incluso durante el freeze.

---

## 14. SIMULACROS DE INCIDENTES — GAMEDAY

### 14.1 ¿Qué es el GameDay?

El GameDay es un ejercicio controlado y planificado en el que el equipo de ingeniería de Santo Pegasus simula un incidente real en un ambiente controlado (preferiblemente Staging, pero con posibilidad de ambientes de producción reducidos cuando aplique) con el objetivo de:

- Validar la eficacia de los runbooks y procedimientos de respuesta.
- Identificar gaps en el proceso de detección y alertas.
- Entrenar a los ingenieros en los roles de IC, Communications Lead y Technical Lead.
- Evaluar el comportamiento del sistema ante fallos inyectados (Chaos Engineering).
- Fortalecer la cultura de preparación ante incidentes y reducir el "pánico" en situaciones reales.

## 14.2 Frecuencia y Planificación

Los GameDays en Santo Pegasus ocurren con frecuencia trimestral (4 ediciones por año), con cada edición enfocándose en un servicio o escenario diferente.

Calendario anual sugerido:

Trimestre	Mes Sugerido	Foco del Simulacro
Q1	Febrero/Marzo	Agendío — Fallo de conexión con RDS PostgreSQL
Q2	Mayo/Junio	Pipeline de Notificaciones — Saturación de cola SQS
Q3	Agosto/Septiembre	API Gateway / Autenticación — Fallo de servicio de tokens JWT
Q4	Noviembre/Diciembre	Simulacro integral multi-servicio — Fallo en cascada

## 14.3 Proceso de Planificación del GameDay

**Etapas 1 — Definición del Escenario (2 semanas antes):**

- El Chapter Lead de SRE y el Technical Lead del servicio en foco definen el escenario de fallo.
- El escenario es documentado de forma confidencial (solo el equipo organizador conoce los detalles).
- Se define el ambiente de ejecución (Staging o producción reducida).
- Se define el "botón de pánico" (cómo abortar el ejercicio en caso de impacto real no deseado).

**Etapas 2 — Preparación (1 semana antes):**

- Los participantes son notificados de que habrá un GameDay, pero no del escenario específico.
- Se revisa si los runbooks y documentación relevante están actualizados.

- Se configura el mecanismo de inyección de fallos (puede ser manual o via herramientas de Chaos Engineering).

### **Etapá 3 — Ejecución del Simulacro:**

- El equipo organizador inyecta el fallo según el escenario planificado.
- Los participantes responden como si fuera un incidente real, siguiendo todos los pasos de este protocolo.
- El equipo organizador observa y registra todas las acciones, decisiones y tiempos de respuesta.
- El organizador puede insertar "complicaciones" adicionales durante el ejercicio para simular escenarios más complejos.

### **Etapá 4 — Debriefing (inmediatamente después):**

- Sesión de debriefing de 90 a 120 minutos con todos los participantes.
- El equipo organizador revela el escenario completo y los fallos inyectados.
- Cada participante comparte su perspectiva: ¿qué fue fácil? ¿qué fue difícil? ¿qué necesitaría para responder mejor?
- Los gaps identificados son documentados en formato similar a un Post-Mortem del GameDay.

## **14.4 Objetivos Medidos en Cada GameDay**

Métrica del GameDay	Objetivo
Tiempo hasta la detección de la alerta	< 5 minutos
Tiempo hasta la declaración del incidente	< 10 minutos
Tiempo hasta la identificación de la causa raíz	Referencia del incidente real más reciente
Tiempo hasta la mitigación	Referencia del incidente real más reciente
Calidad de la comunicación (evaluación 1-5)	≥ 4/5
Seguimiento correcto del protocolo (checklist)	≥ 90% de los pasos

## **14.5 Herramientas de Chaos Engineering**

Para GameDays de nivel avanzado, Santo Pegasus puede adoptar herramientas de Chaos Engineering tales como:

- **AWS Fault Injection Simulator (FIS):** Para inyectar fallos directamente en servicios AWS (RDS, ECS, etc.).
- **Chaos Monkey (Netflix):** Para terminación aleatoria de instancias de contenedores.
- Scripts customizados de inyección de carga en SQS para simular tormentas de mensajes.

## 14.6 Documentación del GameDay

Cada GameDay genera un informe post-ejercicio con:

- Escenario planificado vs. respuesta real del equipo.
- Gaps identificados en el protocolo o en los runbooks.
- Mejoras a implementar antes del próximo GameDay.
- Evaluación de desempeño de los roles (IC, TL, CL).

El informe del GameDay es archivado junto a los Post-Mortems reales en el repositorio de confiabilidad.

---

# 15. DISPOSICIONES FINALES, VIGENCIA Y REVISIÓN DEL PROTOCOLO

## 15.1 Vigencia

Este protocolo entra en vigor a partir de la fecha de su aprobación por el Chapter Lead de SRE y la Dirección de Ingeniería de Santo Pegasus Soluciones, y permanece válido hasta que una versión revisada sea oficialmente publicada y comunicada a todos los equipos.

## 15.2 Política de Revisión

Este documento debe ser revisado en las siguientes circunstancias:

Disparador de Revisión	Plazo para Revisión
Revisión periódica obligatoria	Semestral (cada 6 meses)
Incidente SEV-1 que exponga gaps críticos en el protocolo	30 días después del incidente
Cambio significativo en la arquitectura de producción	Dentro del ciclo de planning del cambio
Adopción de nuevas herramientas de observabilidad o CI/CD	En el momento de la adopción
Solicitud formal de cualquier equipo de ingeniería	A criterio del Chapter Lead de SRE

## 15.3 Proceso de Actualización

1. Cualquier ingeniero de Santo Pegasus puede proponer una alteración en este protocolo abriendo un ticket en Jira con la label `sre-protocol-update`.
2. Las propuestas son revisadas mensualmente por el Chapter Lead de SRE.
3. Las alteraciones significativas deben ser revisadas por al menos 2 engineers Senior o Semi-Senior antes de ser aprobadas.

4. Toda nueva versión del protocolo debe ser comunicada a todos los equipos de ingeniería con al menos 2 semanas de antelación antes de entrar en vigor.
5. El historial de versiones del protocolo es mantenido en el repositorio oficial.

15.4 Responsabilidades de Cumplimiento

Responsable	Obligación
Chapter Lead de SRE	Garantizar la aplicación, actualización y mejora continua de este protocolo
Tech Leads de Equipo	Garantizar que sus equipos conozcan y apliquen el protocolo
Engineers Senior y Pleno	Participar activamente en la rotación de on-call y cumplir todos los roles
Engineers Junior	Conocer el protocolo, participar como observadores en War Rooms y GameDays, y prepararse para asumir roles progresivamente
Dirección de Ingeniería	Apoyar la cultura blameless, garantizar recursos para las acciones correctivas y participar en las revisiones ejecutivas de incidentes críticos

15.5 Contactos de Emergencia

Rol	Canal Principal	Canal de Backup
IC de Turno	PagerDuty	Canal `#incidents` en Slack
Chapter Lead SRE	PagerDuty + WhatsApp	E-mail corporativo
CTO / Director de Ingeniería	WhatsApp	Llamada directa
AWS Support (Plan Business/Enterprise)	Console AWS Support	support.aws.amazon.com

15.6 Historial de Versiones

Versión	Fecha	Autor	Descripción de los Cambios
1.0.0	Junio 2026	Chapter Lead SRE	Versión inicial del protocolo

---

APÉNDICES

Apéndice A — Glosario

Término	Definición
---------	------------

Blameless Culture	Cultura organizacional que foca el análisis de incidentes en el sistema, no en individuos.
Canary Release	Estrategia de deploy en que la nueva versión recibe un porcentaje pequeño del tráfico para validación.
Blue-Green	Estrategia de deploy con dos ambientes idénticos, permitiendo rollback instantáneo.
DLQ	Dead Letter Queue — Cola de mensajes fallidos en AWS SQS.
Error Budget	Tiempo máximo de indisponibilidad permitido por el SLO en un período determinado.
GameDay	Simulacro planificado de incidente para entrenamiento del equipo.
IC (Incident Commander)	Rol responsable de coordinar la respuesta a un incidente.
SLI	Service Level Indicator — Métrica que mide el comportamiento del servicio.
SLO	Service Level Objective — Objetivo interno de desempeño para cada SLI.
SLA	Service Level Agreement — Acuerdo formal de nivel de servicio con los clientes.
SME	Subject Matter Expert — Especialista convocado durante un incidente.
War Room	Canal de comunicación temporal dedicado a la respuesta de un incidente específico.

## Apéndice B — Checklist Rápido de SEV-1 (Para Imprimir / Fijar en el Workspace)

...

### CHECKLIST RÁPIDO — SEV-1 SANTO PEGASUS

- ☐ 1. Reconocer alerta en PagerDuty (< 5 min)
- ☐ 2. Confirmar el incidente en Datadog/Prometheus
- ☐ 3. Declarar el incidente en #incidents
- ☐ 4. IC asigna roles (TL, CL, SMEs)
- ☐ 5. Abrir #sev1-war-room en Slack
- ☐ 6. Abrir documento de timeline
- ☐ 7. CL publica primera actualización en #incidents
- ☐ 8. TL inicia diagnóstico (último deploy? RDS? ECS?)

- ☐ 9. IC aprueba acción de mitigación
- ☐ 10. TL ejecuta mitigación / rollback
- ☐ 11. Monitorear métricas por 10 minutos
- ☐ 12. IC declara resolución
- ☐ 13. CL publica comunicado de resolución
- ☐ 14. IC abre ticket de Post-Mortem (< 48h)

...

---

## **Santo Pegasus Soluciones**

**Departamento: Ingeniería de Software / Chapter de SRE**

**Versión: 1.0.0 | Junio 2026**

**Propietario del Documento: Chapter Lead de SRE**

---

*Este documento debe ser tratado como la "fuente de la verdad" para todos los procedimientos de respuesta a incidentes en Santo Pegasus Soluciones. Cualquier duda, sugerencia o solicitud de alteración debe ser dirigida al Chapter Lead de SRE a través del canal `#sre-chapter` en Slack.*